CPSC 220 Labs

version 1.0

Last generated: August 28, 2016

© 2016 University of Mary Washington. All rights reserved. No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

Table of Contents

CPSC 220 Labs

Labs

You accumulate as many points as possible when completing labs. Labs consist of questions, programming problems, and potentially a programming contest problem.

- Questions - The questions are placed in the overview of the lab. Answering questions accumulates points as described with the questions. Some of the questions have been copied from **David Eck's book, Java Notes** (http://math.hws.edu/javanotes/index.html).
- Programming Problems - a lab has several individual programming problem, each of which is worth a spedified number of points.
- Programming Contest (page 0) - Our class has a website with the Mooshak programming contest. Some labs have a corresponding programming contest problem. In those labs, each team mush submit a solution to the programming contest problem in order to receive the total points earned on the lab. If the programming contest solution successfully solves the problem, team members are awarded additional lab points.

Lab Links

Intro & 1st Programs (page 0)

Expressions, Methods (page 0)

Simple Objects (page 0)

Control Flow (page 0)

Arrays, ArrayLists (page 0)

Stack, Heap, Parameters (page 0)

Exception, Try, File I/O (page 0)

Classes, Objects, ... (page 0)

Graphics (page 0)

Packages, ADTs (page 0)

Recursion, Search, Sort (page 0)

Lab Guidance

- After studying material in a module, you complete a lab to practice the concepts of the module.
- Each module directs you to its corresponding lab.
- Individual labs are accessed via the Labs Sidebar.
- Each lab has Lab Overview section that contains the following.
    - An overview discussion of the lab.
    - The submission requirements of the lab.
    - Possibly a sample lab specification and solution that you can use as a template.
    - A list of questions that you have to answer.
- Labs are where you learn. You are allowed to help team members learn. Help is not restricted except that learning must occur.
    - You accumulate points by completing sections of the labs.
    - You assign yourself points based upon how well you complted each lab section.
    - You must demonstrate you lab to a team mate.
    - Gusty selects the section(s) of the lab for you to demontrate to him.
    - Depending upon your demonstration to Gusty, you may be asked to demonstrate additional lab sections.
    - Depending upon your demonstration to Gusty, your lab points may be increased or decreased.

Steps to Solving Labs

1. Understand the Problem
    - Analyze specification - study the specification as soon as the problem is assigned. This is important for projects, which (a) longer and more difficult than labs and (b) you complete by yourself. You will often discover solutions to problems as you are wandering about campus, but if you have not studied the specification, you cannot solve the problem. Also, Gusty tries to make things clear, but often he does not. Ask questions untuil you understand the specification.
    - Identify inputs and outputs
    - Think about the data
4. Initial Algorithmic Thinking
    - Manually solve small concrete cases - **I cannot over-emphasize this step.** If you cannot create and solve small concrete cases of the problem, you will never discover the underlying algorithm and data strucutes.
        - Get out paper and pencil.
        - Draw out the problem.
        - Solve small concrete cases.
    - Confirm your inputs, outputs, and data that resulted from Understand the Problem step
2. Design – Algorithms, Data Structs, Tests
    - Generalize the small cases into a detailed design with data structures. You may use pseudo code or flow charts for your design.
    - Create test cases
3. Write Java Code
    - Translate your design into Java Code
2. Test and Debug
    - Execute your test cases
    - Debug code when test cases fail

Lab Submissions on Canvas

Learning Objective(s):

- Understand the labs and their corresponding Canvas assignments.
- Understand what is expected to be submitted on Canvas for each lab.
- Understand what is expected to be submitted for the Total Points Lab Assignment.
- Provide a sample Canvas lab submission so that students know what is expected for Canvas submssions.

Lab Submissions on Canvas

The due dates for lab submissions are part of the Canvas lab assignments. The points for each lab must be submitted prior to its due date. Canvas has an assignment submission for each of the following labs.

- Intro and 1st Programs
- Expressions, Methods
- Simple Objects
- Control Flow
- Arrays, ArrayLists
- Classes, Objects, …
- Packages, ADTs
- Graphics
- Recursion
- Searching
- Sorting
- Total Points

Sample Lab Submission

Each lab submission requires the following.

- The UMW Honor Pledge with your name that serves as your signature.
- The name of your teammate who verfied your lab.
- The .java files that comprise your code for the labs.
- You do not have to submit your answers to the questions.
- The total points earned on the lab.
- A list showing the individual labs and the points earned.

The following demonstrates what is expected for submitting your lab completion results on Canvas.

```
I hereby declare upon my word of honor that I have neither give
n nor received unauthorized help on this work. /Gusty Cooper

Verified by: Jerri Anne Cooper

Total Points: 110 points

Individual Labs

* Questions: 20 points
* Codepad Expressions Lab: 10 points
* Split the Bill Lab: 10 points
* Compute Pay Check Lab: 10 points
* Split the Bill and Pay Check Lab: 10 points
* Compute Average Lab: 10 points
* Compute Largest Lab: 10 points
* Compute Temperature Lab: 10 points
* Compute Dice Lab: 10 points
* Use Strings Lab: 10 points
```

Total Points Lab Assignment

The Total Points Lab Assignment is where you submit the total points received on our labs. This is a summation of the points achieved on the individual labs. This must be submited prior the last day of class. The total points are used to determine your lab grade.

Lab Points

Learning Objective(s):

- View the lab points in a Google spreadsheet.

Lab Points

Lab Points (https://docs.google.com/spreadsheets/d/1tnNJB9O56irMDk8PL9s0q08mmol4qG8wYb2mReHcims/edit?usp=sharing) can be viewed in a Google spreadsheet, which shows a list of labs and the points achievable.

Programming Contest

Learning Objective(s):

- Understand the problems that comprise the programming contest, which accompanies the labs.

Programming Contest

The CPSC220 Programming Contest (http://ec2-54-86-10-46.compute-1.amazonaws.com/~mooshak/) is a website with Mooshak, which is a programming contest software package. Usually programming contests have a short period of time to complete the problems. Our programming contest runs the entire semester. Each team has a login. The contest has 14 problems. Problems A through I are associated with specific labs. A team must submit a solution to these problems to get maximum points on the lab. Problems J through N are provided for extra credit points - they are more difficult problems.

Programming Contest Guidance

For each programming contest problem, you should adhere to the following rules.

- You should use a `Scanner` to read the input values.
- Do not prompt for input. The input will be there.
- You should use `System.out` to generate your output.
- The output has to match exactly.
- The Java Math class has a sqrt method.

Problem A - Adding Doubles

Problem A - Adding Doubles

Write a program that reads two doubles from the console and prints their sum.

Input

The input will consist of one line with two doubles separated by a space.

Output

The output will be one line that is the sum of the two doubles.

Sample Input

```
43.5 51.2
```

Sample Output

```
94.7
```

Problem B - Who is Closer

Problem B - Who is Closer

Write a program that determines which of two people are closer to you.

- The people are in a grid that can be viewed as a plane.
- The x-coordinates have a range of -20 to 20.
- The y-coordinates have a range of -20 to 20.
- You are located at position (0,0).
- The two people are somewhere in the grid.
- If both people are the same distance from you, your answer shall be Person 2 is closer

Input

The input is defined as follows.

- One line with four doubles with a space between each value.
- The first pair of numbers are the x and y coordinates of person number 1.
- The second pair of numbers are the x and y coordinates of person number 2.

Output

The output will be one line that is the person that is closest to you

Sample Input - Example 1

```
3 -2 -5 -3
```

Sample Output - Example 1

```
Person 1 is closer
```

Sample Input - Example 2

```
3 3 -2 2
```

Sample Output - Example 2

```
Person 2 is closer
```

Problem C - Counting Letters

Problem C - Counting Letters

Write a program that counts the number of times a specific letter occurs on a line.

Input

The input is defined as follows.

- The first line is the letter that is to be counted.
- The second line is the number of subsequent lines of inputs.
- There will be an additional X lines of input where X is the number on line 2.

The number of subsequent lines of input defined by input line 2 will be greater than or equal to 1. You shall ignore case when counting.

Output

The output will be one line for each of the X input lines where each output line contains the count of characters of the corresponding input line.

Sample Input - Example 1

```
a
3
hat
this
available
```

Sample Output - Example 1

```
1
0
3
```

Sample Input - Example 2

```
s
2
MissiSSippi Rivers
This is my time to Shine.
```

Sample Output - Example 2

```
5
3
```

Problem D - Palindromes

Problem D - Palindromes

Write a program that checks for palindromes and reverses strings.

Input

The input is defined as follows.

- The first line is the number of subsequent lines of input. This number will be greater than or equal to 1.
- The subsequent lines contain text - up to 50 characters.

Output

The output will be one line for each of the input lines with text. The output line will be Palindrome if the input line is a palindrome; otherwise, the output line will be a reverse of the input line. When checking for palindrome, you will ignore case and you will skip punctuation and spaces.

Sample Input - Example 1

```
3
Hello there!
Madam I'm Adam
Rise to vote sir.
```

Sample Output - Example 1

```
!ereht olleH
Palindrome
Palindrome
```

Sample Input - Example 2

```
s
2
A nut for a jar of tuna.
This is my time to Shine.
```

Sample Output - Example 2

```
Palindrome
.enihS ot emit ym si sihT
```

Problem E - Shopping Spree

Problem E - Shopping Spree

Write a program that goes shopping. You receive a credit C at a local store and would like to buy two items. You first walk through the store and create a list L of all available items. From this list you would like to buy two items that add up to the entire value of the credit. The solution you provide will consist of the two integers indicating the positions of the items in your list (smaller number first).

Input

The first line of input gives the number of cases, N. N test cases follow. For each test case there will be:

- One line containing the value C, the amount of credit you have at the store.
- One line containing the value I, the number of items in the store.
- One line containing a space separated list of I integers. Each integer P indicates the price of an item in the store.
- Each test case will have exactly one solution.

Output

For each test case, output one line containing “Case #x: “ followed by the indices of the two items whose price adds up to the store credit. For this problem, the index number is the position of the item - e.g., the second item is index 2. The lower index should be output first.

Limits

```
N <= 10, 5 <= C <= 1000, 1 <= P <= 1000, 3 <= I <= 100
```

Sample Input

```
3
100
3
5 75 25
200
7
150 24 79 50 88 345 3
8
8
2 1 9 4 4 56 90 3
```

Sample Output

```
Case #1 2 3
Case #2 1 4
Case #3 4 5
```

Problem F - Three N Plus One

Problem F - Three N Plus One

Background

Problems in Computer Science are often classified as belonging to a certain class of problems (e.g., NP, Unsolvable, Recursive). In this problem you will be analyzing a property of an algorithm whose classification is not known for all possible inputs.

Consider the following algorithm.

1. Input n
2. Print n
3. If n == 1 then STOP
4. If n is odd then n = 3*n + 1
5. Else n = n/2
6. GOTO 2

Given the input 22, the following sequence of numbers will be printed

```
22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
```

It is conjectured that the algorithm above will terminate (stops after printing 1) for any integer value. Despite the simplicity of the algorithm, it is unknown whether this conjecture is true. It has been verified, however, for very large integers.

Given an input n, the number of numbers printed (including 1) is called the cycle-length of n. In the example above where n is 22, the cycle length is 16.

For any two integers I and J, you shall determine the maximum cycle length and the minimum cycle length over all numbers between (and including) I and J.

Input

The first line of the input gives the number of cases, N. N test cases follow. For each test case there will be one line that contains a pair of integers I and J. All integers will be less that 10,000 and greater than 0.

Output

For each pair of input integers I and J you shall output one line that contains I, J, the minimum cycle length, and the maximum cycle length. The cycle lengths shall be computed for integers between and including I and J. The I and J output must be in the same order in which they appeared in the input. The exact format for each output line is shown in the sample output.

Sample Input

```
4
1 10
100 200
201 210
900 1000
```

Sample Output

```
Range: 1...10 Min: Num: 1, Cycle: 1 Max: Num: 9, Cycle: 20
Range: 100...200 Min: Num: 128, Cycle: 8 Max: Num: 171, Cycle:
125
Range: 201...210 Min: Num: 208, Cycle: 14 Max: Num: 206, Cycl
e: 89
Range: 900...1000 Min: Num: 904, Cycle: 16 Max: Num: 937, Cycl
e: 174
```

The following explains the first output line above.

- The range of numbers is 1 to 10 – Range: 1…10
- The number 1 generates the minimum cycle for the range 1 to 10 and its cycle is 1 – Min: Num: 1, Cycle: 1
- The number 9 generates the maximum cycle for the range 1 to 10 and its cycle is 20 – Max: Num: 9, Cycle: 20

Problem G - Paired Isograms

Problem G - Paired Isograms

Write a program that identifies pair isograms. An isogram is a word without a repeating letter. Example isograms are palindromes, incomputable, and computerizably. A pair isogram is a word where each letter appears exactly twice. Example pair isograms are teammate, intestines, and noon.

Input

The first input line will be a positive integer n, indicating the number of words to be processed. Each of the following n input lines contains a word.
Output Print each input word along with a message indicating whether or not it is pair isograms. Follow the format illustrated in Sample Output. The input word may contain uppercase, lowercase, and punctuation, which you shall ignore in determining whether the word is a pair isogram. For examples, Shannah's and Hello.O'he are a pair isograms.

Sample Input

```
3
teammate
ali
dood
```

Sample Output

```
teammate --- pair isogram
ali --- not pair isogram
dood --- pair isogram
```

Problem H - Permutation Array

Problem H - Permutation Array

Write a program that uses the contents of one array as indices into another array. In many computer problems, the data in an array must be re-arranged in some specified order. One way to rearrange data in an array is to create an index array to point out the position of the elements in the new array. The index array (or permutation array) specifies a permutation of the second array (or data array).

Input

The first line of input gives the number of cases, N. N test cases follow. For each test case there will be three lines of input specified as follows.

- One line containing the value I, the number of items in the arrays on the next two lines.
- One line containing the permutation array. This line is a space separated list of I integers. Each integer will be an index into the array on the next line.
- One line containing the data array. This line is a space separated list of I integers.

Output

For each test case, output one line that contains I+2 integers, which are the following.

- The largest value in the data array.
- The smallest value in the data array.
- A space separated list of I integers that is a reordered list of the data array. The data array is reordered as defined by the permutation array.

Sample Input

```
3
3
3 1 2
32 54 -2
5
1 4 5 2 3
150 24 79 50 88
2
2 1
200 101
```

Sample Output

```
54 -2 -2 32 54
150 24 150 50 88 24 79
200 101 101 200
```

Problem I - Stack Machine

Problem I - Stack Machine

Write a program that uses a stack to add, subtract multiply, and divide. Many virtual machines, like the JVM, are based on the notion of a run-time stack to hold operands and operators. This is like a stack of plates where operands/operators may only be added to or removed from the top. A program in this stack-language looks like the following.

```
PUSH 1
PUSH 2
ADD
RESULT
=> 3 (output from stack)
```

To interpret this code, consider the following.

- PUSH 1 pushes a 1 onto the stack and PUSH 2 pushes a 2 onto the stack, which means the stack has two elements and looks like the following.

```
2  (top of stack)
1
```

- ADD removes the top 2 values on the stack, adds them and pushes the result (so the 2 and 1 would be replaced by 3).
- RESULT is a special operation that will display the current value at the top of the stack.

Input

The input will be N lines of stack code where the Nth line contains the letter R.

Output

There will be one line of output, which is the value computed by the stack machine program.

Sample Input - Example 1

```
P 1
P 2
A
P 3
M
P 1
S
P 2
D
R
```

Sample Output - Example 1

```
=> 4
```

Sample Input - Example 2

```
P 1
P 2
P 3
P 4
P 5
M
A
P 6
S
A
A
P 10
D
R
```

Sample Output - Example 2

```
=> 2
```

Problem J - Sum Sequence

Problem J - Sum Sequence

Write a program that computes the nth value in the following sequence.

```
Position in sequence: 0 1 2 3 4  5  6  7  8  9 10 11 12
Value to be computed: 1 1 2 4 8 16 23 28 38 49 62 70 77
```

Input

The first line of input gives the number of cases, N. N test cases follow. For each test case there will be one line of input where each line contains a position in the sequence.

Output

The output will be N lines of output where each line is the sequence value.

Sample Input

```
3
10
6
3
```

Sample Output

```
62
23
4
```

Problem K - Base 6

Problem K - Base 6

The Island of Calduum has a tribe of unusual creatures - Gabrexheons. The Gabrexheons have 2 fingers and an opposable thumb on each hand. The Gabrexheon numbering system is base 6, and they use a unique set of symbols for digits, but anthropologists have been able to represent them as follows.

- % represents 0
-) represents 1
- ~ represents 2

- @ represents 3
- ? represents 4
- \ represents 5
- $ represents -1

Examples of interpreting Gabrexheon numbers are the following. Note: six-squared is represented as 6**2.

-)@% is $1*6^2 + 3*6 + 0 = 36 + 18 + 0 = 54$
- ?$~~ is $4*6^3 + (-1)*6^2 + 2*6 + 2 = 864 -36 + 12 + 2 = 842$
- $~~ is $(-1)*6^2 + 2*6 + 2 = -36 + 12 + 2 = -22$
- $)) is $(-1)*6^2 + 1*6 + 1 = -36 + 6 + 1 = -29$
- $%$ is $(-1)*6^2 + 0*6 + -1 = -36 + 0 - 1 = -37$
- \$@ is $5*6^3 + 5*6^2 + 0*6 + 3 = 1080 + 180 - 6 + 3 = 12570$

You have to convert Gabrexheon numbers to base 10 numbers.

Input

The first line of input gives the number of cases, N. N test cases follow. For each test case there will be one line of input that is a Gabrexheon numbers. Each number consists of a sequence of 1 to 10 Gabrexheon digits.

Output

Output will be the corresponding decimal numbers, one per line.

Sample Input

```
4
)@%
?$~~
$~~
%
```

Sample Output

```
54
842
-22
0
```

Problem L - Can You Dig It

Problem L - Can you Dig It?

Note: This problem is a duplicate from the UMW CPSC Fall 2013 Programming Contest created by Ian Finlayson.

The Pewter Museum of Science in Kanto is holding a treasure hunt for young Pokemon trainers. The hunt consists of an 8x8 plot of land. In one of the sections, is buried a Pokemon fossil, a rare prize.

In each of the other 63 sections, is a piece of paper with a 2-digit number written on it. This number is a clue directing the trainer to the next section of land to search. The tens digit indicates the row number, and the ones digit indicates the column number.

Participants in the treasure hunt start in the first section (row 1, column 1) and follow the clues until they either find the fossil, or go back to a section of land they have already searched. Your goal is to write a program which, given what's buried in each section of land, will determine whether or not the participants will find the fossil.

Input

Input will consist of 8 lines. Each line corresponds to a row in the plot of land, and contains 8 entries. Each entry is either a two-digit number indicating the cell of the next clue, or the value 0 which indicates that the cell contains the fossil.

You can assume that no clue will refer to a non-existent cell.

Output

Your program should output output the cell numbers that are searched at each step. If the participant finds the fossil, you should end by printing “Found the fossil!”. If the participant is sent back to search a section that they have already searched, you should print “Fossil not found!”.

Sample Input

```
31 21 66 12 52 87 64 44
11 13 78 17 44 51 36 33
76 44 12 54 84 71 12 33
24 53 15 68 58 0 14 88
22 44 61 11 87 0 28 64
88 22 43 43 54 17 42 81
56 23 0 11 17 24 56 71
61 19 21 82 39 0 77 27
```

Sample Output

```
11
31
76
24
17
64
43
15
52
44
68
81
61
88
27
36
71
56
Found the fossil!
```

Problem M - I Choose Who?

Problem M - I Choose Who?

Note: This problem is a duplicate from the UMW CPSC Fall 2013 Programming Contest created by Ian Finlayson.

During a Pokemon battle, trainers try to send out Pokemon who will beat their opponent's Pokemon. Which Pokemon to send against another can be a difficult decision. Trainers primarily base this on the level and type of the Pokemon involved.

A level is a number in the range of 1 - 100 indicating how strong and experienced the Pokemon is.

There are different types of Pokemon, and some types are have advantages in battle over other types. The following table lists seven common types of Pokemon along with estimates on how strong they are against other types.

	Normal	**Fire**	**Water**	**Grass**	**Electric**	**Fighting**	**Psychic**
Normal						- 5	- 5
Fire			/ 2	* 2	+ 10		
Water		* 2		/ 2	/ 2	+ 5	
Grass		/ 2	* 2				
Electric		- 10	* 2			/ 2	
Fighting	+ 5		- 5		* 2		/ 3
Psychic	+ 5					* 3	

The rows indicate the type of your Pokemon, while the columns indicate the type of your opponent's. The entries in the table indicate a modifier on the levels of the Pokemon involved.

For example, if you have a Fire type Pokemon, while your opponent has a Water type, you will have a "* 2" modifier. So, for example, a level 10 Fire Pokemon would be equivalent to a level 20 Water Pokemon. Likewise, a level 35 Psychic Pokemon would be equivalent to a level 40 Normal Pokemon.

If there is no entry in the table, it means the corresponding Pokemon have no advantage or disadvantage against each other.

Your goal is to write a program to help trainers choose which Pokemon to send into battle based on the level and type of the opponent, and the level and type of the Pokemon the trainer has available to them.

The strategy your program should take is to send out a Pokemon with a type-adjusted level that is greater than the opponent. If there are multiple such Pokemon, you should send the one with the lowest actual level (to keep your strongest Pokemon as long as possible). If there are no Pokemon with a type-adjusted level higher than the opponent, you should send the Pokemon with the lowest actual level (to keep your strongest Pokemon as long as possible).

Input

Input will consist of a list of 7 Pokemon. The first Pokemon given will be the opponent. The remaining 6 Pokemon will be the ones in the trainer's party that you have to choose from.

Each Pokemon will have a name, level, and type.

Output

Output will consist of the phrase "<Name>, I choose you!", where <Name> is replaced with the name of the Pokemon being chosen.

Sample Input

```
Growlithe Fire 27
Meowth Normal 30
Magikarp Water 3
Pidgey Normal 14
Quilladin Grass 52
Squirtle Water 15
Electrike Electric 24
```

Sample Output

```
Squirtle, I choose you!
```

Problem N - Lucky Numbers

Problem N - Lucky Numbers

Note: This problem is a duplicate from the UMW CPSC Fall 2013 Programming Contest created by Ian Finlayson.

The Poke Center engraves serial numbers onto each of their Pokeballs. Each of these serial numbers has an even number of digits, N. A Pokeball is considered lucky if the sum of the first N/2 digits is equal to the sum of the last N/2 digits. For example, A Pokeball with the serial number “068437” would be considered lucky since 0 + 6 + 8 = 4 + 3 + 7. Some suggest using a lucky Pokeball is the only way to catch a Mewtwo, not that that’s relevant to the problem.

Your task is to count the number of lucky Pokeballs in the general case. As input, you’ll get two numbers: the number of digits that make up the Pokeball’s serial number (a positive even number) and the base of the ball’s serial number (a number between 2 and 10). As output, you need to print the count of lucky Pokeballs (in decimal) that are possible with the given number of digits in the given base.

Input

```
Input consist of two lines.
The first line contains the number of digits used in the seria
l number.
The second line contains the base of the serial number.
```

Output

Your output should consist of a single integer: the number of lucky Pokeballs possible given the serial number and base.

Sample Input

```
6
2
```

Sample Output

```
20
```

First Java Programs Labs

Learning Objective(s): First Java Programs Lab Overview

First Java Programming Lab

The First Java Programming Lab is your first chance to solve problems with Java. The problems in this lab are not too difficult

Programming Contest

To receive maximum points on this lab, submit a solution to Programming Contest (page 0) Problem A - Adding Doubles.

Submission Requirements

Follow guidance in Sample Lab Submission (page 0) for submitting your lab points.

Install Java and BlueJ

Points: Install Java and BlueJ for 10 points.

1. Download and Install the Java SDK - see Java Downloads (page 0).
2. Download and Install the BlueJ IDE - see Java Downloads (page 0).

Questions

Points: Answer these questions for 20 points. You have to handwrite or type the answers to get the points. For each question that you do not answer, subtract one point. If you do not answer any questions, you get negative points.

1. What is a compiler?
2. What is an interpreter?
3. Is Java compiled or interpreted or both? Explain your answer.
4. What is a CPU and what role does it serve in a computer?

5. What is RAM?
6. What is Secondary Storage?
7. What is the difference between a high-level language and a low-level (or machine language)?
8. Is Java high-level or low-level language?
9. True/False: Java is an expensive language?
10. Explain the phrase, “Java is platform-independent.”
11. Why do we normally use base 10?
12. Why does a computer use binary?
13. How many values does a byte hold?
14. How many values does a short hold?
15. How many values does an int hold?
16. True/False: A double and a long can hold the same number of values.
17. True/False: Memory is a sequence of bytes.
18. What is the binary equivalent of 10 decimal?
19. What is the decimal equivalent of 0b1010 binary?
20. True/False: In Java, a variable has a name, a value, a type, and associated memory locations.
21. What are the steps of a software development process?
22. What three steps of software development will this class focus on?
23. Define an algorithm. Is a computer program an algorithm? Can you have an algorithm that is not a computer program? Is the following an algorithm?

    a. Roll a six sided die

    b. If the number rolled is between 1 and 6 repeat step a.
24. Define a data type. Compare a Java byte and a Java short using your definition of a data type. Explain a Java boolean type in terms of your definition.
25. Explain syntax and semantics of a programming language. Give an example of a syntax error and a semantic error.

First Program Lab

Learning Objective(s):

- Understand how to use BlueJ to write your first Java program.

First Program

Create Your First Java Program – 10 Points

Points Complete this entire section for 10 points.

Use BlueJ to write a Java program that does the following.

1. Create a classic Hello World program. You should create a `HelloWorld` class that is in a file named `HelloWorld.java` . In Java the filename matches the class name. BlueJ will create some template code for your `HelloWorld class` that has placeholders for instance variables, constructors, and methods. Your HelloWorld class will not have instance variables and its only method will be a `public static void main()` that calls `System.out.println` . You can delete unnecessary code. We will study instance variables, constructors, and methods in Simple Objects (page 0). The following is an example `HelloWorld class` that contains a `main()` method that prints to the console.

```
public class HelloWorld {
   public static void main(String[] args) {
      System.out.println("Hello World, I am a Java progr
ammer.");
   }
}
```

2. Use BlueJ to run your `HelloWorld` program. You will see output in the terminal window. Your program uses the System class, which has a `static` field `out` , which has the `println` method. The `System class` is available to use without having to `import` it.
3. Congratulations on completing your first Java program.

Second Program Lab

Learning Objective(s):

- Understand how to use terminal input and output in your Java program.

Second Program Lab

Java Input and Output

Points: Complete this entire section for 10 points.

Use BlueJ to write a Java program that reads numbers from the terminal window, performs some arithmetic operation, and displays the results.

1. The following is a Java program that reads two integers and prints their sum. This program is more complex that your Hello World program because it uses the `Scanner class` , which is in the `java.util` package. For this week, you should just mimic the mechanics of creating and using a `Scanner` object for reading input. You will learn the details of defining classes and constructing objects in Simple Objects (page 0). `Scanner` objects have several methods for reading input. The example below uses the `nextInt` method. You should use the Oracle Java documentation to discover other Scanner object methods. For examples, `nextDouble` and `nextLine` . We will use many of the `Scanner` methods during this course.

```
import java.util.Scanner;
public class SimpleIO {
   public static void main(String[] args) {
      Scanner in = new Scanner(System.in);
      System.out.print("Enter an integer: ");
      int a = in.nextInt();
      System.out.print("Enter an integer: ");
      int b = in.nextInt();
      System.out.print("The sum of " + a + " and " + b
+ " is ");
      System.out.println(a+b);
   }
}
```

2. Create the above program and execute it.
3. Update the above program so that it does the following.
    - a. Reads two doubles and computes the sum as double.
    - b. Reads two bytes and computes the sum as a byte. You should enter a number that is too large for a byte to see what Java does.

Primitive Types Lab

Learning Objective(s):

- Understand Java primitive types from a programming perspective.

Java Primitive Types Lab

Points: Complete this entire section for 20 points. To get the full 20 points, you have to add your own code to the sample provided.

1. For this section you can continue adding code to your public static void main() function.
2. Declare at least one variable of every primitive Java type.
3. Assign a literal constant to each of your declared variables.
4. Create assignment statements for at least three variables of different primitive types that require casting. The following is an example

```
double d = 2.3;
float f = 1.1F;
f = (float) d;
```

5. Create assignment statements for at least three variables that cast literals that are too large for the variable. The following is an example. byte b = (byte) 200;
6. Your program shall print all of your variables to the standard output. The following is an example. System.out.println(b);
7. Use System.out.print() and System.out.println() so that you understand the difference between them.
8. Use System.out.println() to print numbers as a number and numbers as a string. Observe the difference.

   a. System.out.println(3 + 4 + 5);

   b. System.out.println("3 + 4 + 5");
9. Use System.out.println() to print the following and observe.

a. “Double ” + 3.1415

b. “Int “ + 45

10. The following is some sample code, but you should experiment with your own variations of these samples.

```
public static void main(String[] args) {
    String g = "Gusty";
    String c = "Cooper";
    byte b1 = (byte)127;
    byte b2 = (byte)128;
    short s1 = (short)65535;
    short s2 = (short)65536;
    int i = 32000;
    long l = 32000;
    System.out.println("Hello world.");
    System.out.println("Hello "+g);
    String str128_zeroes = String.format("%32s",
         Integer.toBinaryString(128)).replace(" ", "0");
    String str128_spaces = String.format("%32s",
         Integer.toBinaryString(128));

    System.out.println(" (int)128 in Binary (leading spaces)"
+ str128_spaces);
    System.out.println(" (int)128 in Binary (leading zeroes)"
+ str128_zeroes);
    System.out.println("(byte)128 in Binary " + Integer.toBinar
yString((byte)128));
    System.out.println("     -128 in Binary "+ Integer.toBinary
String(-128));
    System.out.println("b1: "+b1+" b2: "+b2);
    System.out.print("s1: "+s1+" s2: "+s2);
    System.out.print("i and l:"+i+" ");
    System.out.print(l+" ");
    System.out.println();
    System.out.println("(byte) 2**32: " +
                        (byte)Math.pow((double)2,(double)32)+"
");
    System.out.println("(short) 2**32: " +
                        (short)Math.pow((double)2,(double)3
2)+" ");
    System.out.println("(int) 2**32: " +
                        (int)Math.pow((double)2,(double)32)+"
");
    System.out.println("(long) 2**32: " +
                        (long)Math.pow((double)2,(double)32)+"
");
    long two_32 = (long)Math.pow((double)2,(double)32);
    System.out.print("(long) 2**32 as binary" +
                             Long.toBinaryString(two_32));
    System.out.println("");
    System.out.print(1+" ");
    System.out.print(2+" ");
```

```
    System.out.print(3+" ");
    System.out.println(100);
    System.out.println("Gusty "+c);
    System.out.println();
    boolean b = 3 < 5;
    System.out.println("b: "+b);
    System.out.println(g);
    char a = ‘a’;
    System.out.println(a);
}
```

Java Expression Labs

Learning Objective(s): The learning objectives for the entire Java Expression Labs.

- Practice expressions using the BlueJ Codepad.
- Practice writing a methods and main
- Practice using System.out and Scanner.
- Practice creating a class that has methods with the same name.

Java Expression Labs Overview

All of the problems in Java Expression Labs can be solved in a Java class that has `main``, Scanner for input, ``System.out` for output, helper methods with parameters that you write, and expressions. A few of the problems require a conditional expression.

Programming Contest

To receive maximum points on this lab, submit a solution to Programming Contest (page 0) Problem B - Who is Closer.

Submission Requirements

Follow guidance in Sample Lab Submission (page 0) for submitting your lab points.

Sample Lab

This section provides a sample lab specication and a solution. You can follow this sample in creating your solutions to the various labs in Java Expression Labs.

Sample Lab Specification: Add Double

Write a Java `AddDouble` class that does the following.

`public static double addDouble(double x, double y)` - returns the sum of `x` and `y`.

`public static void main(String[] args)` - uses a `Scanner` for input and `System.out.println` for output to accomplish the following.

```
Enter a number: 63.2
Enter a number: -3.2
63.2 + -3.2 = 60.0
```

Sample Lab Solution: Add Double

The solution to this problem is the following code.

```
import java.util.Scanner;

public class AddDouble {

   public static double addDouble(double x, double y) {
      return x + y;
   }

   public static void main(String[] args) {
      Scanner in = new Scanner(System.in);
      System.out.println("Enter a number: ");
      double x = in.nextDouble();
      System.out.println("Enter a number: ");
      double y = in.nextDouble();
      System.out.println(x + " + " + y + " + " + (x+y));
   }
}
```

Questions

Points: Answer these questions for 20 points. You have to handwrite or type the answers to get the points. For each question that you do not answer, subtract one point. If you do not answer any questions, you get negative points.

1. See Java Expressions in Codepad (page 0) for questions on evaluating expressions. You should understand how to evaluate Java expressions so that you can compute in your mind the same answer as Java.
2. Explain an assignment statement. Be sure to include a description of both sides of the assignment operator (=).
3. Write the truth tables for `&&`, `||`, and `!`.

4. Write the truth table for `P && Q || R`.
5. What is meant by precedence of operators? Write the Java Operator Precedence.
6. True/False: In Java, the operators = and == are used interchangeably (i.e., they mean the same thing.)
7. True/False: The first Java assignment statement is a shorthand version of the second.

```
points += 2;
points = points + 2;
```

8. Does the following code have a problem? Y/N ___, if so, explain how to remove.

```
double gravity = 9.8;
```

9. Does the following code have a problem? Y/N ___, if so, explain how to remove.

```
int pie = 3.14159;
```

10. Put the following three statements in the BlueJ Code Pad, observe the answers, and provide an explanation.

```
2 + 3 + “test”
“test” + 2 + 3
“test” + 2 * 3
```

11. What is an integrated development environment?
12. True/False: If I have two Java `String` s, `s1` and `s2`, the following is how to compare them for equality. If this is not the correct way, describe the correct way.

```
s1 == s2
```

13. Provide the meaning for the following Java operators.

    `java ++ && !=`

14. Is the following code sequence legal? If so, what are the values of `a`, `b`, and `c`? Use the BlueJ Code Pad to type these in if needed. Do you think this is good programming style?

```
int a = 5;
int c = 10;
boolean b  = a + c < (c = 20)
```

15. True/False: Java does not have a conditional operator.

16. True/False: The Java shortcut assignment statement only works for the addition operator as follows.

```
points += 2;
```

17. True/False: If I have two Java `doubles`, `d1` and `d2`, the following is how to compare them for equality. If this is not the correct way, describe the correct way.

```
d1 == d2
```

18. If I have two Java variables defined as follows. Provide the code that allows me to assign `myInt` to `myShort`.

```
int myInt = 25;
short myShort = 0;
```

19. Provide two assignment statements that assign the variable `randomNumber` to a random number between 1 and 6. Use `Math.random()` for one assignment statement and a `java.util.Random` object for the other assignment statement.

20. Does the following code have a problem? Y/N ___, if so, explain how to remove.

```
int pie = 3; // crude approximation of pie
double dPie = pie + .14159
```

21. Does the following code have a problem? Y/N ___, if so, explain how to remove.

```
double pie = 3.14159; // approximation of pie
int iPie = pie - .14159
```

22. What does it mean to say Java short circuits the Boolean operators `&&` and `||` ? Provide examples of how this short circuit works.
23. We know that Java implements a `short` as 2 bytes of memory. We also know that we can represent 65536 numbers in 16 bits. The Java wrapper class `Short` shows the largest value for a short as follows. Explain this difference.

```
int maxShort = Short.MAX_VALUE;  // maxShort is 32767
```

Codepad Expressions Lab

Learning Objective(s):

- Practice expressions using the BlueJ Codepad.

Create Java Expressions in BlueJ Codepad

Points: Understand this entire section for 10 points. If you do not understand Java expressions, this section provides you with the opportunity to practice in the BlueJ Codepad. The BlueJ Codepad is a nice tool that I use to experiment with Java features. If you understand Java expressions, then proceed to the next section.

Expressions are a central component of programming. You will use expressions on the right-hand side of an assignment statement, controlling if statements, controlling switch statements, controlling loop statements, as actual parameters, and as the return expression. You have to master expressions to be a good programmer. For the most part, expressions are intuitive; however, mixing types can be tricky, Boolean expressions require careful thought, and Java has several fancy operators such as ++, +=, *=, and ? :. The more you practice creating expressions, the better programmer you will be.

Use the BlueJ Codepad window to create arithmetic expressions so that you understand how operators and types interact. When creating statements in the BlueJ Codepad, the semicolon, which terminates Java statements, can be inserted or omitted. When a statement in the BlueJ Codepad does not have a semicolon the BlueJ Codepad will display the results. Consider the following several example statements as entered into the BlueJ Codepad with and without semicolons. The font colors are indicative of the font colors in BlueJ Codepad.

```
double x = Math.PI;
int ten = 10;
x
3.141592653589793   (double)
ten += 1
11   (int)
double y;
Note: Codepad variables are automatically initialized
 in the same way as instance fields.
y
0.0   (double)
y = x + 10
13.141592653589793   (double)
byte b = 300;
Error: incompatible types: possible lossy conversion from int t
o byte
byte b = 128;
Error: incompatible types: possible lossy conversion from int t
o byte
byte b = 127;  // finally
byte bb = (byte)300;
bb
44   (byte)
bb = (byte)255
-1   (byte)
boolean myBool = 5 < 6;
myBool
true   (boolean)
double d = 5.0;
double dd = 10.0;
double e = d < 10 ? 20.0 : 25.0;
e
20.0   (double)
```

I have provided a list of expressions that you should enter into the BlueJ Codepad, but you should experiment with your own expressions. Be sure to notice which of the following is terminated with a semicolon and enter them as such. I will ask you to evaluate expressions on tests. The following are words from David Eck's book describing Java operators.

Here is a listing of the operators discussed in this section, listed in order from highest precedence (evaluated first) to lowest precedence (evaluated last):

Description	Operators
Unary	`++ -- ! unary - unary + type-cast`

Description	Operators
Multiply Divide	`* / %`
Add Subtract	`+ -`
Relational	`< > <= >=`
Equality	`== !=`
Boolean and	`&&`
Boolean or	`||`
Conditional	`?:`
Assignment	`= += -= *= /= %=`

Operators on the same line have the same precedence. When operators of the same precedence are used in an expression without parentheses, unary operators and assignment operators are evaluated right-to-left, while the remaining operators are evaluated left-to-right. You are most likely familiar with left-to-right evaluation. For example, `x * y / z` is evaluated as `(x * y) / z`. For right-to-left evaluation consider multiple assignments. `i = j = k` is evaluated as `i = (j = k)`.

The following are examples of expression evaluation questions I will ask on tests.

```
int i = 24;
double x = 1.123;
int j = i / 5;              // What is the value of j?
double d = i / 5;           // What is the value of d?
d = i / 5.0;                // What is the value of d?
boolean b = true && false;  // What is the value of b?
boolean c = b || d < 5;     // What is the value of c?
c = d < 5;                  // What is the value of c?
i = d > 5 ? 20 : 21;        // What is the value of d?
double y = 4+5 * 6;         // What is the value of y
i = y - 3;                  // What is the value of i?
                            // Maybe this statement is illega
l.
d = Math.sqrt(25) * 5;      // What is the value of d?
```

As you work through the above and following expressions, you should practice evaluating them so that you know the answers and are prepared for our tests. Be sure to create expressions that use all of the Java operators.

```
int oneMillion = 1000000;
int mystery = oneMillion * oneMillion; // -727379968
oneMillion // 1000000
mystery // -727379968
double unitPrice = 4.35;
double totalPrice = 100 * unitPrice; // 434.999999999999994
oneMillion++ // post increment, displays 1000000
oneMillion   // 1000001
++oneMillion // pre increment, displays 1000002
oneMillion   // 1000002
oneMillion-- // post decrement, displays 1000002
oneMillion   // 1000001
--oneMillion // pre decrement, displays 1000000
oneMillion += 1 // 1000001
oneMillion -= 1 // 1000000
oneMillion*oneMillion  // -727379968
double d = 3;
d
int i = d; // incompatible types
int i = (int) d;
i
i = i--
i
i = --i
i
int j = 3;
i = j = i+10 // a legal but confusing statement
i
j
boolean b = 5 < 6;
b
false && b
false || b
!b
```

Split the Bill Lab

Learning Objective(s):

- Practice writing a method and main - Split the Bill.
- Practice using System.out and Scanner.

Create a Java Program to Split the Bill

Points: Complete this entire section for 10 points.

When friends dine together in restaurants, they often split the bill. For example, if 5 friends are dining where the bill is $50 and we are giving a 20% tip, then the total is $60 divided by 5 people, which means each person pays $12.

Write a Java `SplitTheBill` class that has two public static methods: `main` and `pay`. Both `main()` and `pay()` will be `public static` methods since we will not create objects of type `SplitTheBill`.

`public static double pay(double bill, double tip, int numOfPeople)` – the 3 formal arguments are the total of the bill, the percent tip we want to give, and the total number of people in the group. The pay method computes and returns the amount each person in the group should pay. The following is an example call.

```
double amount = pay(50, .20, 5) ; // amount is 12.0
```

`public static void main(String[] args)` – This is your program's main method that calls your pay method to test that it is correct. Use `System.out.print`, `System.out.println`, and a `Scanner` object to create the dialog for running your program. The following is an example dialog.

```
Enter restaurant bill: 20
Enter tip (.2 for 20%): .2
Enter number of people dining: 4
Price per person is 6.0
```

Compute Pay Check Lab

Learning Objective(s):

- Practice writing a method and main - Compute Pay Check.
- Practice using System.out and Scanner.

Create a Java Program to Compute Paycheck

Points: Complete this entire section for 10 points.

For some jobs, when you work over 40 hours in one week you get what is called "time and a half" meaning the employee gets paid 1.5 times their usual rate for any hours over 40. For example, if a person gets paid $10 per hours and works 50 hours in a week, then the person gets paid $10 per hour for the first 40 hours and $15 per hour for 10 hours for a total of $550.

Write a Java `ComputePayCheck` class that has two public static methods: `main` and `pay`. Both `main` and `pay` will be `public static` functions.

`public static double pay(int hours, double hourlyRate)` – the two formal arguments are the number of hours worked and the hourly rate. The `pay` method computes and returns the total pay for that week. You shall use the Java conditional operators ?: in implementing pay. The following is an example statement that computes the number of overtime hours.

```
int otHours = (hours > 40) ? hours-40 : 0;
```

The following are example calls to your pay method.

```
double weeksWages = pay(10, 0);  // weeksWages is 0
double weeksWages = pay(36, 10); // weeksWages is  360
double weeksWages = pay(50, 10); // weeksWages is  550
```

`public static void main(String[] args)` – This is your program's `main` method that calls your `pay` method to test that it is correct. Use `System.out.print`, `System.out.println`, and a `Scanner` object to create the dialog for running your program. The following is an example dialog.

```
Enter hours worked: 50
Enter hourly rate: 10
Week's salary is 550.0
```

Split the Bill and Pay Check Lab

Learning Objective(s):

- Practice creating a class that has methods with the same name.
- Practice using System.out and Scanner.

Create a Java Program to Compute Paycheck and Split the Bill

Points: Complete this entire section for 10 points.

Split the Bill (page 0) and Compute Pay Check (page 0) each created a `pay` method. The formal parameters for these two `pay` methods are different, which allows us to put both `pay` methods in the same class.

Write a Java `TwoPays` class that has three public static methods: `main` , `pay` , and `pay` . All three methods will be `public static` functions. Yo have to copy the code from Split the Bill (page 0) and Compute Pay Check (page 0) into `TwoPays` .

`public static double pay(double bill, double tip, int numOfPeople)` – the 3 formal arguments are the total of the bill, the percent tip we want to give, and the total number of people in the group. The pay method computes and returns the amount each person in the group should pay. The following is an example call.

`public static double pay(int hours, double hourlyRate)` – the two formal arguments are the number of hours worked and the hourly rate. The `pay` method computes and returns the total pay for that week. You shall use the Java conditional operators ?: in implementing pay. The following is an example statement that computes the number of overtime hours.

`public static void main(String[] args)` – This is your program's `main` method that calls your `pay` methods to test them. Use `System.out.print` , `System.out.println` , and a `Scanner` object to create the dialog for running your program. Combine the dialogs from Split the Bill (page 0) and Compute Pay Check (page 0).

Compute Average Lab

Learning Objective(s):

- Practice writing a method and main - Compute Average
- Practice using System.out and Scanner.

Create a Java Method that Returns the Average

Points: Complete this entire section for 10 points.

Write a Java `Math1 class` that has two `public static method` s: `main` and `average` .

`public static double average(double x1,double x2,double x3,double x4)` – returns the average of four `double` s. The following is an example call

```
double d = average(1.0, 3, 4.0, 8); // d is 4.0
```

`public static void main(String[] args)` – This is your program's `main` method that calls your `average` method to test that it is correct. Use `System.out.print` , `System.out.println` , and a `Scanner` object to create the dialog for running your program. The following is an example dialog.

```
Enter number 1: 4
Enter number 2: 5
Enter number 3: 6
Enter number 4: 7
Average is 5.5
```

You will reuse your `Average` code in Simple Objects - MyMath Type Lab (page 0).

Compute Largest Lab

Learning Objective(s):

- Practice writing a method and main - Compute Largest
- Practice using conditional expressions.
- Practice using System.out and Scanner.

Create a Java Method to Return the Largest

Points: Complete this entire section for 10 points.

Write a Java `Math2 class` that has two `public static method` s: `main` and largest.

`public static double largest(double x1, double x2)` – returns the largest of `x1` and `x2` . You shall use the Java conditional operators `?:` in implementing largest. The following is an example statement that computes the larger of two numbers.

```
double largest = (x1 > x2) ? x1 : x2;
```

The following are example calls to your largest method.

```
double d = largest(1.0, 100); // d is 100.0
```

`public static void main(String[] args)` – This is your program's `main` method that calls your `largest` method to test that it is correct. Use `System.out.print` , `System.out.println` , and a `Scanner` object to create the dialog for running your program. The following is an example dialog.

```
Enter number 1: 40
Enter number 2: 1.23
Largest is 40.0
```

Bonus Points: Complete this for an additional 5 points.

`public static double largest(double x1, double x2, double x3)` – returns the largest of `x1`, `x2`, and `x3`. You shall use the Java conditional operators `?:` in implementing largest.

You will reuse your `Largest` code in Simple Objects - MyMath Type Lab (page 0).

Compute Temperature Lab

Learning Objective(s):

- Practice writing a method and main - Compute Temperature.
- Practice using System.out and Scanner.

Create Java Methods to Compute Temperature

Points: Complete this entire section for 10 points.

Write a Java `Temperature class` that has `public static method` s described below.

`public static double celsius(double fahrenheit)` – returns the Celsius equivalent of the parameter fahrenheit.

`public static double kelvin(double fahrenheit)` – returns the Kelvin equivalent of the parameter fahrenheit.

- The conversion algorithms shall be as follows.
    - To convert from Fahrenheit to Celsius, subtract 32, and then multiply by 5/9.
    - To convert from Celsius to Kelvin, add 273.15. You shall call the celsius() method of Temperature when implementing the kelvin() method.

The following are example calls to your `celsius` and `kelvin` methods.

```
double c = celsius(32); // c is 0.0
double k = kelvin(32);  // k is 273.15
```

`public static void main(String[] args)` – This is your program's `main` method that calls your `celsius` and `kelvin` methods to test they are correct. Use `System.out.print` , `System.out.println` , and a `Scanner` object to create the dialog for running your program. The following is an example dialog.

```
Conversions of freezing: 32.0
Celsius Expected: 0
Celsius Actual: 0.0
Kelvin Expected: 273.15
Kelvin Actual: 273.15

Conversions of boiling: 212.0
Celsius Expected: 100
Celsius Actual: 100.0
Kelvin Expected: 373.15
Kelvin Actual: 373.15

Conversions of a hot July day: 89.6
Celsius Expected: 32
Celsius Actual: 32.0
Kelvin Expected: 305.15
```

Compute Dice Lab

Learning Objective(s):

- Practice writing a method and main - Compute Dice.
- Practice using System.out and Scanner.

Create a Java Method to Compute a Dice Roll

Points: Complete this entire section for 10 points.

Write a Java `Dice class` that has `public static roll` described below.

`public static int roll1()` – uses `Math.random()` to returns random number between 1 and 6

In the section *Math.random Method* of Methods (page 0) we studied how to generate random integers. `Math.random` returns a `double` that has to be propberly manipulated. The following code generates a random number between 1 and 6 using `Math.random`.

```
int i2 = (int)(6*Math.random())+1;
```

The following are example calls to your `roll` method.

```
int r1 = roll(); // 1 <= r1 <= 6
int r2 = roll(); // 1 <= r2 <= 6
```

`public static void main(String[] args)` – This is your program's `main` method that calls your `roll` method to test they are correct. Use `System.out.print`, `System.out.println`, and a `Scanner` object to create the dialog for running your program. The following is an output where `roll` is called 5 times.

```
roll 1: 1

roll 2: 6

roll 3: 6

roll 4: 3

roll 5: 5
```

Using Strings Lab

Learning Objective(s):

- Practice writing a method and main - Using Strings.
- Practice using System.out and Scanner.

Create a Java Program with Strings, Input, and Output – 10 Points

Points: Complete this entire section for 10 points.

This lab allows you to practice using Java `String` methods. Write a Java `public class StringManip` that satisfies the following requirements

1. Your class shall contain a `public static void main()` method that does all of the work.
2. Use a `Scanner` object to read from the console.
3. `System.out` .println() to print to the console.
4. Prompt the user to enter a string, which you read into a variable of type `String` .
5. Use the `String` method `toUpperCase` to print the string and the uppercase version of the string.
6. Use the `String` method `length` to print the number of letters in the string.
7. Use the `String` method `indexOf` to print the index of the first space in the string.
8. The following is an example run of your program.

```
Enter a String: Gusty is teaching Java.
GUSTY IS TEACHING JAVA.
Your input has 23 characters.
The first space is at index 5.
```

9. Test your program to make sure it prints the correct values.

First Classes and Objects Labs

Learning Objective(s): The learning objectives for the entire First Classes and Objects Labs.

- Practice writing classes that define types.
- Practice creating instance variables, constructors, and instance methods within your class.
- Practice creating test cases for your types.
- Practice placing your test cases in tester classes that have a main method.
- Practice using your types to declare variables.
- Practice constructing objects of your types.
- Practice calling instance methods in the objects you construct.

First Classes and Objects Labs Overview

First Classes and Objects Labs is your first opportunity to create your own classes that are types, declare variables of your types, and construct objects of your types. This concept may appear difficult at first, but before long you will be an expert. All of the problems can be solved using `main``, Scanner for input, ``System.out` for output, helper methods with parameters that you write, and expressions. A few of the problems require a conditional expression.

Programming Contest

There is not a programming contest problem associated with this lab.

Submission Requirements

Follow guidance in Sample Lab Submission (page 0) for submitting your lab points.

Sample Lab

This section provides a sample lab specication and a solution. You can follow this sample in creating your solutions to the various labs in First Classes and Objects Jabs. For labs that you create a `class` that is a type, you will create two classes, each in its onw file.

1. The class that is your type, e.g., `public class Person` in file `Person.java`. Recall that a Java `class` that is used for a type has the following pattern.

```
public class MyType {
   // instance variables
   // constructors
   // instance methods
}
```

2. The class that tests your type, e.g., `public class PersonTester` in file `PersonTester.java`. This class will have a `main` method that declares and uses variables of type `MyType`.

Sample Lab Specification: Flower

Write a Java `class` that creates a `Flower` type, which has the following attributes.

- Instance variables for color and height.
- `Flower(String color)` - constructs a flower that is color `color` and height of 0 inches
- `water()` - causes the flower to grow two inches
- `squirrelHungry()` - causes the flower to be 0 inches
- `dye(Sring color)` - dyes the flower to be `color`
- `getColor()` - returns the color of the flower
- `getHeight()` - returns the height of the flower

Sample Lab Solution: Flower

File: **Flower.java**

```
public class Flower {

   String color;
   int height;

   public Flower(Sting c) {
      color = c;
      height = 0;
   }

   public void water() {
      height += 2;
   }

   public void squirrelHungry() {
      height = 0;
   }

   public void dye(String c) {
      color = c;
   }

   public String getColor() {
      return color;
   }

   public int getHeight() {
      return height;
   }
}
```

File: **FlowerTester.java**

```
public class FlowerTester {

   public static void main(String[] args) {
      Flower f = new Flower("Yellow");
      System.out.println(f.color() + " " + f.height());
      f.water();
      f.water();
      f.dye("Blue);
      System.out.println(f.color() + " " + f.height());
   }
}
```

Questions

Points: Answer these questions for 20 points. You have to handwrite or type the answers to get the points. For each question that you do not answer, subtract one point. If you do not answer any questions, you get negative points.

1. What is a literal? Provide examples for int, double, boolean, and String.
2. True/False: A Java String is a primitive data type.
3. Object-oriented programming uses classes and objects. What are classes and what are objects? What is the relationship between classes and objects?
4. What are instance variables and instance methods?
5. What is a constructor? What is the purpose of a constructor in a class?
6. Suppose that Kumquat is the name of a class and that fruit is a variable of type Kumquat. What is the meaning of the statement “fruit = new Kumquat();”? That is, what does the computer do when it executes this statement? (Try to give a complete answer. The computer does several things.)
7. Modify the following class so that the two instance variables are private and there is a getter method and a setter method for each instance variable:

```
public class Player {
   String name;
   int score;
}
```

8. True/False: Suppose there is a method declared as follows. If your main method wants to call getCellNumber, it must have a variable named exactly “friend” that it passes as the argument.

```
private static String getCellNumber(String friend)
```

9. True/False: There is no limit on how many lines of code a method can contain, but it can be called only once.

10. True/False: Two different methods can each have a variable named “x”. If one of the methods changes its value of “x”, this has no effect on the value of the other method’s “x” variable.
11. True/False: The variables used by Java methods are stored in something called the “stack.”
12. True/False: When a “return” statement is encountered while executing the code in a method, the method immediately terminates and gives an output. No more code in that method is executed at that time.
13. True/False: An “instance variable” is a variable which has a potentially different value for each instance of a class.
14. True/False: When an object comes into existence, sometimes a constructor is called, but sometimes not.
15. True/False: A class may only have one constructor, although it may have any name the programmer desires.
16. True/False: Making an instance variable “static” basically means that there is only one value for the variable shared by all objects of that type.
17. True/False: In Java, a two-dimensional array must be perfectly rectangular: all rows must be the same size.
18. Bessie is looking through a Java file, trying to find the class’s constructor. What two things are true about all constructors that will help her locate it?
19. The methods of a class can be called. Methods have parameters. Discuss the concept of parameters. What is the difference between formal parameters and actual parameters.
20. Explain how you use the terms method, subroutine, and function.
21. A subroutine/method/function is said to have a contract. What is meant by the contract? When you want to use a subroutine/method/function, why is it important to understand its contract? The contract has both “syntactic” and “semantic” aspects. What is the syntactic aspect? What is the semantic aspect?
22. A “black box” has an interface and an implementation. Explain what is meant by the terms interface and implementation.

Person Type Lab

Learning Objective(s):

- Practice writing a Person type.
- Practice creating instance variables, constructors, and instance methods within your Person type.
- Practice creating test cases for your Person type.
- Practice placing your test cases in PersonTester class that has a main method.
- Practice using your Person type to declare variables.
- Practice constructing objects of your Perons type.
- Practice calling instance methods in the Person objects you construct.

Create a Java Person Type

Points: Complete this entire section for 30 points. You must include JavaDoc in your code to receive the 30 points. If you choose to omit JavaDoc, you can only receive 20 points.

As we have learned, Java classes contain instance variables, constructors, and methods. We use the class as a type to declare variables and we use the constructors to create objects that we assign to our variables. A `Person class` is a great introduction to defining classes and constructing objects. For example, I can create a `Person` variable `gusty` and assign it to a person object by the following.

```
Person gusty = new Person("Gusty", "Cooper);
```

The class definition of Person will look like the following.

```
public class Person {
// instance variables

// constructors

// methods

}
```

We will create various implementations of a Person class throughout the semester. In this section, we will develop our first Java `public class Person` in two steps.

Step 1 – Fields `firstName` and `lastName`

1. Create a `public class Person` that has two `public` instance variables. By making these instance variables `public`, we can directly access them in a `Person` object.
    - a. `public String firstName;`
    - b. `public String lastName;`
2. Your `Person class` shall have one constructor that assigns the two `public` instance variables to their corresponding constructor parameters. The constructor is shown as follows.
    - a. `Person(String fn, String ln)`
3. After implementing your `class Person`, use BlueJ to construct a `Person` object.
    - a. Manipulate your `Person` object from the BlueJ GUI and the BlueJ code pad.
    - b. You can right-click on your `Person class` to create a `Person` object, which will appear in the BlueJ object window.
    - c. You can manipulate the object by right-clicking on it.
    - d. You can also manipulate the object in the BlueJ code pad. You may have to select Show Code Pad in the BlueJ View pull-down menu to view the code pad. The BlueJ code pad allows you to interactively enter Java commands. For example, if you have constructed a Person object with a name of person1, you can enter the following in the BlueJ code pad to directly access the instance variables.

```
> person1.firstName
“Gusty” (String)
> String s = person1.lastName;
> s
“Cooper” (String)
> person1.firstName = “Coletta”;
```

4. Edit your `Person class` to change the two instance variables to be `private`. `Private` instance variables cannot be directly accessed in a `Person` object. This is the typical technique for developing classes. We will add `public` getter and setter methods that users of the class must use to access the instance variables.
    - a. ```private String firstName;
    - b. ```private String lastName;
5. Add getter and setter public methods for the `firstName` and `lastName` instance variables. The following define the getters and setters.
    - a. `public String getFirstName() { return firstName; }`
    - b. `public void setFirstName(String fn) { firstName = fn; }`
    - c. `public String getLastName() { return lastName; }`
    - d. `public void setLastName(String ln) { lastName = ln; }`
6. After implementing your modified class `Person`, use BlueJ to construct a `Person` object.
    - a. Manipulate your `Person` object from the BlueJ GUI and the BlueJ code pad. You have to use the getters and setters. For example, you will have to do the following.

```
> person1.getFirstName
“Gusty” (String)
> String s = person1.getLastName;
> s
“Cooper” (String)
> person1.setFirstName(“Coletta”);
```

Step 2 – Field `friends`

1. Add the following instance variable to your `Person` class.

- a. `private String friends;` – this variable will contain the friends of your `Person`
- b. Java refers to instance variables as fields.

2. Add the following methods to your `Person` class.
   - a. `addFriend(String friend)` – this method will add the `String` friend to your `Person` by concatenating it onto the private `String friends` instance variable. String concatenation is performed using the + operator.

```
friends = friends + “ “ + friend;
```

   - b. String getFriends() { return friends; } – this method will return the friends of your Person that are stored in the private String friends.

3. Use BlueJ to create a Person, use the Person methods to get the first / last names, add some friends, and get the friends. The following shows a BlueJ Code Pad sequence of statements manipulating a Person p that has been created using the BlueJ object creation.

```
> person1.getFirstName()
"Gusty"   (String)
> person1.addFriend("Grandma");
> person1.addFriend("Tweetie");
> String f = person1.getFriends();
> f
" Grandma Tweetie"   (String)
```

4. Create a Java `public class personTester` that has a `static public void main()` method that performs the following operations.
   - a. Declares and constructs a `Person p` that you provide a first and last name.

```
Person p = new Person(“JerriAnne”, “Cooper”);
```

   - b. Uses the `addFriend()` method to add `“Gusty”`, `“Coletta”`, and `“Emily”` as friends.

```
p.addFriend("Gusty");
p.addFriend("Coletta");
p.addFriend("Emily);
```

- c. Uses the following Java statements to print information to the console.

```
System.out.println("Friends of " + p.getFirstNam
e() +
                    " " + p.getLastName();
System.out.println("Expected Friends: Gusty Colet
ta Emily");
System.out.println("Actual Friends: " + p.getFrie
nds());
```

Car Type Lab

Learning Objective(s):

- Practice writing a Car type.
- Practice creating instance variables, constructors, and instance methods within your Car type.
- Practice creating test cases for your Car type.
- Practice placing your test cases in CarTester class that has a main method.
- Practice using your Car type to declare variables.
- Practice constructing objects of your Perons type.
- Practice calling instance methods in the Car objects you construct.

Create a Java Car Type

Points: Complete this entire section for 30 points. You must include JavaDoc in your code to receive the 30 points for this section. If you choose to omit JavaDoc, you can only receive 20 points.

This lab has been modeled after a similar lab in the Udacity Java Course (https://www.udacity.com/course/intro-to-java-programming--cs046).

This section describes how to create a Java `Car class` and its corresponding `CarTester class`. Since we have been through the details of creating a couple of classes, the specification of the `Car class` is sparser than the previous section.

1. Create a `Car class` that has the following public interface.
    - a. `public Car(double mpg) // constructs a Car that has mpg`
    - b. `public void addGas(double amount)`
    - c. `public void drive(double distance)`
    - d. `public double getMilesDriven()`
    - e. `public double getGasInTank()`
2. Your `Car class` shall have the following `private` instance variables.
    - a. `private double milesDriven;`

- b. `private double gasInTank;`
- c. `private double milesPerGallon;`

3. After implementing your `class Car` , use BlueJ to construct a `Car` object.
   - a. Create a `Car` object using the BlueJ GUI.
   - b. Manipulate your `Car` object using the BlueJ code pad. The following shows an example manipulation of a car object created with an mpg of 20.

```
> car1.addGas(15);
> car1.drive(10);
> car1.getGasInTank()
14.5   (double)
```

4. Now add a `CarTester class` to your BlueJ project to test your `Car class` .
   - a. This will require analysis. Suppose you have a car that gets 20 mpg, you put 10 gallons of gas in the tank, and you drive the car on two trips, where the first trip is 100 miles and the second trip is 25 miles. Here are some questions to ponder.
     - i. How many gallons of gas do you have in your tank?
     - ii. How many miles have you driven your car?
   - b. Perform the type of analysis described in a for two cars that get different mpg.
   - c. In your `CarTester class` , create a `public static void main()` function that does the following.
     - i. Constructs a `Car` object for each car analyzed in b.
     - ii. Puts 10 gallons of gas in each car.
     - iii. Drives each car two times. The first time, drive 55 miles. The second time drive 62 miles.
     - iv. Given the above, compute by hand the expected values for the following. You have to think what the values for C1x, C1y, C2x, and C2y are.

```
car1: gasInTank ____C1
x________             milesDriven ____C1
y_________
car2: gasInTank ____C2x________          m
ilesDriven ____C2y_________
```

- v. Now add to your `CarTester class` , the following statements. The values from step c(iv) are substituted into C1x, C1y, C2x, and C2y below.

```
System.out.println(car1.getGasInTank());
System.out.println("Expected: C1x");
System.out.println(car1.getMilesDriven());
System.out.println("Expected: C1y");
System.out.println(car2.getGasInTank());
System.out.println("Expected: C2x");
System.out.println(car1.getMilesDriven());
System.out.println("Expected: C2y");
```

- vi. Now run your `CarTester class` to determine if your `Car class` is correctly implemented
- vii. If you get wrong answers, study your code and analysis to determine why you got the wrong answers. Correct your `CarTester` and `Car class` appropriately and retest.

TwoPays Type Lab

Learning Objective(s):

- Practice writing a TwoPays type.
- Practice creating instance variables, constructors, and instance methods within your TwoPays type.
- Practice creating test cases for your TwoPays type.
- Practice placing your test cases in TwoPaysTester class that has a main method.
- Practice using your TwoPays type to declare variables.
- Practice constructing objects of your Perons type.
- Practice calling instance methods in the TwoPays objects you construct.

Create a Java TwoPays Type

Points: Complete this entire section for 10 points.

The work for this section implements the same two `pay` methods from Split the Bill (page 0) and Paycheck (page 0) as instance methods in one class, `TwoPays` that is a type. You can use `TwoPays` to declare variables and construct objects. This is different from the `TwoPays` class in TwoPays (page 0), which implemented the two `pay` methods as `public static`. In this lab, the two `pay` methods are instance methods. We can have two instance methods in the same class with the same name as long as each has different parameters. The signature of a method is its return type along with the types of its parameters. In this case our two `pay` methods have the following signatures, which are different.

```
double pay(double bill, double tip, int numOfPeople)
double pay(int hours, double hourlyRage)
```

When we place the two `pay` methods in `TwoPays`, they are instance methods and not `static`. The `TwoPays class` does not have instance variables - just the two `pay` methods. Since we do not have any instance variables, we do not require an constructor to initialize them. This means we can use you can use the

default `TwoPays()` constructor, which Java provides to us for free. Once you have successfully created the `TwoPay class`, you construct objects of type `TwoPays` as follows.

```
TwoPays tp = new TwoPays();
double weeksWages = tp.pay(10,50);
double amountPerPerson = tp.pay(50,20,5);
```

Use BlueJ to construct and manipulate some TwoPays objects. The following are example commands in the BlueJ Codepad after you have constructed `twoPays1`.

```
> twoPays1.pay(50,.2,5);
12.0   (double)
> twoPays1.pay(50,10);
550.0   (double)
```

MyMath Type Lab

Learning Objective(s):

- Practice writing a MyMath type.
- Practice creating instance variables, constructors, and instance methods within your MyMath type.
- Practice creating test cases for your MyMath type.
- Practice placing your test cases in MyMathTester class that has a main method.
- Practice using your MyMath type to declare variables.
- Practice constructing objects of your Perons type.
- Practice calling instance methods in the MyMath objects you construct.

Create a Java MyMath Type

Points: Complete this entire section for 10 points.

This section will implement the same `average` and `largest` methods from Average (page 0) and Largest (page 0) as instance methods in one class, `MyMath`. If you completed bonus in Largest (page 0), include both `largest` methods in `MyMath`. This time the `average` and `largest` methods will not be `static`. The `MyMath class` will not have any instance variables. Since we do not have any instance variables, we will not require a constructor to initialize them. This means we can use you can use the default `MyMath()` constructor, which Java provides to us for free. You will be able to construct objects of type `MyMath` as follows.

```
MyMath m = new MyMath();
double average = m.average(10, 50, 20, 100);
double largest = m.largest(50, 20);
```

In addition to the `average` and `largest` methods, you shall include the following methods in your `MyMath class`.

`public static double crement(double x, char op)` – returns `x + 1` if `op` is `'+'`, otherwise returns `x - 1`. You shall use a conditional expression and the increment/decrement (++ –) operators in your implementation.

`public static double operate(double x, double y, char op)` – returns `x + y` if `op` is `'+'`, otherwise returns `x - y`. You shall use a conditional expression in your implementation. You can also try to use the increment/decrement (+= -=) operators in your implementation.

Use BlueJ to construct and manipulate some myMath1 objects. The following are example commands in the BlueJ Codepad after you have constructed `myMath1`.

```
> myMath1.average(10, 50, 20, 100)
45.0   (double)
> myMath1.largest(50,20)
50.0   (double)
> myMath1.average(1.0, 3, 4.0, 8)
4.0   (double)
> myMath1.largest(1.0, 100)
100.0   (double)
> myMath1.crement(5,'+')
6.0   (double)
> myMath.operate(10,432,'-')
-422.0   (double)
> myMath1.largest(10,50,2)
50.0   (double)
```

You will add methods to MyMath in Control Flow - MyMath Take 2 Lab (page 0).

Dice Type Lab

Learning Objective(s):

- Practice writing a Dice type.
- Practice creating instance variables, constructors, and instance methods within your Dice type.
- Practice creating test cases for your Dice type.
- Practice placing your test cases in DiceTester class that has a main method.
- Practice using your Dice type to declare variables.
- Practice constructing objects of your Perons type.
- Practice calling instance methods in the Dice objects you construct.

Create a Java Dice Type

Points: Complete this entire section for 15 points.

In this section you will write a Java `Dice class` that implements two `roll` methods. One of the `roll` methods will use the `Math.random` method. This code will be like that created in Dice (page 0). The other `roll` method will use a `java.util.Random` object. Since both `roll` methods have the same signature, you will name the methods `roll1` (uses `Math.random`) and `roll2` (uses `java.util.Random`). The `Math.random` method is available without an `import` statement, but you will have to include the following line

```
import java.util.Random;
```

in order to create `java.util.Random objects`. The `java.util.Random` constructor objects accepts a seed value, which will generate the same sequence of random values. The following code generates a random number between 1 and 6 using `Math.random` and `java.util.Random`.

```
int i1 = generator.nextInt(5)+1;
int i2 = (int)(6*Math.random())+1;
```

The Java Dice class has the following attributes.

- `Dice(int seed)` – this constructor will construct a `Random` generator object using a specific seed value. The following is example for the constructor that initializes a `private` instance variable `Random` generator.

```
generator = new Random(seed);
```

- `int roll1()` – uses `Math.random()` to returns random number between 1 and 6
- `int roll2()` – uses the instance variable that references a `java.util.Random` object to return a random number between 1 and 6

Write a `DiceTester class` that has a `public static void main()` method that uses your `Dice class` to test that it is correct. Your `DiceTester` should create two `Dice` objects (e.g., `d1` and `d2`) with the same seed value. You should make a sequence of calls to `d1.roll1()`, `d2.roll1`, `d1.roll2()`, and `d2.roll2()` of both dice to demonstrate `roll1()` for `d1` and `d2` has two different sequences of numbers and the `roll2()` has the same sequence of numbers for both `d1` and `d2`. The following is an example output from `DiceTester`.

`d1.roll1()` and `d2.roll1()` will roll a different sequence of numbers
`d1.roll2()` and `d2.roll2()` will roll the same sequence of numbers

```
d1.roll1(): 1   d2.roll1(): 1   d1.roll2(): 4   d2.roll2(): 4

d1.roll1(): 6   d2.roll1(): 3   d1.roll2(): 2   d2.roll2(): 2

d1.roll1(): 6   d2.roll1(): 1   d1.roll2(): 4   d2.roll2(): 4

d1.roll1(): 3   d2.roll1(): 2   d1.roll2(): 1   d2.roll2(): 1

d1.roll1(): 5   d2.roll1(): 1   d1.roll2(): 3   d2.roll2(): 3
```

MyString Type Lab

Learning Objective(s):

- Practice writing a MyString type.
- Practice creating instance variables, constructors, and instance methods within your MyString type.
- Practice creating test cases for your MyString type.
- Practice placing your test cases in MyStringTester class that has a main method.
- Practice using your MyString type to declare variables.
- Practice constructing objects of your Perons type.
- Practice calling instance methods in the MyString objects you construct.

Create a MyString Type

Points: Complete this entire section for 15 points.

This lab allows you to practice creating a Java class – `MyString` . Your `MyString` class will be your own simplified version of the Java `String` class. You will use the same `String` methods that you used to complete 2.1. Write a Java public class `MyString` that satisfies the requirements listed in this section. This section also describes steps to test your `MyString` class. You should recall that the general format of a Java class is as follows.

```
public class ```MyString``` {
// instance variables

// constructors

// methods

}
```

1. Your `MyString` class shall have one constructor that saves the parameter in a private instance variable `myString` . The constructor signature shall be as follows.

```
MyString(String s)
```

2. Your `MyString` class shall have the following public interface. You will have to implement each of these methods using the Java `String` methods that you used in section 2.1.
    - `public String capitalCase()` – returns the uppercase of the `String` used to construct the object.
    - `public int count()` – returns the number of characters in the `String` used to construct the object.
    - `public int firstSpace()` – returns the index of the first space of the `String` used to construct the object.
3. After implementing your class `MyString` , use the BlueJ GUI to construct `MyString` object. Test your `MyString` class using the BlueJ GUI and the BlueJ code pad to manipulate the `MyString` methods.
4. Create a Java public class `MyString` Tester that has a public static void main() method that performs the following operations.
    - A. Prompt the user to enter a string, which you will read into a variable of type `String` .
    - B. Using the `String` variable from step a, construct a `MyString` variable `myString` that contains the contents.

    ```
    MyString myString = new MyString(s);  // s is a S
    tring
    ```

    - C. Use the `myString.captialCase()` method to generate the uppercase of `myString` . Print the result.
    - D. Use the `myString.count()` method to compute the number of characters in `myString` . Print the result.
    - E. Use the `myString.firstSpace()` method to compute the index of the first space in `myString` . Print the result.
    - F. The following is an example run of your `myStringTester` .

    ```
    Enter a String: Gusty is teaching Java.
    GUSTY IS TEACHING JAVA.
    Your input has 23 characters.
    The first space is at index 4.
    ```

Temperature Type Lab

Learning Objective(s):

- Practice writing a Temperature type.
- Practice creating instance variables, constructors, and instance methods within your Temperature type.
- Practice creating test cases for your Temperature type.
- Practice placing your test cases in TemperatureTester class that has a main method.
- Practice using your Temperature type to declare variables.
- Practice constructing objects of your Perons type.
- Practice calling instance methods in the Temperature objects you construct.

Create a Temperature Type

Points: Complete this entire section for 15 points.

The work for this section implements the same methods from Temperature (page 0). In this section you write a Java `Temperature class` , which will use expressions to convert from Fahrenheit to Celsius and Kelvin. You will also write a `TemperatureTester class` that uses your `Temperature class` to construct objects to test your expressions. The Java `Temperature class` and `TemperatureTester class` that have the following attributes.

1. The `Temperature class` has the following constructor and methods. You will have to create the appropriate instance variable.

```
Temperature(double temperature) // constructor
double getTemperature() // getter for temperature
void setTemperature(double temperature) // setter for te
mperature
double celsius() // returns the Celsius equivalent of th
e temp
double kelvin()  // returns the Kelvin  equivalent of th
e temp
```

2. The conversion algorithms shall be as follows.

- a. To convert from Fahrenheit to Celsius, subtract 32, and then multiply by 5/9.
- b. To convert from Celsius to Kelvin, add 273.15. You shall call the `celsius()` method of `Temperature` when implementing the `kelvin()` method.

3. Write a `TemperatureTester class` that has a `main()` program that uses your `Temperature class` to test that it is correct. Your `TemperatureTester class` should be something like the following. You can augment this code with your own expected values.

```
public class TemperatureTester {
    /**
     * main() for class SplitTheBill
     */
    public static void main(String[] args) {
        Temperature t1 = new Temperature(32);  // freezi
ng
        Temperature t2 = new Temperature(212); // boilin
g
        System.out.println("Conversions of freezing: "
+
                            t1.getTemperature());
        System.out.println("Celsius Expected: 0");
        System.out.println("Celsius Actual: " + t1.celsi
us());
        System.out.println("Kelvin Expected: 273.15");
        System.out.println("Kelvin Actual: " + t1.kelvi
n());
        System.out.println("\nConversions of boiling: "
+
                            t2.getTemperature());
        System.out.println("Celsius Expected: 100");
        System.out.println("Celsius Actual: " + t2.celsi
us());
        System.out.println("Kelvin Expected: 373.15");
        System.out.println("Kelvin Actual: " + t2.kelvi
n());
        t2.setTemperature(89.6); // hot July day
        System.out.println("\nConversions of a hot July
day: " +
                           t2.getTemperature());
        System.out.println("Celsius Expected: 32");
        System.out.println("Celsius Actual: " + t2.celsi
us());
        System.out.println("Kelvin Expected: 305.15");
        System.out.println("Kelvin Actual: " + t2.kelvi
n());
    }
}
```

4. The output of the `TemperatureTester` that corresponds to the above example is the following.

```
Conversions of freezing: 32.0
Celsius Expected: 0
Celsius Actual: 0.0
Kelvin Expected: 273.15
Kelvin Actual: 273.15

Conversions of boiling: 212.0
Celsius Expected: 100
Celsius Actual: 100.0
Kelvin Expected: 373.15
Kelvin Actual: 373.15

Conversions of a hot July day: 89.6
Celsius Expected: 32
Celsius Actual: 32.0
Kelvin Expected: 305.15
```

Control Flow Lab

Learning Objective(s):

- Practice writing code that uses loops and conditional statements.
- Practice solving problems with a while loop.
- Practice solving problems with a for loop.
- Practice solving problems with a do-while loop.
- Practice solving problems with if-else statements.
- Practice solving problems with a switch statement.

Control Flow Lab Overview

The Control Flow Lab has many problems. The Control Flow Lab strikes at the heart of solving basic problems with algorithms. You have to complete the following by the first due date.

- First Loops
- Potato Class
- Loops, Ifs, Switches
- Millionaire
- MyMath Take 2
- Twitter Type
- Gerbil Game

You can work on the following until the second due date.

- Triangle
- Credit Card Type
- Pig Game People
- Pig Game Computer

Programming Contest

To receive maximum points on this lab, submit a solution to Programming Contest (page 0) Problem C - Counting Letters.

Submission Requirements

Follow guidance in Sample Lab Submission (page 0) for submitting your lab points.

Sample Lab

See previous labs for a sample lab specification and solution.

Questions

Points: Answer these questions for 20 points. You have to handwrite or type the answers to get the points. For each question that you do not answer, subtract one point. If you do not answer any questions, you get negative points.

Some of the questions ask if the code have problems? If there is a problem, state what it is and a way to remove the problem.

1. What is the last value printed by the following loop?

```
int n = 2;
while (n < 1000) {
  n = 10 * n;
  System.out.println(n);
}
```

2. How many numbers are printed by the following loop?

```
for (int i = 10; i <= 20; i++) {
  System.out.println(i);
}
```

3. How many numbers are printed by the following loop?

```
for (int n = 10; n >= 0; n--) {
  System.out.println(n);
}
```

4. How many numbers are printed by the following loop?

```
for (int n = a; n <= b; n++) {
  System.out.println(n);
}
```

5. How many numbers are printed by the following loop?

```
for (int n = a; n < b; n++) {
  System.out.println(n);
}
```

6. How many times is the condition of the following loop evaluated? How many times does the loop body execute?

```
for (int n = 1; n <= 6; n = n + 2) {
  System.out.println(n);
}
```

7. Can a for loop's body execute 0 times?
8. True/False: When the computer encounters an "if/else" construct, it is sometimes the case that both the "if" body's code and also the "else" body's code will get executed.
9. Does the following code have a problem? Y/N ___, if so, explain how to remove.

   `java if (salary > 40000 && < 60000) { taxRate = .4; }`

10. Does the following code have a problem? Y/N ___, if so, explain how to remove.

```
if (weapon == "halberd") {
    hitPoints = hitPoints - 8;
}
else {
    hitPoints = hitPoints - 5;
```

11. Does the following code have a problem? Y/N ___, if so, explain how to remove.

```
if (yearGraduated = currentYear) {
    System.out.println("We'd like to welcome you to the
alumni
association!");
}
```

12. True/False: Java's "do" loop (a.k.a. "do/while" loop) is a very useful programming construct that is used quite often by Java programmers.
13. True/False: When the condition of a while loop becomes false, the program will immediately exit the loop, even if this occurs somewhere in the middle of the loop body.
14. True/False: Every "for" loop can be converted into an equivalent "while" loop – the "for" loop is just a convenient way of bundling together the operations involving a loop counter.
15. True/False: It's best programmatic style to use a "for" for fixed-iteration loops (a.k.a. definite loops, a.k.a. counter-based loops) and a "while" for variable-iteration loops (a.k.a. indefinite loops, a.k.a. condition-based loops.)
16. What is a block statement? Why is a block statement useful?
17. What is the difference between a while loop and a do-while loop?
18. Write a for loop that will print out all the multiples of 3 from 3 to 36, that is: 3 6 9 12 15 18 21 24 27 30 33 36.
19. The following do-while loop forces the user to enter a number that is less than 100. Update the loop so that the user has to enter a number between 0 and 100 inclusive.

```
do {
  System.out.print("Enter a value < 100: ");
  value = in.nextInt();
} while (value >= 100);
```

20. What are the two conditional statements provided by Java?
21. True/False: A while loop is all that is needed to solve problems. Explain your answer.

First Loops Lab

Learning Objective(s):

- Practice writng for, while, and do-while loops.
- Practice using an if statement within a loop.

Create a Java Program to Study and Play Video Games

Points: Complete this entire section for 10 points.

Create a Java class MyFirstLoop that has a public static void main(String[] args) method that contains a while loop, a for loop, and a do-while loop. Each of your loops will print Study to the console on 6 separate lines. The following is example output (the numbers are part of your output).

```
1. Study
2. Study
3. Study
4. Study
5. Study
6. Study
```

The following for loop accomplishes the above.

```
for (int i = 1; i < 7; i++)
    System.out.println(i + ". Study");
```

You have to write the equivalent while and do-while loops.

Update the code in your MyFirstLoop class so that it writes Study on the odd lines and Video Game on the even lines. The following is an example output (the numbers are part of your output).

```
1. Study
2. Video Game
3. Study
4. Video Game
5. Study
6. Video Game
```

You can choose to put the conditional code in one of the three loops. The following code demonstrates determining whether an int number is even or odd.

```
if (number % 2 == 0)
   System.out.println("Even");
else
   System.out.println("Odd");
```

Potato Class Lab

Learning Objective(s):

- Practice writing a Potato class.
- Practice writing loops and conditional statements.

Create a Java Potato Class

Points: Complete this entire section for 10 points.

Some may remember the children's counting rhythm, "One potato, two potato, three potato, four…, five potato, six potato, seven potato, more."

Create a Potato class that has the following public interface.

```
Potato()          // Constructs a Potato
void count(int c) // prints the children's counting rhythm
```

Use BlueJ to construct and manipulate some Potato objects. The following are example commands in the BlueJ Codepad after you have constructed potato1.

```
potato1.count(5);
One potato
two potato
three potato
four…
five potato
potato1.potato(0);  // prints nothing
potato1.count(10);
One potato
two potato
three potato
four…
five potato
six potato
seven potato
more.
One potato
two potato
```

One way to implement the count method is to use Java's modulo operator (%) and a switch statement. The following shows this technique. You have to place the switch statement inside a loop that correctly counts the int counter.

```
switch (counter % 8) {
   case 0: System.out.println("more"); break;
   case 1: System.out.println("One potato"); break;
   … more code …
   case 7: System.out.println("seven potato"); break;
}
```

Loops, Ifs, Switches Lab

Learning Objective(s):

- Practice writing for, while, and do-while loops.
- Practice using conditional statements in loops.

Create a Java Program with Various Loops, Ifs, and Switches

Points: Complete this entire section for 20 points.

Create a Java `VariousControlStructs class` that has a `public static void main(String[] args)` method that does the following.

1. Use a `Scanner` object for input and System.out for output. The following shows a prompt and a read.

```
import java.util.Scanner;
Scanner in = new Scanner(System.in);
System.out.print("Enter num of times to loop: ");
int usersNumber = in.nextInt();
```

2. Prompt the user for an integer number, which you will use as the number of times to loop. You will use a for-loop, a while-loop, or a do-while loop to accomplish the following. Make sure you understand the differences between a for-loop, a while-loop, and a do-while loop. For this problem, you should select the loop that you need the most practice. This means your main() method will contain this loop, plus other loops described in this section. If you select a for loop, your loop will look like the following.

```
for (int i = 1; i <= usersNumber; i++) {
// place your code here
}
```

- a. User you loop to compute the sum of the numbers from 1 to the user's input number. This is the accumulator pattern. You will have to initialize a variable to 0 and add the current loop count to the variable on each iteration. The following demonstrates the accumulator pattern for accumulating a sum.

```
int sum = 0;
for (int i = 1; i <= usersNumber; i++) {
   sum += i;
   // you will have more code in your loop
}
```

- b. On each iteration of the loop, you will have four prompts to the user asking them to input a number.
    - i. The first prompt will ask the user to input any integer. You will use the user's entered number as the control variable of a three-way if-statement that prints "Negative", "Zero", or "Positive" to the console based upon the value of the user's number. One way to implement the three-way if-statement is the following. You should notice that the if-statement did not use curly braces because there was only a single-statement after the test conditions.

```
if (usersNumber < 0)
   System.out.println("Negative");
else if (usersNumber == 0)
   System.out.println("Zero);
else
   System.out.println("Positive);
```

    - ii. The second prompt will ask the user for an integer number between 1 and 4, which you will use as the control variable for a switch statement that prints `"One"`, `"Two"`, `"Three"`, `"Four"`, or `"Invalid Choice"` based upon the value of the user's number. One way to implement the `switch` statement is the following. Be sure to include the `break` statement after each case.

```
switch (usersInput) {
   case 1: System.out.println("One"); brea
k;
   //... more code ...
   default: System.out.println("Invalid Ch
oice"); break;
}
```

- iii. The third prompt will ask the user for an integer number between 1 and 10. You will print two lines (first line is `"Happy"` and second line is `"Birthday"`) if the user enters 1, 2, 3, 4, or 5. You will print one line that is `"Birthday"` if the user enters 5, 6, 7, 8, 9, or 10. You will print `"Invalid Choice"` if the user does not enter 1, 2, …, 10. One way to implement this is to use a switch statement that does not contain a break. The following is sample code.

```
switch (switchValue) {
case 1: case 2: case 3: case 4: case 5:
   System.out.println("Happy");
case 6: case 7: case 8: case 9: case 10:
   System.out.println("Birthday");
   break;
default:
   System.out.println("Invalid Choice");
   break;
}
```

- iv. The fourth prompt will ask the user for a countdown number. You will print a launch countdown beginning with the user's number and counting down to 0. For example, if the user entered a 10, your program will print the following. `10 9 8 7 6 5 4 3 2 1 0 Blastoff!` One way to implement this is to use a `for` loop with its variable decrementing. The following is a sample.

```
for (int j = countdown; j >= 0; j--) {
    System.out.print(j + " ");
}
```

3. In addition to the loop described in step 2, you are to create a sentinel loop that prompts the user to enter a `double` . This sentinel loop will not be nested in the loop described in 3. This sentinel loop continues reading a `double` until the user enters `Q` . You should use a `Scanner hasNextDouble()` method to control your while loop. When `hasNextDouble()` is `false` , you can assume the user has entered a `Q` . The following demonstrates this technique using the Scanner in object. When the user enters a Q, your program should print the following information. You should realize that a, b, c, and d are various implementations of the accumulator pattern.
    - a. The sum of the numbers entered by the user.
    - b. The product of the numbers entered by the user.
    - c. The average of the numbers entered by the user.
    - d. The largest of the numbers entered by the user.
    - e. The following is the start of the code.

```
while (in.hasNextDouble())
{
   userNum = in.nextDoubl
e();
   // use userNum to compute sum, product, and av
era ge
}
```

4. The following is an example output of VariousControl Structs.

```
Enter number of times to loop: 3
The sum from 1 to 3 is 6
---------------------------------------
Iteration 1 of 3
Enter an integer: 0
Zero
Enter 1, 2, 3, or 4: 2
Two
Enter 1, 2, ... or 10: 5
Happy
Birthday
Enter a launch countdown number: 10
10 9 8 7 6 5 4 3 2 1 0 Blastoff!
---------------------------------------
Iteration 2 of 3
Enter an integer: -1
Negative
Enter 1, 2, 3, or 4: 5
Invalid Choice
Enter 1, 2, ... or 10: 10
Birthday
Enter a launch countdown number: 20
20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0  Bl
astoff!
---------------------------------------
Iteration 3 of 3
Enter an integer: 100
Positive
Enter 1, 2, 3, or 4: 1
One
Enter 1, 2, ... or 10: 100
Invalid Choice
Enter a launch countdown number: 3
3 2 1 0 Blastoff!

Enter a sequence of numbers one at a time. Q to quit
Enter a number: 5
Enter a number: 4
Enter a number: 6
Enter a number: Q
Sum of your numbers is 15.0
Average of your numbers is 5.0
Product of your numbers is 120.0
Largest of your numbers is 6
```

Millionaire Lab

Learning Objective(s):

- Practice using a while loop to solve a problem.

When Will You Be a Millionaire

Points: Complete this entire section for 10 points.

This lab has been modeled after a similar lab in the Udacity Java Course (https://www.udacity.com/course/intro-to-java-programming--cs046).

If you have $100.00 to invest and the interest rate is 1% annually, how many years before your investment is worth $1,000,000.00? Create a Java class `Millionaire` that has a `public static void main(String[] args)` method that computes the answer. You should use a `Scanner` to ask the user for their initial investment (`double`) and the interest rate (`double`). You shall write a `while` loop that uses the input values to determine how long it will take to become a millionaire. With big numbers, the Java technique of using an _ in a numeric literal can be helpful. The following is an example of the literal billion with an underscore.

```
int billion = 1_000_000_000;
```

The following is an example output of the main method of the Millionaire class.

```
Enter initial investment: 100
Enter interest rate (e.g., enter .01 for 1%): .01
You will be a millionaire in 926 years.
```

MyMath Type - Take 2 Lab

Learning Objective(s):

- Practice maintenace programming by adding methods to MyMath class.
- Practice using loops and conditionals to solve problems.

Create a Java MyMath Type - Take 2

Points: Complete this entire section for 15 points.

In this lab, you will add two new methods to MyMath (page 0). The new methods are described as follows.

- `int numDigits(int number)` – returns the number of decimal digits in number.
- `int coins(int cents)` – returns the minimal number of coins – quarter, dime, nickel, and penny – that are required to make the amount cents. For example, the minimal number of coins to make 76 cents of change is 4 (3 quarters and 1 penny).

You will be able to construct and manipulate objects of type MyMath as follows.

```
MyMath m = new MyMath();
double average = m.average(10, 50, 20, 100);
double largest = m.largest(50, 20);
int i = m.numDigits(1049); // i is 4
int j = m.numDigits(34);   // j is 2
int c = m.coins(0);   // c is 0
int c = m.coins(76);  // c is 4
int c = m.coins(73);  // c is 7
```

To create the algorithms for `numDigits` and `coins` methods, you should first work through several examples by hand. This will help you discover the algorithm. In fact, I find it helpful to always begin with simple concrete cases that I can solve by hand. I work on these cases until I understand the algorithm enough to capture it as pseudo code, after which I transcribe the algorithm into a programming language.

Let's think about the `numDigits` algorithm. Consider the number 105 for example, which has 3 digits. Notice you can repeatedly perform integer division three times as shown below. The number of divisions is the answer. Work another concrete example, and then generalize into an algorithm using a loop and integer division. You should realize that the Java division operator on int operands computes how many times the divisor goes into the dividend, which is an int. The loop will terminate when the divisor goes into the dividend 0 times.

```
105 / 10 is 10 – 10 goes into 150 10 times, with a remainder o
f 5
 10 / 10 is  1 – 10 goes into 10 1 times, with a remainder of 0
  1 / 10 is  0 – 10 goes into 1 0 times, with a remainder of 1
```

Let's think about the `coins` algorithm. You intuitively know (a) 76 cents can be 3 quarters and 1 penny for a minimum of 4 coins, (b) thirty cents can be 1 quarter and 1 nickel for 2 coins, and (c) twenty cents can be 2 dimes. What is the algorithm? You first count the number of quarters you can get, then count the number of dimes, then nickels, and finally pennies. Sometimes you cannot get a particular coin. For the 76 cents example, you cannot get any dimes or nickels. Work several concrete examples to discover the algorithm. The algorithm can be created with a sequence of if statements that use integer division and remainder.

Use BlueJ to construct and manipulate some MyMath objects. The following are example commands in the BlueJ Codepad after you have constructed myMath1.

```
> myMath1.average(1.0, 3, 4.0, 8);
4.0   (double)
> myMath1.largest(1.0, 100);
100.0   (double)
> myMath1.numDigits(105);
3   (int)
> myMath1.coins(76);
4   (int)
```

Twitter Type Lab

Learning Objective(s):

- Practice creating a Twitter Type.
- Practice using String methods.
- Practice using loops and conditionals to solve problems.

Create a Java Twitter Type

Points: Complete this entire section for 25 points.

This lab has been created by mashing together a similar lab in the Udacity Java Course (https://www.udacity.com/course/intro-to-java-programming--cs046) and some methods from a lab created by Ron Zacharski.

Write a Java `Twitter class` that has the following public interface.

- `Twitter(String tweet) // Constructs a Twitter`
- `void setTweet(String tweet) // Changes value of tweet`
- `String getTweet() // returns tweet`
- `int numVowels() // returns number of vowels in tweet`
- `String noVowels() // returns tweet without vowels`
- `String reverse() // reverses tweet`
- `String pigify() // pigify word – see below`

The following demonstrates using your `Twitter class` .

```
Twitter t = new Twitter("This is a post with vowels);
String shortString = t.noVowels();
// shortString contains Ths s  pst wth vwls
t.setTweet("12345");
String back = t.reverse();  // back contains 54321
```

For the method `int numVowels(String word)` , you will have to loop through all of the characters in the variable word counting the vowels. How do you do this? You will have to use one of the loop statements. There are Java `String` methods that are quite helpful.

- You can use the `String` method `word.substring(i, i+1)` to examine one character of word.
- You can use the `String` method `“aeiouAEIOU”.contains(letter)` to see if a letter is a vowel.

The following demonstrates this technique. You should notice this algorithm is an instance of the accumulator pattern that accumulates a sum in an `int` . Please do not type this algorithm into your code without understanding how the algorithm works. If you do not understand the algorithm, you are not learning to be a computer programmer. If you understand this algorithm, you will easily be able to create the algorithms for noVowels and reverse.

```
int numVowels = 0;
for (int i = 0; i < word.length(); i++) {
     if ("aeiouAEIOU".contains(word.substring(i, i+1))) {
          numVowels++;
     }
}
```

The above algorithm is a clever way of determining is a letter in a `String` is a vowel. A more pedestrian way uses the `String equals` method in a disjunctive `boolean` expression. The following demonstrates this approach.

```
int numVowels = 0;
for (int i = 0; i < word.length(); i++) {
     if (word.substring(i,i+1).equals(“a”) ||
         word.substring(i,i+1).equals(“e”) ||
          … fill in the rest of the vowels ||
         word.substring(i,i+1).equals(“U”))
        numVowels++;
}
```

You will add methods to Twitter in Classes, Objects,… - Twitter Revisited Lab (page 0).

The method `noVowels(String word)` will be similar to `numVowels(String word)` in that you have to loop through the parameter word; however, `noVowels()` uses the accumulator pattern to constructs a new `String` from word without the vowels.

The method `reverse(String word)` will also use the accumulator pattern to construct a new `String` . The reverse method can use the `word.substring(i, i+1)` method beginning at the end of word. The following demonstrates the beginning of the for loop.

```
for (int i = word.length() - 1; i >= 0; i--)
```

The `pigify()` method returns a `String` that is the Pig Latin version of the tweet.

The rules for Pig Latin when applied to words are as follows.

1. For words that start with a vowel, just add way to the end:
    - a. egg → eggway
    - b. and → andway
2. For words that start with a consonant or consonant cluster move the consonant cluster to the end of the word and add ay
    - a. dog → ogday
    - b. kids → idskay
    - c. frog → ogfray
3. Numbers and punctuation are not changed by Pig Latin.

We could pick words out of our tweet, but the algorithm is a bit too complex for this lab. When we study Java arrays, we can combine arrays with the String method split to more easily construct this algorithm. For now we will create a `pigify()` method that uses the rules to construct a suffix to our entire tweet based upon the first characters of our tweet. The rules are rewritten as follows.

1. For tweets that start with a vowel, just add way to the end of our tweet:
    - a. Eggs are good → Eggs are goodway
    - b. and that is a wrap → and that is a wrapway
2. For tweets that start with a consonant or consonant cluster move the consonant or consonant cluster to the end of our tweet and add ay
    - a. Dogs are good → ogs are goodDay
    - b. Kids play outside → ids play outsidekay
    - c. frog level VA → og level VAfray
3. Numbers and punctuation are not changed by Pig Latin.

Use BlueJ to construct and manipulate some `Twitter` objects. The following are example commands in the BlueJ Codepad after you have constructed `twitter1` .

```
> twitter1 = new Twitter("Potato");
> twitter1.numVowels()
3   (int)
> twitter1.noVowels()
"Ptt"   (String)
> twitter1.reverse()
"otatoP"   (String)
> Twitter w1 = new Twitter("egg");
> String s1 = w1.pigify();
> s1
"eggway"  (String)
> Twitter w2 = new Twitter("and");
> String s2 = w2.pigify();
> s2
"andway"  (String)
> Twitter w3 = new Twitter("frog");
> String s3 = w3.pigify();   // s3 is ogfray
> s2
"ogfray"  (String)
> Twitter w4 = new Twitter("dog")
> String s4 = w4.pigify();   // s4 is ogday
> s4
"ogday"  (String)
```

You will add methods to Twitter in Classes, Objects, … - Twitter Revisited Lab (page 0).

Gerbil Game Lab

Learning Objective(s):

- Practice solving a simple game problem with nested ifs.

Create a Java Gerbil Game

Points: Complete this entire section for 10 points.

NOTE: The Gerbil Game lab is a rendition of a project created by Ron Zacharski.

This section defines a game with a pet gerbil that descends into crevices to acquire precious metal nuggets. The gerbil can only carry 10 ounces. The precious metals and their values (Rhodeium is the highest and Ruthenium is the lowest) are as follows.

1. Rhodium
2. Platinum
3. Gold
4. Ruthenium

You want to train your gerbil such that when sent into a crevice, the gerbil returns with the most valuable haul. For example, if the crevice contains 5 ounces of Rhodium, 6 of Platinum, 4 of Gold, and 7 of Ruthenium, I want her to return with 5 ounces of Rhodium and 5 of Platinum.

Create a Java `Gerbil class` that has one `public static void found` method with the following signature.

```
public static void found(int rhodium, int platinum, int gold, i
nt ruthenium)
```

The formal parameters are the amounts of Rhodium, Platinum, Gold, and Ruthenium. The method found prints to the console the best retrieval of metal for that trip into the crevice. The following are example calls to the function found.

```
> found(5, 6, 4, 7);     // prints the following
5 Rhodium
5 Platinum
0 Gold
0 Ruthenium
> found(10, 10, 10, 10); // prints the following
10 Rhodium
0 Platinum
0 Gold
0 Ruthenium
> found(3, 0, 0, 1);     // prints the following
3 Rhodium
0 Platinum
0 Gold
1 Ruthenium
```

Work through several specific, concrete examples to discover the algorithm. The algorithm I created is several nested ifs.

You can solve this problem without a loop.

Simple Triangle Lab

Learning Objective(s):

- Practice using nested loops to draw a simple triangle on the terminal.

Create a Simple Java Triangle Program – 10 Points

Points: Complete this entire section for 10 points.

Create a Java class SimpleTriangle that has the following method

```
public static void drawTriangle(int size)
```

that draws the following triangle if called with an actual argument of 5.

```
<>
<><>
<><><>
<><><><>
<><><><><>
```

The size of the triangle drawn is dependent on the value of parameter `size` . Work through several specific examples to discover the algorithm. The algorithm I created is two for loops, one nested inside of the other.

You should call your `drawTriangle` method from a `main` method.

Credit Card Lab

Learning Objective(s):

- Practice using loops and conditionals to solve a credit card problem.

Create a Java Credit Card Type

Points: Complete this entire section for 20 points.

NOTE: The Credit Card lab is a rendition of a project created by Karen Anewalt.

You are to implement a CreditCard class that has the following public interface.

- `CreditCard(long cardNumber)` - constructs a credit card
- `boolean isValid()` - returns `true` if the card number is valid, returns `false` if invalid

Before providing the pseudo code for the credit card validation algorithm, let's discuss an algorithm to pick each digit of an integer, beginning at the rightmost digit. This is essentially an algorithm that converts a decimal number to a decimal number. Consider the number 452. We can perform the following steps.

1. 452 % 10 yields 2 and 452 / 10 yields 45.
2. 45 % 10 yields 5 and 45 / 10 yields 4.
3. 4 % 10 yields 4 and 4 / 10 yields 0

We can generalize the above steps into the following algorithm to compute the sum of the digits in the number 452.

```
int n = 452;
int sum = 0;
while (n > 0) {
  int digit = n % 10;
  sum = sum + digit;
  n = n / 10; // integer divide
}
System.out.println(sum); // 11, which is sum of 452
```

Now that you know how to pick digits from an integer, the pseudo code for this credit card validation algorithm, which you will use to implement the isValid() method is the following.

```
Pseudocode for isValid
sum = 0
count = 0
for each digit starting from the right
   count ++
   if count is odd
      sum = sum + digit
   else if (digit < 5)
      sum = sum + 2 * digit
   else
      sum = sum + 2 * digit - 9
if the last digit of the sum is zero
   The card number is valid
```

You should note that this discussion is leading you to an implementation where Credit Card numbers are numbers. You will quickly discover that Credit Card numbers are too large to fit into an `int`, and that you need to use a Java `long` type. The following are some credit card numbers, with the suffix `L` indicating they are `long` s.

- 4417123456789113L – Valid
- 4417123456879113L – Invalid
- 4111111111111111L – Valid
- 5155555511115151L – Valid
- 4147400100642181L – Invalid

You could pursue an alternative design where you store Credit Card numbers as `String` s. You have already used the `String substring()` method in your Twitter (page 0). You can loop through a `String` credit card number backwards, using `substring` to examine each number (which is really a `String`). You can use the `Integer.parseInt(str)` method to convert the `String` to an `int`. The following demonstrates this concept.

```
String num = "123";
int i = Integer.parseInt(num.substring(2,3));
i
3   (int)
```

Now you have a choice as to how you want to design your `CreditCard` class. If you choose to represent Credit Card numbers as `String`, your public interface will be the following.

- `CreditCard(String cardNumber)` - constructs a credit card
- `boolean isValid()` - returns `true` if the card number is valid, returns `false` if invalidd

You should implement a `CreditCardTester class` that has a `public static void main(String[] args)`. Your tester should create CreditCard objects for the Credit Card numbers provided above. You should print out the expected value and the actual value. The following is an example run of your `CreditCardTester main()`.

```
Test our CreditCard class.

Actual: 4417123456789113 is true
Expected: true
Actual: 4417123456879113 is false
Expected: false
Actual: 4111111111111111 is true
Expected: true
Actual: 5155555511115151 is true
Expected: true
Actual: 4147400100642181 is false
Expected: false
```

Pig - Two Person Lab

Learning Objective(s):

- Practice using loops and conditionals to create a Pig game.

Create a Java Pig Game (Two-person)

Points: Complete this entire section for 10 points.

NOTE: The Pig Game lab is a rendition of a project created by Ian Finlayson.

Pig is a two-player game in which each player tries to get 100 points before the other. In Pig, players take turns rolling a die. For each turn, the player may roll the die as many times as they like until they get a 1. If a player rolls a 1, then they receive zero points for that turn. If a player elects to stop rolling (or “hold”) before they get a 1, then they get the sum of all die rolls that turn. The score for that turn is then added into their final score. The strategy in Pig is to try to get as many points as you can, then hold before you lose them all, which happens when you roll a 1. The following is an example roll sequence, where I earn 13 points.

- I roll a 5.
- I can accept those 5 points, or I can choose to roll again. In this example, I choose to roll again, and I and roll a 2. Then
- I now have a total of 7 points, which I can accept or I can choose to roll again in hopes of getting more points. Since I am feeling lucky, I choose to roll again and roll a 6.
- I now have 13 points. Since I think my luck has run out, I choose to hold and receive 13 points for that roll sequence.

On the other hand, if I roll a 6, then a 3, then a 1, I receive 0 points for that turn.

The Two-person version of this program allows two people to compete against each other. Your program will roll the die and keep score. Each person playing the game is responsible for his or her own strategy. You should use your `Dice` type created in Dice (page 0). You can simply copy the `Dice.java` file into your BlueJ project folder for this project. You can also ask the user for a winning score so that you do not have to play to 21. A smaller winning score can be easier to test. The following is an example sequence of your program.

```
Let's play some pig.
What is the winning score?
11

Player 1 Your Turn!
Your turn score is 0
Player 1 Total Score: 0;  Player 2 Total Score: 0
Would you like to (R)oll or (H)old? r
You rolled a 1
You scored a 0 on this turn!

Player 2 Your Turn!
Your turn score is 0
Player 2 Total Score: 0;  Player 1 Total Score: 0
Would you like to (R)oll or (H)old? r
You rolled a 5

Player 2 Your Turn!
Your turn score is 5
Player 2 Total Score: 5;  Player 1 Total Score: 0
Would you like to (R)oll or (H)old? r
You rolled a 1
You scored a 0 on this turn!

Player 1 Your Turn!
Your turn score is 0
Player 1 Total Score: 0;  Player 2 Total Score: 0
Would you like to (R)oll or (H)old? r
You rolled a 3

Player 1 Your Turn!
Your turn score is 3
Player 1 Total Score: 3;  Player 2 Total Score: 0
Would you like to (R)oll or (H)old? r
You rolled a 5

Player 1 Your Turn!
Your turn score is 8
Player 1 Total Score: 8;  Player 2 Total Score: 0
Would you like to (R)oll or (H)old? r
You rolled a 2

Player 1 Your Turn!
Your turn score is 10
Player 1 Total Score: 10;  Player 2 Total Score: 0
Would you like to (R)oll or (H)old? r
You rolled a 2
```

```
Your total score is 12

Congratulations Player 1! You won.
```

Pig - Computer Lab

Learning Objective(s):

- Practice solving a computer simulation of the Pig Game.

Create a Java Pig Game (Computer)

Points: Complete this entire section for 10 points.

NOTE: The Pig Game lab is a rendition of a project created by Ian Finlayson.

Now that you have implemented Pig, lets create another version of Pig where the second player is the computer. This is a relative simple simulation. Whenever it is the computer's first time to roll, it will always select yes. You need some algorithm to choose whether to roll on subsequent rolls. The following two steps shows a way to play the computers turn, where the computer has its own `Dice` (page 0) object, shown as `computerDie` in the following. This means that your code would have the following declaration: `Dice computerDie = new Dice(1342);`

- Roll 1: Yes perform the first roll.
- Roll 2, 3, 4, …: Use the following algorithm to determine whether to roll. You should realize the following is an aggressive algorithm since the computer will choose to roll 2 out of 3 times.

```
int choice = computerDie.roll2();
boolean computerRoll;
switch (choice) {
case 2: case 4: case 5: case 6:
     computerRoll = true;
     break;
case 1: case 3:
     computerRoll = false;
     break;
```

The following is an example run.

```
Let's play some pig.
What is the winning score?
11

Player 1 Your Turn!
Your turn score is 0
Player 1 Total Score: 0; Computer Total Score: 0
Would you like to (R)oll or (H)old? r
You rolled a 6

Player 1 Your Turn!
Your turn score is 6
Player 1 Total Score: 6; Computer Total Score: 0
Would you like to (R)oll or (H)old? r
You rolled a 4

Player 1 Your Turn!
Your turn score is 10
Player 1 Total Score: 10; Computer Total Score: 0
Would you like to (R)oll or (H)old? h

Computer Your Turn!
Your turn score is 0
Computer Total Score: 0; Player 1 Total Score: 10
Computer chose to roll.
You rolled a 2

Computer Your Turn!
Your turn score is 2
Computer Total Score: 2; Player 1 Total Score: 10
Computer chose to roll.
You rolled a 5

Computer Your Turn!
Your turn score is 7
Computer Total Score: 7; Player 1 Total Score: 10
Computer chose to hold.

Player 1 Your Turn!
Your turn score is 0
Player 1 Total Score: 10; Computer Total Score: 7
Would you like to (R)oll or (H)old? r
You rolled a 2
Your total score is 12

Congratulations Player 1! You won.
```

Arrays, ArrayLists Labs

Learning Objective(s):

- Practice using arrays and ArrayLists in solving problems.
- Practice looping patterns with arrays and ArrayLists.

Array, ArrayLists Overview

In this lab, we return to looping patterns, but this time we use them on arrays and array lists. For examples, we implement a `MyIntegers class` with an array that has to be extended when it becomes full and we update our `Person` class so that friends is an array list of `Person` .

Programming Contest

To receive maximum points on this lab, submit a solution to Programming Contest (page 0) Problem E - Shopping Spree.

Netbeans Installation

We use Netbeans to complete this lab. See Java Downloads (page 0) for instuctions on installing Netbeans. Installing Netbeans provides you with 10 lab points.

Submission Requirements

Follow guidance in Sample Lab Submission (page 0) for submitting your lab points.

Sample Lab

This section provides a sample lab specication and a solution. You can follow this sample in creating your solutions to the various labs in Java Expression Labs.

Sample Lab Specification: Count Values

Write a Java `CountValues` class that does the following.

`public static int countValue(double[] da, double value)` - returns the count of `value` in `da`.

`public static void main(String[] args)` - uses a `Scanner` for input and `System.out.println` for output to accomplish the following.

```
Enter number of values in array: 5
Enter value[0] for array: 2.0
Enter value[1] for array: 4.0
Enter value[2] for array: 6.0
Enter value[3] for array: 4.0
Enter value[4] for array: 4.0
Enter search value: 4.0
The count of 4.0 is 3.
```

Sample Lab Solution: Count Values

The solution to this problem is the following code.

```
import java.util.Scanner;

public class AddDouble {

   public static double countValue(double[] da, double value) {
      int count = 0;
      for (double d : da)
         if (d == value)
            count++;
      return count;
   }

   public static void main(String[] args) {
      Scanner in = new Scanner(System.in);
      System.out.print("Enter number of values in array: ");
      int size = in.nextInt();
      System.out.println();
      double[] da = new double[size];
      for (int i = 0; i < size; i++) {
         System.out.println("Enter value[" + i + "] for array:
");
         double x = in.nextDouble();
         da[i] = x;
         System.out.println();
      }
      System.out.print("Enter search value: ");
      double value = in.nextDouble();
      System.out.println("The count of  " + value + " is " coun
tValue(da, value));
   }
}
```

Questions

Points: Answer these questions for 10 points. You have to handwrite or type the answers to get the points. For each question that you do not answer, subtract one point. If you do not answer any questions, you get negative points.

1. True/False: It's easy to insert and delete elements from the middle of an array.
2. True/False: It's easy to insert and delete elements form the middle of an `ArrayList`.
3. True/False: Arrays can be used to create large numbers of variables at once.

4. True/False: Arrays can be used to create a variable that references a large number of elements.
5. True/False: Arrays can be used in conjunction with a loop to manipulate all the array elements in a iterative way.
6. True/False: The for-each loop is a convenient way to loop through a partial number of elements in an array or an `ArrayList`.
7. True/False: Arrays are of flexible size, in that if you add more elements to the end of them, they automatically expand to make them fit.
8. True/False: `ArraysList` s are of flexible size, in that if you add more elements to the end of them, they automatically expand to make them fit.
9. True/False: You can pass them to functions, just like you can ordinary variables.
10. True/False: If you create an array called `x` with 14 elements, `x[14]` is the last valid value in the array.
11. When would you choose to use an array as part of your problem solution?
12. When would you choose to use an `ArrayList` as part of your problem solution?

Car Type Revisited Lab

Learning Objective(s):

- Practice using Netbeans to redo the Car type we created earlier.

Java Car Class – Revisited

Points: Complete this entire section for 10 points.

We are going to create a new Netbeans project that uses code we created in Car Type (page 0). We will copy the source code we already created into `Car.java` and `CarTester.java` files we create for the new Netbeans project.

1. Select File New Project. This places you in the Netbeans New Project Wizzard
2. Panel 1 – Choose Project:
    - a. Select Java
    - b. Select Java Application
    - c. Next
3. Panel 2 – Name and Location:
    - a. Enter Project Name: Lab5-2Project (or some other name if you like).
    - b. Enter Project Folder:
    - c. You will see a check in Create Main Class. We want to create a main class `CarTester` that is in the **default package**. In [Packages, ADTs](/gustycooper.github.io/mydoc_8_packages] we study packages. For now and for most of our Netbeans projects we want to place our code in the default package. At this point in Netbeans you will see something like `lab5.pkg2project.Lab52Project` . Change this to `CarTester` .
    - d. Finish
4. At this point you will be editing the `CarTester class` , which as a `main()` method. You should copy and paste the code from BlueJ `CarTester.java main()` method into the `main()` method of this file.

Use whatever editor your like. Click on the Save All icon. You will have icons indicating there are errors in `CarTester.java` . This is ok because you have not created the `Car.java` file.

5. Now right-click on the icon and select New Java Class. This creates the New Java Class panel.
    - a. Change the Class Name: to be `Car`
    - b. Finish.
6. At this point you will be editing the `Car.java` class. You should copy and paste the code from BlueJ `Car.java` into this file. Use whatever editor your like. Click on the Save All icon.
7. Use the green triangle icon at the top of Netbeans to run your project.
8. At this point, you should see something like the following displayed in the Output panel.

```
run:
Tester Program for Car ClassGas left in two cars: 20mpg
and 32mpg
Each car given 10 gallons of gas
Each car driven twice: 55 and 62 miles.
Car 20 mpg Expected: 4.15
Car 20 mpg Actual: 4.15
Car 32 mpg Expected: 6.34375
Car 32 mpg Actual: 6.34375
BUILD SUCCESSFUL (total time: 0 seconds))
```

9. Congratulations, you have successfully used Netbeans to create a new project.

Person Type Revisited Lab

Learning Objective(s):

- Practice using Netbeans to redo the Person type we created earlier.

Java Person Type – Revisited

Points: Complete this entire section for 10 points.

We are going to create a new Netbeans project that uses code we created in Person Type (page 0). We will copy the source code we already created into `Person.java` and `PersonTester.java` files we create for the new Netbeans project. Recall Person Type (page 0) had two steps – Step 1 and Step 2 - where Step 2 added code to Step 1. You should copy the code from Step 2.

1. Select File New Project. This places you in the Netbeans New Project Wizard
2. Panel 1 – Choose Project:
    - a. Select Java
    - b. Select Java Application
    - c. Next
3. Panel 2 – Name and Location:
    - a. Enter Project Name: `Lab5-1Project` (or some other name if you like).
    - b. Enter Project Folder:
    - c. You will see a check in Create Main Class. We want to create a main class `PersonTester` that is in the **default package**. In [Packages, ADTs](/gustycooper.github.io/mydoc_8_packages] we study packages. For now and for most of our Netbeans projects we want to place our code in the default package. At this point in Netbeans you will see something like `lab5.pkg2project.Lab51Project`. Change this to `PersonTester`.
    - d. Finish

4. At this point you will be editing the `PersonTester class` , which has a `main()` method. You should copy and paste the code from BlueJ `PersonTester.java main()` method into the `main()` of this file. Use whatever editor you like. Click on the Save All icon. You will have icons indicating there are errors in `PersonTester.java` . This is ok because you have not created the `Person.java` file.
5. Now right-click on the icon and select New Java Class. This creates the New Java Class panel.
    - a. Change the Class Name: to be `Person`
    - b. Finish
6. At this point you will be editing the `Person.java` class. You should copy and paste the code from BlueJ `Person.java` into this file. Use whatever editor you like. Click the Save All icon.
7. Use the green triangle icon at the top of Netbeans to run your project.
8. At this point, you should see something like the following displayed in the Output panel. NOTE: In the example below I have entered Gusty and Cooper at the two prompts. run:

```
Enter first name: Gusty
Enter last name: Cooper
Friends of Gusty Cooper
Expected: Gusty Coletta Emily
Actual:  Gusty Coletta Emily
BUILD SUCCESSFUL (total time: 10 seconds)
```

9. Now we will edit our `.java` files using Netbeans. You will use the Projects panel to locate your `Person.java` and `PersonTester.java` files. They are in which is in Source Packages. Double click on each of them. This will open tabs in the edit panel for both files. It will also the Navigator window, which shows the methods and instance variables of each class.
10. Select the `Person.java` tab in the Edit window. You can edit your `Person` class using this tab. The Navigator window shows the constructor, methods, and instance variables of the Person class.
    - a. The Edit window has lots of nice features. You will see various hints and tips provided by the editor like providing you with a selection of all String methods. You can use the Expand/Collapse icons in the editor to expand and collapse code segments. I think you will like the editor.

- b. Hover over each of the methods/instance variables in the Navigator window. Netbeans will display the JavaDoc. Pretty cool, huh? That is provided you created JavaDoc when you implemented Person for Lab 3.

11. Use the Netbeans editor to update the `Person` class. There are several updates that you will make, which are described in the following steps.
12. You should update the constructors so that the formal parameters are spelled the same as the instance variables. The following is an example of this for the constructor.

```
public Person(String firstName, String lastName) {
    this.firstName = firstName;
    ...
}
```

13. You should add a new instance variable `int age` and a new constructor with the following signature to construct a `Person` of a specific `age`. NOTE: Your original constructor should assign 25 to age.

```
public Person(String firstName, String lastName, int ag
e)
```

14. You should add a new method `hasFriend`. The following code snippet demonstrates the signature and use.

```
public boolean hasFriend(String friend)

Person p = new Person("Coletta", "Cooper", 1);
p.addFriend("Gusty");
p.addFriend("Emily");
boolean b = p.hasFriend("Gusty"); // b is true
boolean c = p.hasFriend("Mary");  // b is false
```

15. You should add a new method that has a the following signature and returns true if the person can vote, which means the person is at least 18 years old.

```
public boolean canVote()
```

16. You should add a new instance variable `private static int count` along with a corresponding getter method `public static int getCount()` that returns the number of `Person` objects constructed. Your `Person` constructors should increment `count`. The variable `count` is `static` so it is not an instance variable. Each `Person` object created has its own copy of instance variables, but there is only one copy of the variable `count`. The following code demonstrates this discussion. The variables `p1`, `p2`, and `p3` reference `Person` objects that have instance variables `firstName`, `lastName`, `friends`, and `age`. If these instance variables were `public`, we could access (for example) `p1.firstName`. Instead we have made them `private` with getter functions so we do `p1.getFirstName()`. The `Person` objects share the `static int count` and the `static int getCount`, which is accessed using the class name `Person` as `Person.getCount()`.

```
Person p1 = new Person("Coletta", "Cooper", 1);
Person p2 = new Person("Gusty", "Cooper", 22);
Person p3 = new Person("Emily", "Cooper", 25);
int numOfPersonObjects = Person.getCount(); // 3
```

17. You should update your `PersonTester class` to use these new constructors and methods. The following is an example output of my updated `PersonTester`.

```
Enter first name: Gusty
Enter last name: Cooper
Friends of Gusty Cooper
Expected: Gusty Coletta Emily
Actual: Gusty Coletta Emily
Coletta is a friend true
Emily is a friend true
Gusty is a friend true
CAT is a friend false
Gusty can vote true
Number of Person objects: 3
```

18. Congratulations, you have successfully used Netbeans to create your first project.

Looping Patterns, Arrays, ArrayLists Lab

Learning Objective(s):

- Practice using arrays and ArrayLists in the looping patterns.

Looping Patterns with Arrays and ArrayLists

Points: Complete this entire section for 25 points.

This section allows you to practice programming looping patterns. We first studied looping patterns in Loop Patterns (page 0). We first practiced writing code for loopint patterns in Loops, Ifs, Switches Lab (page 0). We continue practicing our looping patterns in this lab. These looping patterns are used over-and-over in programming. All good programmers know these patterns like the back of their hands. All of the patterns are variations of the accumulator pattern. The looping patterns in this section include the following.

- Count the number of times a value is in an array/array list.
- Find the largest value in an array/array list.
- Find the smallest value in an array/array list.
- Compute the sum of the values in an array/array list.
- Compute the product of the values in an array/array list.
- Compute the average of values in an array/array list.

You will recall that we programmed some of these patterns in Lab 4 when we studied loops. These are the same patterns, but now we use them with loops and arrays/array lists. The following shows an accumulator pattern that computes the sum of the numbers from 1 to 10 without an array, and two versions of the pattern that computes the sum of numbers in an int[] array. The first version uses a counting for loop. The second version uses a for-each (or enhanced) for loop.

```
int sum = 0;
for (int i = 1; i <= 10; i++)
   sum += i;

int[] ia = {1,2,3,4,5,6,7,8,9,10};
sum = 0;
for (int i = 0; i < ia.length; i++)
   sum += ia[i];

sum = 0;
for (int ele : ia)
   sum += ele;
```

To complete this section you will write a LoopingPatterns class that has the standard public static void main(String[] args) method. You will write additional public static void methods for each of the patterns. All of these methods can be in the same class. You should realize that this approach means that we will not create objects of LoopingPatterns. Instead, we will simply run the main() method which will call our various pattern methods. By following this style, you will better learn that Java static attribute. The overall structure of the class will be something like the following. Note this example does not show all of the methods.

```
public class LoopingPatterns {
  static void fillArray(String filename, double[] numbers)
              throws java.io.FileNotFoundException { ... }
  static void fillArrayList(String filename, ArrayList<Double>
numbers)
              throws java.io.FileNotFoundException { … }
  static class Person { …  }
  static Person oldest(ArrayList<Person> people) { ... }
  static int[] allMatches(double[] numbers, double value) { … }
  public static void main(String[] args)
         throws FileNotFoundException { ... }
}
```

Techniques to Populate Arrays and ArrayLists

In order to write programs that process arrays and array lists, you have to populate them. This subsection discussing populating arrays and array lists. Two techniques are discussed. The first technique does not read data from files. The first technique is covered in sections 6.0.1.1 through 6.0.1.4. The second technique reads data from files. The second technique is covered in sections 6.0.1.5 through 6.0.1.7.

Populating an Array When Declared

Populating an array can be done when the array is declared. The following are examples of populating an int[] and a double[].

```
int[] ia = {1,2,3,4,5,6,7,8,9,10};
double[] da = {3.2, 5, 6.9, 50000.4321};
```

This same technique can be used with multiple dimensioned arrays. The following is an example of populating a 3x3 int[][].

```
int[][] ia2d = { {1,2,3},
                 {4,5,6},
                 {7,8,9} };
```

This same technique can be used with an array that is a reference type (like Person). One technique is to create Person objects and use them as part of the initialization. The following demonstrates this technique.

```
Person gusty = new Person("Gusty");
Person coletta = new Person("Coletta");
Person [] people = {gusty, coletta, new Person("JerriAnne")};
```

Populating an Array with a Loop

You should realize that a Java array is a reference variable, similar to String, Person, Car, etc. You can declare a Java array that has an initial value of null – it does not reference an array. When you do this, you have to use the new operator to create an array object. Then you need some other technique to populate your array. The following demonstrates this with a for loop to populate an int[].

```
int[] ia;          // at this point ia's value is null
// int x = ia[0]; // generates an array index exception
ia = new int[10]; // create array object of size 10
for (int i = 0; i < ia.length; i++)
    ia[i] = i+1;  // populate ia with 1,2,3,4,5,6,7,8,9,10
```

Populating an ArrayList with a Loop

You cannot populate an ArrayList when it is declared. You will have to use some looping technique as described in the last example with arrays. The following shows how to populate an ArrayList using an array.

```
int[] ia = {1,2,3,4,5,6,7,8,9,10};
ArrayList<Integer> ial = new ArrayList<Integer>();
for (int ele : ia)
   ial.add(ele);
```

Example of main() Populating Arrays and ArrayLists

You main() method will have to declare and populate an array/array list in order to pass it to your pattern methods. The following demonstrates main() populating an array of doubles, and array list of Doubles, and passing them to their respective average methods.

```
public static void main(String[] args) {
   double[] da = {3.2, 5, 6.9, 50000.4321};
   ArrayList<Double> dal = new ArrayList<Double>();
   for (double ele : da)
      dal.add(ele);
   double a1 = average(da);
   double a2 = average(dal);
}
```

Populating an Array and ArrayLists by Reading a File

Another technique to populating arrays and array lists is to read data from file. If you use this technique, you will get to practice with File input. Connecting a Java File class to a Java Scanner class does this. We have used Java Scanner class objects to read input from the console. By connecting a Scanner to a File, you can read input from text files. You will create two static methods that will read the contents of a file. The first method reads the file into a double[] array. The second method reads the contents of a file into an ArrayList.

static void fillArray(String filename, double[] numbers)

`static void fillArray(String filename, double[] numbers)` – uses a Scanner and a File to create a fillArray() method that you can use to fill arrays from file contents for use in the subsequent methods. The signature for this method will throw a FileNotFoundException, which is shown as follows. I include the File and Scanner for your review. You have to be careful with this method so that it does not overflow the double[] array actual parameter.

```
static void fillArray(String filename, double[] numbers) throw
s
     java.io.FileNotFoundException
{
    File inputFile = new File(filename);
    Scanner inDouble = new Scanner(inputFile);
    int index = 0;
    while (inDouble.hasNextDouble()) {
        double number = inDouble.nextDouble();
        numbers[index] = number;
        index++;
    }
}
```

You should notice how fillArray() throws a java.io.fileNotFoundException. We study exceptions in Exceptions, Try (page 0). For now you can mimick this code. Since you will be calling this method from your main() method, your main() must either catch of throw the exception. For example, you can declare you main() as follows.

```
public static void main(String[] args) throws FileNotFoundExcep
tion
```

In order for you Netbeans project to be able to read a file, you must put the file in the correct folder. You should notice that Netbeans creates a folder for each of your projects. Suppose you have created a Netbeans project named LoopingPatterns to contain all of your code for section 6.0. Netbeans would have the following folder structure.

- LoopingPatterns – top-level folder for your project.

- build – this folder contains a classes folder that contains your .class files.
- nbproject – this folder contains information that Netbeans uses to build and run you code.
- src – this folder contains your .java files
- smallDoubleFile.txt – this is the text file that contains the values you want to fill your array. Notice how this file is placed in the LoopingPatterns folder.

The contents of your smallDoubleFile.txt can be something like the following.

```
100
2.0
3.2
100
5.4
7
100
```

Given the above information, you main() method can call your fillArray() method as follows. You should notice how the array numbers can hold 20 doubles, and the file smallDoubleFile.txt contains 7 values. If the size of the array numbers was too small, the fillArray() method would generate an array out of bounds exception.

```
        double[] numbers = new double[20];
        fillArray("smallDoubleFile.txt", numbers);
```

You have to make sure that your methods to compute average and the array input match. For example, if you called the method average() from 6.0.2 with the array numbers from above, you would get an incorrect average because numbers.length is 20, but numbers only contains 7 values. When Java processes the statement

```
        double[] numbers = new double[20];

Java will fill numbers with 20 values of 0.0.   This means whe
n using fillArray() you should match the number of elements in
the array with the number of data values in the file.  The foll
owing is a better example of using with fillArray() and averag
e().

```java
        double[] numbers = new double[7];
        fillArray("smallDoubleFile.txt", numbers);
        double ave = average(numbers);
```

static void fillArrayList(String filename, ArrayList numbers)

`static void fillArrayList(String filename, ArrayList<Double> numbers)` – This method is just like fillArray() except it is filling an ArrayList. This is a safer method since you can keep adding to an ArrayList. Remember that ArrayLists contain objects – not Java's primitive data types. That is why we use Double, which is the object that Java provides to box the elementary double type. This method will throws java.io.FileNotFoundException just like fillArray. The following is the code for this method. When Java executes the statement numbers.add(number); Java performs boxing to change the double number to a Double. Java also boxes in the other direction – for example number = numbers.get(0);

```
static void fillArrayList(String filename, ArrayList<Double> nu
mbers) throws java.io.FileNotFoundException
{
   File inputFile = new File(filename);
   Scanner inDouble = new Scanner(inputFile);
   int index = 0;
   while (inDouble.hasNextDouble()) {
      double number = inDouble.nextDouble();
      numbers.add(number);
      index++;
   }
}
```
```

Methods to Compute an Average from an Array and ArrayList of Doubles

You shall write the code for two average methods, one has a double[] formal parameter and the other has an ArrayList formal parameter. You should populate some arrays and ArrayLists and test your average methods.

```
static double average(double[] numbers) - returns the average o
f the doubles in the array numbers.
static double average(ArrayList<Double> numbers) - returns the
average of the doubles in the ArrayList numbers.
```

Method to Count the Number of Times a Value is in an Array/Array List

You shall write the code for two count methods, one has a double[] formal parameter and the other has an ArrayList formal parameter. You should populate arrays and ArrayLists and test your count methods.

```
static int count(double[] numbers, double value) - returns the
number of times value appears in the array numbers.

static int count(ArrayList<Double> numbers, double value) - ret
urns the number of times value appears in the ArrayList number
s.
```

Method to Find the Largest Value is in an Array/Array List

You shall write the code for two largest methods, one has a double[] formal parameter and the other has an ArrayList formal parameter. You should populate arrays and ArrayLists and test your largest methods.

```
static double largest(double[] numbers) - returns the largest v
alue in the array numbers.
static double largest(ArrayList<Double> numbers) - returns the
largest value in the ArrayList numbers.
```

Method to Find the Oldest Person in an ArrayList of Persons

You shall write the code for a method that returns the oldest Person in an ArrayList of Person. For this problem, you should create a simple static class Person inside of your LoopingPatterns class. Why does it have to be a static class? The following demonstrates the beginnings of how to put a static class Person inside of your LoopingPatterns class. You will have to create a way to populate your ArrayList. You may want to create a method fillPerson(String filename, ArrayList) to fill an ArrayList from a file. If you do pursue this, the file would contain a sequence of lines where each line has a name and age. The next 5 lines are an example of the file contents.

```
Gusty 22
Coletta 1
JerriAnne 20
Jeremy 29
Brandalee 29
```

The following code shows LoopingPatterns with a static class Person and the method that returns the oldest person in an ArrayList.

```
public static class LoopingPatterns {

   static class Person {
      private String name;
      private int age;
      Person(String name, age) {
        this.name = name;
        this.age = age;
      }
      String getName() {
        return this.name;
      }
      ... more code for person
   }

   static Person oldest(ArrayList<Person> people) {

      // put you code here to find the oldest person

   }

   public static void main(String[] args)
    throws java.io.FileNotFoundException
   {
     Person p1 = new Person("Gusty",100);
     Person p2 = new Person("Coletta",1);
     ArrayList<Person> pal = new ArrayList<Person>();
     pal.add(p1);
     pal.add(p2);
     Person p3 = oldest(pal);
     System.out.println(p3.getName());
   }
}
```

Method to Convert String to Char[] to String

`static String stringToCharToString(String s, char toReplace, char newChar)` – Use the `String` method `toCharArray` to first convert the formal parameter s to a local `char[]`. Then loop through the local `char[]` replacing each occurrence of the `char toReplace` with the `char newChar`. Finally, the method uses the `String` constructor `String(char[])` to convert the changed local `char[]` back to a `String` in order to return it.

MyIntegers Type

Learning Objective(s):

- Practice using arrays to implement a simplified ArrayList-like data structures for ints.
- Practice extending an array when the array becomes full.

Create a Java MyIntegers Class

Points: Complete this entire section for 20 points.

By completing this section, you will have a better appreciation of Java's ArrayLists, which as we know can grow and shrink. We can create Java's ArrayLists to hold any object. We will create simplified version of a ArrayLists that only contains Java ints.

Create a Java MyIntegers class that will contain a private int[] array along with various methods for manipulating the MyIntegers. The MyIntegers class will allow you to become proficient in basic array algorithms such as the following.

- Inserting / deleting elements in an array
- Extending an array when it becomes full
- Finding matches (first and all) in an array
- Finding the min / max elements in an array
- Computing the sum / product of the elements of an array

The MyIntegers class has the following instance variables, constructor, and methods. There is a brief description for each. In many cases you can choose either a counting for loop or a for each loop; however, the MyIntegers class requires a counting for loop in all looping methods. This is because we have to stop looping at count and a for each loop iterates over the entire array. The following is an example of a counting for loop that computes the sum of the integers in the int[] myIntegers.

```
int sum = 0;
for (int i = 0; i < count; i++) {
   sum += myIntegers[i];
}
```

Instance Variables

```
public int count;
```

Normally, you make instance variables as private, but in this case, we want to examine count as part of the learning experience. We can declare a MyIntegers m and examine the count as m.count and m.size().

```
public int[] myIntegers;
```

Normally, you make instance variables as private, but in this case, we want to examine myIntegers as part of the learning experience. We can declare a MyIntegers m and examine values as m.myIntegers[1] and m.get(1).

Constructor

```
MyIntegers() { // constructor which will have this code
   count = 0;
   myIntegers = new int[5]; // a small array so we can extend i
t
}
```

You will have to decide the initial value of count based upon your design.

Methods

`void add(int ele)` - Adds ele to the end of our array myIntegers. You have to make sure that myIntegers has space to hold ele. You can use the Java Arrays.copyOf() method to allocate additional space in myIntegers. The following algorithm demonstrates this.

```
count++;
if (count > myIntegers.length) {
   myIntegers = Arrays.copyOf(myIntegers, 2 * myIntegers.lengt
h);
}
```

`void add(int index, int ele)` – adds ele in position index of myIntegers.

`int get(int index)` - Returns the element in index. If you do not have an element in that index, your get() method will throw IndexOutOfBoundsException. The following code demonstrates this throw.

```
throw new IndexOutOfBoundsException("MyIntegers exception");
```

`boolean contains(int ele)` – returns true if ele is in myIntegers.

`boolean isEmpty()` - Returns true if myIntegers does not contain elements; otherwise returns false.

`int size()` - Returns size (count) of myIntegers.

`int findFirstMatch(int ele)` - Returns the index of the first element of myIntegers that matches ele. Suppose I have a variable m of type MyIntegers and m.myIntegers contains `{2,4,10,4}` . m.findFirstMatch(4) returns 1. Returns -1 if there are no matches.

`void printAllMatches(int ele)` - Prints to the console the indices of all matches of ele. Suppose I have a variable m of type MyIntegers and m.myIntegers contains `{2,4,10,4}` . m.printAllMatches(4) prints 1, 3.

`int deleteFirstMatch(int ele)` - Deletes the first element of myIntegers that matches ele. And returns the index of the element deleted. If a match was not found, returns -1. Suppose I have a variable m of type MyIntegers and m.myIntegers contains `{2, 4, 10, 4}` . The following statements demonstrate deleteFirstmatch.

```
int val = m.deleteFirstMatch(4); // val is 1
for (int i = 0; i < m.size(); i++
    System.out.print(m.get(i) + " ");  // prints 2 10 4
```

`boolean deleteAllMatches(int ele)` - Deletes all elements of myIntegers that matches ele. Returns true if any elements are deleted; otherwise returns false. Suppose I have a variable m of type MyIntegers and m.myIntegers contains `{2,4,10,4}` . The following statements demonstrate deleteAllMatches.

```
boolean b = m.deleteAllMatches(4);  // b is true
for (int i = 0; i < m.size(); i++
    System.out.print(m.get(i) + " ");  // prints 2 10
```

`int max()` - Returns the largest element of myIntegers. Suppose I have a variable m of type MyIntegers and m.myIntegers contains `{2,4,10,4}` . m.max() returns 10. Throws IndexOutOfBoundsException if myIntegers is empty.

`int min()` - Returns the smallest element of myIntegers. Suppose I have a variable m of type MyIntegers and m.myIntegers contains `{2,4,10,4}` . m.min() returns 2. Throws IndexOutOfBoundsException if myIntegers is empty.

`int sum()` - Returns the sum of the elements of myIntegers. Suppose I have a variable m of type MyIntegers and m.myIntegers contains `{2,4,10,4}` . m.sum() returns 20. Throws IndexOutOfBoundsException if myIntegers is empty.

MyIntegersTester Class

Create a MyIntegersTester class to demonstrate that your MyIntegers class is correct. Your MyIntegersTester class has to catch IndexOutOfBoundsException. The following is a simple MyIntegersTester class.

```
public class MyIntegersTester {
    public static void main(String[] args) {
        // TODO code application logic here
        MyIntegers m = new MyIntegers();
        System.out.println(m.isEmpty());
        m.add(2);
        System.out.println(m.isEmpty());
        m.add(4);
        m.add(6);
        m.add(8);
        m.add(2);
        m.printAllMatches(2);
        System.out.println(m.get(3));
        try {
            System.out.println(m.get(20));
        } catch (IndexOutOfBoundsException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Person Type - Revisited Again Lab

Learning Objective(s):

- Practice using an ArrayList to keep track of Person's friends.

Java Person Type – Revisited

Points: Complete this entire section for 20 points.

You created a Java Person class in Person Type Lab (page 0) and updated it in Person Revisited Lab (page 0). In this lab you update your Java Person class again. We will **refactor** Person, meaning we will reorganize the code where the implementation changes but the observable behavior does not. The biggest change is that you are going to keep the friends in an ArrayList of Person (i.e., ArrayList). Up until now, we have implemented friends as a ```private String```. When we call the method ```addFriend(String friend)```, we concatenated the String friend onto the String friends. For this implementation we will implement friends as an ArrayList of Person. We will have the following API on your Person class. You will notice, I have changed the API some so this is refactoring with a slight change to the API.

Constructor

`Person(String name)` - Constructs a Person. Notice we have simplified the API to include a single name. You are welcome to retain the prior constructor.

Methods

`String getName()` - Returns the name of the person

`void addFriend(Person friend)` - Add the Person friend to the ArrayList

`void unFriend(Person friend)` - Removes the Person friend from the ArrayList. If the Person friend is not a friend, this method does not do anything.

`String getFriends()` - Returns a String that is all of the Person's friends. The friends should be separated by commas.

`String getFavFriend()` - Returns a String that is the name of the first friend in ArrayList, which we will consider the favorite. If the ArrayList is empty, return the String "Name does not have any friends." where name is the name of the person.

`void myNewFavFriend(Person friend)` - Makes the Person friend the favorite friend, by moving them to the first element in the ArrayList, bumping other friends down the ArrayList. What should you do if the argument Person friend is not already a friend? Let's add them. This means is Person friend is already a friend, they get moved to the front. If the Person is not a friend, they are added to the front.

`boolean hasFriend(Person friend)` - Returns true if Person friend is in the ArrayList

This is not too complicated of a program, but you have to think about it. The first step is to visualize what a Person consists of. Consider a scenario described by the following.

- Gusty has Emily and Zachary as friends.
- Emily has Coletta and Zachary as friends.
- Zachary has Gusty as a friend.
- Coletta does not have any friends.

The following diagram of variables of type Person captures this scenario.

The following code creates this scenario and uses some of the methods.

```
Person gusty = new Person("Gusty");
Person emily = new Person("Emily");
Person zachary = new Person("Zachary");
Person coletta = new Person("Coletta");
gusty.addFriend(emily);
gusty.addFriend(zachary);
emily.addFriend(coletta);
emily.addFriend(zachary);
zachary.addFriend(gusty);
String fav = gusty.getFavFriend() // "Emily"
gusty.myNewFavFriend(zachary);
fav = gusty.getFavFriend() // "Zachary"
String gustysFriends = gusty.getFriends(); // "Zachary, Emily"
boolean b = emily.hasFriend(gusty); // false
b = emily.hasFriend(coletta); // true
zachary.unFriend(gusty);
String zacsFriends = zachary.getFriends() // ""
```

Create a PersonTester class to demonstrate that your Person class is correct.

My Trip

Learning Objective(s):

- Practice using a class that has instance variables, but does not have methods. For some problems, classes without methods are the appropriate data structure.
- Practice using the printf method.

Create a Java Program to Plot My Trip

Points: Complete this entire section for 15 points.

This lab has been created from a lab created by Ron Zacharski. Ron rides a motor bike. I ride bicycles without motors.

This summer I took a motorcycle trip through various cities. Each city has a gas station. I want to complete my trip with the fewest stops at the gas stations. The route is described as follows.

```
My house to Mesilla                 5 miles
Mesilla to Leasburg                25 miles
Leasburg to Hatch                 37 miles
Hatch to Truth                    33 miles
Truth to Winston                 40 miles
Winston to Datil                 120 miles
Datil to Grants                        70 miles
Grants to Nageezi                 30 miles
Nageezi to Blanco                 40 miles
Blanco to Dzilth                 8 miles
Dzilth to Bloomfield                40 miles
Bloomfield to Farmington         20 miles
Farmington to Shiprock                40 miles
Shiprock to Red Valley                70 miles
Red Valley to Chinle                50 miles
Chinle to Window Rock                82 miles
Window Rock to Gallup                30 miles
Gallup to Black Rock                45 miles
Black Rock to El Morro                60 miles
```

To interpret the above, it means it is 5 miles from my house to Mesilla, 25 miles from Mesilla to Leasburg, etc.

Sadly, my motorcycle only holds 2 gallons and I get 75 miles to the gallon. I would like you to write a program that processes this route and prints the cities where I should stop and get gas and the distance between those stops. The answer for the above route is the following.

```
java Winston 140 Datil 120 Dzilth 148 Shiprock 100 Chinle 120 Gallup 112
```

In writing this program, you should design it such that the information is stored in an array such that you can loop through the array to compute the answer. For example, one design consists of an array of objects where each object consists of two items.

- `String cityName;`
- `int distanceToNextCity;`

This can be a class with two public instance variables as follows.

```
public class City {
   public String cityName;
   public int distanceToNextCity;
}
```

Using the class City you can declare an array as follows.

```
City[] trip = new City[19];
trip[0] = new City("my house", 5);
trip[1] = new City("Mesilla", 25);
... complete filling in the trip[] array
```

Once you have your trip array, you can process it with a for-each loop.

```
for (City c : trip) { ... }
```

You should first construct a loop that displays your information to make sure you typed everything in correctly. You should use a System.out.printf() in this loop to get some nicely formatted output. The following is what I generated in my first loop. My printf() statements are as follows.

```
System.out.printf("%-12s %7s %7s", "City", "ToNext", "Total");
System.out.printf("%-12s %7d %7d", c.cityName, c.distanceToNext
City,
                  distance);
```

Intial table of information.

```
City          ToNext   Total
my house           5       0
Mesilla           25       5
Leasburg          37      30
Hatch             33      67
Truth             40     100
Winston          120     140
Datil             70     260
Grants            30     330
Nageezi           40     360
Blanco             8     400
Dzilth            40     408
Bloomfield        20     448
Farmington        40     468
Shiprock          70     508
Red Valley        50     578
Chinle            82     628
Window Rock       30     710
Gallup            45     740
Black Rock        60     785
El Morro           0     845
```

I also used a printf() in my second loop to get the following.

Where to get gas.

```
Winston          140
Datil            120
Dzilth           148
Shiprock         100
Chinle           120
Gallup           112
```

Passing Parameters Overview

Learning Objective(s):

- Practice passing primitive types as parameters to methods.
- Practice passing reference types as parameters to methods.
- Practice passing String types as parameters to methods.
- Create a better understanding of pass by value with primitive types.
- Create a better understanding of pass by reference with reference types.
- Create a better understanding of pass by value with String types.

Passing Parameters Labs

In this lab we create three simple programs to emphasize the passing of parameters in Java.

Programming Contest

There is not a programming contest problem associated with this lab.

Submission Requirements

Follow guidance in Sample Lab Submission (page 0) for submitting your lab points.

Sample Lab

See previous labs for a sample lab specification and solution.

Questions

Passing Primitive Types Labs

Learning Objective(s):

- Practice passing primitive types as parameters to methods.
- Create a better understanding of pass by value with primitive types.

Create a Java Program that Passes Primitive Types

Points: Complete this entire section for 10 points.

Java is pass by value. Called methods can manipulate - change, swap, do whatever - formal parameters, but their corresponded actual parameters are unchanged.

Create a class `SwapInts` that has two `public static void` methods, `swap` and `main`. The swap code is shown as follows. The `swap` method actually swaps the contents of its formal parameters. You can add a `println` at the end of the method to verify this if you like.

```
public static void swap(int x, int y) {
   int t = x;
   x = y;
   y = t;
// System.out.println("Formal Paramters - x: " + x + " y: " +
y);
}
```

Your `main` method shall call `swap` as follows. You can try addtional example calls to swap.

```
public static void main(String[] args) {
   int x = 10;
   int y = 100;
   swap(x,y);
   System.out.println("After swap");
   System.out.println("x is " + x + " and y is " + y);
}
```

Passing Reference Types Labs

Learning Objective(s):

- Practice passing reference types as parameters to methods.
- Create a better understanding of pass by reference with reference types.

Create a Java Program that Passes Reference Types

Points: Complete this entire section for 10 points.

Java is pass by value; however, when you pass a reference type, the called method can reach change the object such that the calling method's object is changed.

This lab provides code for you to use, but you are welcome to experiment by changing the code. You have to create and run this class to get the points. Notice how the `swap` method swaps the contents of the two objects created in `main` .

The `SwapReference` class is given by the following.

```
public class SwapReference {
    private static class TwoValues {
        int i;
        double x;
        public String toString() {
            return "" + i + " " + x;
        }
    }

    public static void swap(TwoValues tv1, TwoValues tv2) {
        int ti = tv1.i;
        double tx = tv1.x;
        tv1.i = tv2.i;
        tv1.x = tv2.x;
        tv2.i = ti;
        tv2.x = tx;
    }

    public static void main(String[] args) {
        TwoValues tv1 = new TwoValues();
        TwoValues tv2 = new TwoValues();
        tv1.i = 10;
        tv1.x = 10.0;
        tv2.i = 100;
        tv2.x = 100.0;
        System.out.println("Before Swap");
        System.out.println("tv1: " + tv1 + " tv2: " + tv
2);
        swap(tv1, tv2);
        System.out.println("After Swap");
        System.out.println("tv1: " + tv1 + " tv2: " + tv2);
    }
}
```

Passing String Types Labs

Learning Objective(s):

- Practice passing String types as parameters to methods.
- Create a better understanding of pass by value with String types.

Create a Java Program that Passes Reference Types

Points: Complete this entire section for 10 points.

Java is pass by value; however, when you pass a reference type, the called method can reach change the object such that the calling method's object is changed. A Java `String` is reference type, but Strings are immutable - there are no methods that change their values. Simply swapping the formal parameters is just like swapping the values of primitive types - the value of the actual parameters are not changed.

This lab provides code for you to use, but you are welcome to experiment by changing the code. You have to create and run this class to get the points. Notice how the `swap` method does not swap the contents of the two `String` s created in `main` .

The `SwapStrings` class is given by the following.

```
public class SwapStrings {
    public static void swap(String s1, String s2) {
        String t = s1;
        s1 = s2;
        s2 = t;
    }

    public static void main(String[] args) {
        String x = "Hello", y = "Goodbye";
        System.out.println("Before Swap");
        System.out.println("x: " + x + " y: " + y);
        swap(x, y);
        System.out.println("After Swap");
        System.out.println("x: " + x + " y: " + y);

    }
}
```

Exception, Try, File I/O Overview

Learning Objective(s):

- Practice writing a class that performs file I/O.
- Practice using String methods.

Exception, Try, File I/O Labs

In this lab we create three simple programs to emphasize the passing of parameters in Java.

Programming Contest

To receive maximum points on this lab, submit a solution to Programming Contest (page 0) Problem D - Palindromes.

Submission Requirements

Follow guidance in Sample Lab Submission (page 0) for submitting your lab points.

Sample Lab

See previous labs for a sample lab specification and solution.

Questions

Points: Answer these questions for 10 points. You have to handwrite or type the answers to get the points. For each question that you do not answer, subtract one point.

1. Describe checked and unchecked exceptions.
2. Give an example of a checked exception.
3. Give an example of an unchecked exception.
4. True/False: You must throw or catch an unchecked exception.

5. My code has a bug that generates a StringIndexOutOfBoundsException. Is this a checked or unchecked exception?
6. My code attempted to open a file that was not available. Is this a checked or unchecked exception?
7. True/False: Java is the only language that supports file I/O.
8. True/False: MS-Word is the only program that can read a file that has a .docx extenston.
9. True/False: I can use a Java `Scanner` to read information from a file.
10. True/False: Java's file I/O is easy to understand.

Book Lab

Learning Objective(s):

- Practice writing a class that performs file I/O.
- Practice using String methods.

Book Class

Points: Complete this entire section (Book and BookTester) for 30 points. You must include JavaDoc in your code to receive the 30 points. If you choose to omit JavaDoc, you can only receive 20 points.

This lab has been modeled after a similar lab in the Udacity Java Course (https://www.udacity.com/course/intro-to-java-programming--cs046).

The requirements for the Book class are specified by describing the Book's public interface. By conforming to this interface, you allow an independent tester to test your program. Suppose that your Book class spells the `randomWord` method as `randomword` (notice the lowercase w). Your Book class does not work with a tester created by an independent tester who would get an undefined method compile error.

A public interface for a class defines all of the methods that exist for each object of the class. The definition provides each method's signature along with a brief description of the method, where a method signature is the method's name, its return type, and all of its formal parameters.

In Interface (page 0) we study the Java `interface` which allows us to syntactically enforce an interface. For now, we use the term interface to mean the public methods of a class.

The following code snippet demonstrates constructing a String object, calling the String contains method, constructing a Book object, and calling the Book method randomWord.

```
String s1 = “This is a string”;
String s2 = new String(“This is a string”);
Book b = new Book(“aliceInWonderland.txt”,”Alice in Wonderlan
d”,”Lewis Carrol”);
boolean b1 = s1.contains(“This”); // b1 is true
boolean b2 = s2.contains(“Gusty”); // b2 is false
String randomWord = b.randonWord()
```

The `Book` class public interface is given by the following. You have to select the most appropriate String methods to implement the Book’s methods, which can be coded using without conditionals or loops.

- `public Book(String filename, String bookTitle, String author)` - This is a constructor for you Book class. The code for the constructor opens the file and reads it into a `private String` instance variable. You also have to provide the code that places the title and author of the book into private String instance variables.
- `public boolean containsTitle()` - returns true if the book’s title is contained somewhere in the text of the book.
- `public int getNumOfCharacters()` - returns the number of characters in the book. Your constructor code reads the book’s contents into a private String variable.
- `public int countWords(String word)` - returns the number of times word occurs in the book. This may seem difficult to accomplish without loop and compares, but the following example shows how to count the number of times the word `"Gusty"` is in the `String bookString`. You should work through several small examples of this algorithm to make sure you understand how the algorithm works.

  ```
  String bookString = “Gusty is in this string Gusty 2 tim
  es”;
  int length = bookString.length();
  int lengthNoGusty = bookString.replace(“Gusty”, “”).leng
  th();
  int gustyCount = (length – lengthNoGusty) / 5;
  ```

- `public int countSentences()` - returns the number of sentences in the book. You can use a simple algorithm for counting sentences. For examples, you could count all of the periods or you could count all of the periods and question mark.

- `public String randomWord(int length)` - returns a random word of length in the book. You can assume the length will be positive and less than the length of `bookString` . Be sure to return one from throughout the entire book. For the `randomWord` method you do not have to find a word that is separated by spaces because we have not studied enough of Java yet. For example, one implementation for such a nice solution requires using the `String split()` method which requires `String[]` arrays. You do not have to do this unless you desire. Instead you can simply use the String method `String substring(int beginIndex, int endIndex)` method, where you use a random number for `beginIndex` and the proper sum for `endIndex` .
- `public int firstOccurrence(String word)` - returns the `word` position of the first occurrence of the `word` in book, with -1 indicating `word` is not in book.
- `public String getSecondSentence()` - returns the second sentence of the book. Your implementation of this can assume that a period terminates the first and second sentence. You will use the `String indexOf(String s)` and the `String substring(int beginIndex, int endIndex)` methods to compute the `getSecondSentence()` method.

Book Tester

Create a `BookTester` class to test your `Book` . When you run `BookTester` , it will construct one or more Book objects, and verify that each Book method correctly implements the specification. You must perform some analysis on your chosen book to manually compute the correct expected answers for selected input values for each of your methods. Your BookTester will print the actual and expected values. The following shows a BookTester class that tests the getNumOfCharacters() method. You have to test all methods.

```
public class BookTester {
   public static void main(String[] args) {
      Book alice = new Book("aliceInWonderland.txt",
                            “Alice in Wonderland”,
                            “Lewis Carol”);
      System.out.println(“Testing “ + alice.getTitle());
      System.out.println("Num Chars Actual "+alice.getNumCharac
ters());
      System.out.println("Num Chars Expected 144331");
    }
}
```

Book Program

Points: Complete this entire section for 10 points.

Extend the `main()` function in `BookTester` so that it prompts the user for input that is used as arguments to your `Book` class constructor and methods. Your program must construct at least one `Book` object and call all of the methods. The following is an example console dialog.

```
Enter Book Name: aliceInWonderland.txt
The following are allowable commands:
characters, sentences, randomWord, countWord, findWord
Enter command: characters
144331 characters in aliceInWonderland
Enter command:
```

Second Classes and Objects Lab

Learning Objective(s): The learning objectives for the entire Second Classes and Objects Labs.

- Practice writing classes that define types.
- Practice creating instance variables, constructors, and instance methods within your class.
- Practice creating test cases for your types.
- Practice placing your test cases in tester classes that have a main method.
- Practice using your types to declare variables.
- Practice constructing objects of your types.
- Practice calling instance methods in the objects you construct.

Second Classes and Objects Lab

In Second Classes and Object Lab, we return to creating types. We update some of the types we created in First Classes and Objects Lab (page 0). We begin use the Netbean IDE in Lab 5. Section 5.0 installs Netbeans. Sections 5.1 and 5.2 uses Netbeans to re-implement the Student class and Car class from Lab 2. Section 5.3 adds a couple of methods to the Twitter class we implemented in Lab 4. This means the sections 5.1, 5.2, and 5.3 copy information from previous .java files. Section 5.4 uses some new material (subclass and interfaces).

Programming Contest

To receive maximum points on this lab, submit a solution to Programming Contest (page 0) Problem F - Three N Plus One.

Submission Requirements

Follow guidance in Sample Lab Submission (page 0) for submitting your lab points.

Sample Lab

See previous labs for a sample lab specification and solution.

Questions

Points: Answer these questions for 20 points. You have to handwrite or type the answers to get the points. For each question that you do not answer, subtract one point. If you do not answer any questions, you get negative points.

1. Object-oriented programming uses classes and objects. What are classes and what are objects? What is the relationship between classes and objects?
2. What are instance variables and instance methods?
3. Explain carefully what `null` means in Java, and why this special value is necessary.
4. What is a constructor? What is the purpose of a constructor in a class?
5. Suppose that `Kumquat` is the name of a class and that `fruit` is a variable of type `Kumquat`. What is the meaning of the statement `fruit = new Kumquat();` ? That is, what does the computer do when it executes this statement? (Try to give a complete answer. The computer does several things.)
6. What is meant by the terms instance variable and instance method?
7. Explain what is meant by the terms subclass and superclass.
8. Modify the following class so that the two instance variables are private and there is a getter method and a setter method for each instance variable:

```
public class Player {
   String name;
   int score;
}
```

9. WRONG: Explain why the class `Player` that is defined in the previous question has an instance method named `toString()`, even though no definition of this method appears in the definition of the class.

10. True/False: Two different methods can each have a variable named `x` If one of the methods changes its value of `x` this has no effect on the value of the other method's `x` variable.
11. True/False: The variables used by Java methods are stored in something called the "stack."
12. True/False: Making an instance variable `static` basically means that there is only one value for the variable shared by all objects of that type.
13. True/False: In Java, a two-dimensional array must be perfectly rectangular: all rows must be the same size.
14. The methods of a class can be called. Methods have parameters. Discuss the concept of parameters. What is the difference between formal parameters and actual parameters?
15. Explain the term polymorphism. You will have to look this up on the Internet because we have not studied in it class.
16. Java uses "garbage collection" for memory management. Explain what is meant here by garbage collection. What is the alternative to garbage collection?
17. What is an abstract class, and how can you recognize an abstract class in Java.
18. What is `this` ?
19. For this problem, you should write a very simple but complete class. The class represents a counter that counts 0, 1, 2, 3, 4, ... The name of the class should be Counter. It has one private instance variable representing the value of the counter. It has two instance methods: increment() adds one to the counter value, and getValue() returns the current counter value. Write a complete definition for the class, Counter.
20. This problem uses the Counter class from the previous question. The following program segment is meant to simulate tossing a coin 100 times. It should use two `Counter` objects, `headCount` and `tailCount` , to count the number of heads and the number of tails. Fill in the blanks so that it will do so:

```
Counter headCount, tailCount;
tailCount = new Counter();
headCount = new Counter();
for(intflip=0; flip<100; flip++){
   if (Math.random() < 0.5)  // There's a 50/50 chance t
his is true.
        ____________;   // Count a "head".
    else
        ____________;   // Count a "tail".
;
System.out.println("There were " + ___________ + " head
s."); System.out.println("There were " + ___________ +
" tails.");
```

21. True/False: Java's `final` keyword does not have much use. Explain your answer.

Twitter Type Revisited Lab

Learning Objective(s):

- Practice maintenance programming on our Twitter class to add some exciting new features.
- Practice writing classes that define types.
- Practice creating instance variables, constructors, and instance methods within your class.
- Practice creating test cases for your types.
- Practice placing your test cases in tester classes that have a main method.
- Practice using your types to declare variables.
- Practice constructing objects of your types.
- Practice calling instance methods in the objects you construct.

Java Twitter Type – Revisited

Points: Complete this entire section for 30 points.

NOTE: The Twitter Type is a rendition of a project from a Udacity class.

In Twitter Type (page 0) we created a `Twitter class` . That lab has been created by mashing together a similar lab in the Udacity Java Course (https://www.udacity.com/course/intro-to-java-programming--cs046) and some methods from a lab created by Ron Zacharski. This lab will use Netbeans to copy our existing `Twitter class` , add some exciting new methods to our `Twitter class` , and create a `TwitterTester class` .

Using Netbeans, perform the following sequence of steps that solves a problem.

1. Select File New Project. This places you in the Netbeans New Project Wizard
2. Panel 1 – Choose Project:
    - a. Select Java
    - b. Select Java Application
    - c. Next
3. Panel 2 – Name and Location:

- a. Enter Project Name: `TwitterProject` (or some other name if you like).
- b. Enter Project Folder:
- c. You will see a check in Create Main Class. We want to create a main class `PersonTester` that is in the **default package**. In [Packages, ADTs](/gustycooper.github.io/mydoc_8_packages] we study packages. For now and for most of our Netbeans projects we want to place our code in the default package. At this point in Netbeans you will see something like `lab5.pkg2project.TwitterProject`. Change this to `PersonTester`.
- d. Finish

4. At this point you will be editing the `PersonTester class`, which has a `main()` method. You should copy and paste the code from BlueJ `PersonTester.java main()` method into the `main()` of this file. Use whatever editor you like. Click on the Save All icon. You will have icons indicating there are errors in `PersonTester.java`. This is ok because you have not created the `Person.java` file.
5. Now right-click on the icon and select New Java Class. This creates the New Java Class panel.
    - a. Change the Class Name to be `Twitter`
    - b. Finish.
6. At this point you will be editing the `Twitter.java class`. You should copy and paste the code from BlueJ Twitter Type (page 0) `Twitter.java` into this file. Use whatever editor your like. Click on the Save All icon.
7. Now edit `TwitterTester.java` to construct a `Twitter` and use all of the methods. This will make sure you `Twitter.java` works before adding two new methods. Here is the output from running my `TwitterTester`. You will notice that my tweet is actually the sentence, `“This is my Tweet”`.

```
run:
My tweet is This is my Tweet
My tweet reversed is teewT ym si sihT
Number of vowels in my tweet are 4
My tweet without vowels is Ths s my Twt
BUILD SUCCESSFUL (total time: 0 seconds)
```

8. Add two new methods to `Twitter.java` .
    - a. `simplePalindrome()` – returns true if word is a palindrome. This method will consider palindromes without spaces. A palindrome is a string that reads the same backwards as forwards. For example `"madamImadam"` and `"risetovotesir"` are simple palindromes.
    - b. `palindrome()` – returns true if word is a palindrome. Typically spaces, capitalization and punctuation are ignored in palindromes, meaning `"Madam I'm Adam"` and `"Rise to vote sir"` are considered palindromes. `simplePalindrome()` did not have to consider these; however, `palindrome()` will recognize these are palindromes. For example, the following is a good palindrome: `“Madam ! I’m ? Adam”` . You should realize that for the `palindrome()` method, you have to remove characters from the string, and the call `simplePalindrome()` . You can remove characters from a string by the following algorithm. You should make sure you understand this algorithm before incorporating it into your code. The best way to understand the algorithm is to exercise the algorithm in your mind using small tweets. This algorithm would be in your `Twitter.java` and the variable tweet is the private variable representing the tweet.

```
java String removeChars = " .!?,()'"; String tweetToChange = this.tweet; f
```

9. Update your `TwitterTester.java` class to adequately test `simplePalindrome()` and `palindrome()` . One option would be to update `TwitterTester.java` such that it has does the following.
    - Read in a `String` variable from the user using a `Scanner` object.
    - Construct several `Twitter` (s) using the user’s input, and use `simplePalindrome()` , `palindrome()` methods, and other `Twitter` methods on the user’s input.
10. The following are example runs of your program.

```
run:
My tweet is This is my Tweet
My tweet reversed is teewT ym si sihT
Number of vowels in my tweet are 4
My tweet without vowels is Ths s my Twt
Is madam I madam a palindrome true it is.
Is madam    I'm ..! adam a palindrome? true it is
Enter a potential palindrom: Gusty     !!!! ytsuG
Gusty     !!!! ytsuG palindrome status is true
BUILD SUCCESSFUL (total time: 16 seconds):
```

Student Type Lab

Learning Objective(s):

- Practice writing a Student class that extends our favorite Person class.
- Practice determining the type of a variable using instanceof.
- Practice writing classes that define types.
- Practice creating instance variables, constructors, and instance methods within your class.
- Practice creating test cases for your types.
- Practice placing your test cases in tester classes that have a main method.
- Practice using your types to declare variables.
- Practice constructing objects of your types.
- Practice calling instance methods in the objects you construct.

Java Student Type – Subclass of Person

Points: Complete this entire section for 30 points.

We have practiced creating a `Person` type in Person Type Lab (page 0) and Person Type - Revisited Lab (page 0). This lab uses Netbeans to create a project that will copy the code from the `Person class` in Person Type - Revisited Lsb (page 0) and add a `Student class` that `extends` the `Person class`. Additionally, you will create a `StudentTester class` that has a `main()` to test you `Student Class`.

Follow the steps 1 through 4 of 5.2 to create a `StudentTester class`, which will have a `main()` method. Do not forget 3, d where you will change the name in the Create Main Class entry to be `StudentTester`. This will result in an edit tab for `StudentTester.java`, which has a `main()` function. For now leave the main() method without adding any new code.

Using Netbeans, perform the following sequence of steps that solves a problem.

1. Select File New Project. This places you in the Netbeans New Project Wizard
2. Panel 1 – Choose Project:

    - a. Select Java
    - b. Select Java Application
    - c. Next
3. Panel 2 – Name and Location:
    - a. Enter Project Name: `StudentProject` (or some other name if you like).
    - b. Enter Project Folder:
    - c. You will see a check in Create Main Class. We want to create a main class `StudentTester` that is in the **default package**. In [Packages, ADTs](/gustycooper.github.io/mydoc_8_packages] we study packages. For now and for most of our Netbeans projects we want to place our code in the default package. At this point in Netbeans you will see something like `lab5.pkg2project.TwitterProject`. Change this to `PersonTester`.
    - d. Finish
4. At this point you will be editing the `StudentTester class`, which has a `main()` method. You will have icons indicating there are errors in `StudentTester.java`. This is ok because you have not created the `Student.java` file.
5. Now right-click on the icon and select New Java Class. This creates the New Java Class panel.
    - a. Change the Class Name to be `Person`
    - b. Finish.
6. At this point you will be editing the `Person.java` class. You should copy and paste the code from Person Type - Revisited (page 0) Person.java into this file. After you have pasted the code into the file, click save all.
7. Now right-click on the icon and select New Java Interface. This creates the New Java Interface panel.
    - a. Change the Class Name: to be `BannerInterface`
    - b. Finish.
8. At this point you will be editing the `BannerInterface interface`. One attribute that college students have is a grade point average (GPA). You will create a simple interface that has two public methods for manipulating a GPA. The methods are defined as follows. After you have finished editing your BannerInterface, click save all.

```
public void setGpa(double gpa);
public double getGpa();
```

9. Now right-click on the icon and select New Java Class. This creates the New Java Class panel.
    - a. Change the Class Name to be `Student`
    - b. Finish.
10. At this point you will be editing the `Student.java class` . You should create your `Student class` such that it `extends Person` and `implements BannerInterface` . Your implementation of `Student` will not be many lines of code. `Student` will have the following public constructors and methods.

```
public Student(String firstName, String lastName)
public Student(String firstname, String lastName, int ag
e)
public Student(String firstname, String lastName, int ag
e,
               double gpa)
public void setGpa(double gpa)
public double getGpa()
```

11. Now you should return to editing your `StudentTester.java class` . You should construct some objects of type `Person` and type `Student` . You should give this people some friends. You should exercise the various methods of your objects. You should realize that you can assign a `Student` to a `Person` . The following is examples of constructors and output from my `StudentTester` .

```
Student gusty = new Student("Gusty", "Cooper", 39,
3.5);;
Person coletta = new Person("Coletta", "Cooper", 1);
Student brandalee = new Student("Brandalee", "Cooper", 2
8, 3.9);
Person jerrianne = new Person("JerriAnne", "Cooper", 3
7);


Gusty has friends: Coletta Brandalee JerriAnne
Gusty can vote: true
Gusty GPA is 3.5

Coletta has friends: Gusty Brandalee JerriAnne
Coletta can vote: false

Brandalee has friends: Coletta
Brandalee can vote: true
Brandalee GPA is 3.9

JerriAnne has friends:
JerriAnne can vote: true
coletta = gusty; // You can assign a Student to a Person
The following shows coletta's first name after the assig
nment.
coletta.getFirstName is Gusty
```

Person Type One More Time Lab

Learning Objective(s):

- Practice writing enum types.
- Practice using enum types in conjuction with other types.
- Practice writing classes that define types.
- Practice creating instance variables, constructors, and instance methods within your class.
- Practice creating test cases for your types.
- Practice placing your test cases in tester classes that have a main method.
- Practice using your types to declare variables.
- Practice constructing objects of your types.
- Practice calling instance methods in the objects you construct.

Java Person Class – Revisited One More Time

Points: Complete this entire section for 30 points.

This lab uses the Person class from Person Revisited (page 0) and the `Student` subclass (of `Person`) along with the `Banner` interface from Student Type Lab (page 0).

1. Begin with the Java Person class from Person Revisited (page 0). This is the `Person` class that has `friends` that implemented as an `ArrayList<Person>` along with all of the methods defined.
2. Override the Java Object method `toString()` method for `Person` so that it prints the name of the person.
3. Update your `Student` class that implements the `Banner` interface so that it extends the `Person` class from step 1.
4. Implement a `Dog` class that has a `DogType` enum that defines the following types of dogs: `HOUND`, `COLLIE`, `GERMAN_SHEPARD`, and `POODLE`. You may add more types if you like. You Dog class should define the DogType as public so that users of the class can do the following. Your Dog class should satisfy the following API.
    - a. `public Dog(String name, int age, DogType type)` – Constructor where age is people years.

- b. `public String getName()` – returns the name of the dog.
- c. `public DogType getType()` – returns the type of the dog.
- d. `public int getAge()` – returns the age of the dog in people years.
- e. `public int getDogAge()` – returns the age of the dog in dog years where a people year is equal to 7 dog years.

```
java Dog d = new Dog("Gusty", 5, Dog.DogType.HOUND);
```

5. Add the following methods to your Person class.
    - a. `void addDog(Dog dog)` – adds the Dog dog to the Person. You can think of this as the Person now has a pet dog. You can add as many pet dogs as you lik.
    - b. `boolean hasDog()` – returns true of the Person has a pet dog, otherwise returns false
6. Since you the methods `addDog()` and `hasDog()` are `Person` methods, your `Students` can have dogs for pets.
7. Create a `PersonTester` class that creates `Person` s and `Student` s that allows you to do the following. You can use your imagination to add to this scenario.
    - a. You will create five Persons, some of which are Students: Gusty, Jeremy, Brandalee, Coletta, and Emily
    - b. Gusty is a Person, but not a Student. Gusty does not have any dogs.
    - c. Jeremy is a Person, but not a Student. Jeremy has two Dogs. One is a poodle named Dot, who is 4 people years old. The other is named collie named Elle who is 3 people years old.
    - d. Brandalee is a Student with a GPA of 3.8. Brandalee has the same two Dogs as Jeremy.
    - e. Coletta is a Person, but not a Student.
    - f. Emily is a Student with a GPA of 3.6. Emily does not have any dogs.
    - g. Gusty's friends are Jeremy, Brandalee, and Coletta. Gusty's favorite friend is Coletta.
    - h. Emily has Jeremy and Coletta as friends. Emily's favorite friend is Jeremy.

- i. Coletta's friends are Jeremy, Brandalee, Gusty, and Emily. Coletta's favorite friend is Gusty.
- j. Brandalee's friends are Jeremy and Coletta. Brandalee's favorite friend is Coletta.
- k. Jeremy's friends are Brandalee, Coletta, and Emily. Jeremy's favorite friend is Brandalle.
- l. Create an array of Persons – in the following, I use Person[] people as the name of this array. Put Gusty, Brandalee, Jeremy, Coletta, and Emily in the array. You should notice that Brandalee and Emily are Students, but they can be placed in the array.
- m. Create a for-each loop that loops through the array of Persons. Print the name of each Person. If the Person is an instance of a Student, print the GPA. Since you have overridden the toString() method, given that the you can do this by the following.

```
for (Person p : people) {
    System.out.println(p)
    if (p instanceof Student)
       System.out.println(p.getFirstName() + " GP
A is " +
                           ((Student)p).getGpa());
}
```

- n. Create a for-each loop that loops through the array of Persons. On each iteration of the loop do the following.
    - i. Print the favorite friend of the Person.
    - ii. Print the favorite friend of the favorite friend of the Person.
    - iii. Print all Dogs that the Person has as pets.
    - iv. Print all Dogs of the favorite friend of the Person.
    - v. Print the age in Dog years of all poodles that any person may have as a pet.

Packages, ADTs Labs

Learning Objective(s):

- Practice creating abstract data types.
- Practice sovling problems with a stack ADT.
- Practice creating a Java package.

Abstract Data Types, Packages, Interfaces Overview

In this lab we create a Stack ADT, use the Stack to solve a couple of problems, and create a Java package.

Programming Contest

To receive maximum points on this lab, submit a solution to Programming Contest (page 0) Problem G - Paired Isograms.

Submission Requirements

Follow guidance in Sample Lab Submission (page 0) for submitting your lab points.

Sample Lab

See previous labs for a sample lab specification and solution.

Questions

Points: To earn these 10 points, you have to write or type answers to 4 questions; otherwise you receive 0 points.

1. What is an abstract data type?
2. Compare ADT's to Java Elementary Data Types.
3. Explain a Java Interface.

4. Discuss Java packages and how they are used in software development.

Stack ADT Lab

Learning Objective(s):

- Practice creating a stack ADT.
- Practice creating generic types - if desired.

Create a Stack ADT

Points: Complete this entire section for 30 points.

In this lab you will implement a Stack ADT. You will create an interface, implement the class, and implement a tester of your ADT. You will create the StackInterface.java, Stack.java, and StackTester.java. These are described below. There are two versions of the Stack ADT. One version allows pushes and pops of char. This is the simpler of the two versions. The other version is a Java generic type, which is a class that has been parameterized over types. You can choose which version you want to implement. Perhaps you want to implement the simpler char stack first to get a better understanding of a stack, and then implement the generic stack. Either implementation can be used to solve the problems in Palindrome Checker Lab (page 0) and Parentheses Balance Checker Lab (page 0).

I will lead you through this lab. In doing so, we will define our own excpetions, `EmptyStackException` and `FullStackException`.

See code in
`Google Drive > 00UMW > 1Java > 0Class > Netbeans > Stack > src`.

StackInterface.java

The interface for a stack of `char` is given by the following.

```
public interface StackInterface {
    /**
     * pushes an char onto the stack
     * @param ele the char to be pushed
     */
    public void push(char ele);
   /**
     * pops a char from the stack
     * @return returns the top char on the stack
     */
    public char pop();
   /**
     * peeks at the top of the stack
     * @return returns the top char on the stack but it stays o
n top
     */
    public char pop();

   /**
     *
     * @return true if the stack is empty
     */
    public boolean isEmpty();
}
```

The generic stack interface is given by the following.

```
public interface StackInterface<E> {
    /**
     * pushes an element onto the stack
     * @param E the element to be pushed
     */
    public void push(E ele);
   /**
     * pops an element from the stack
     * @return returns the top E ele on the stack
     */
    public E pop();
    public E pop();
   /**
     * peeks at the top of the stack
     * @return returns the top E ele on the stack but it stays
on top
     */
    public char pop();
   /**
     *
     * @return true if the stack is empty
     */
    public boolean isEmpty();
}
```

Stack.java

Your `Stack` class will implement the `StackInterface` described in the previous section.

If you are implementing a char stack, your `Stack` class will be the following.

```
public class Stack implements StackInterface { /* your code */
}
```

Declaring variables of type `Stack` and constructing objects is just like what we have been doing. Given you have the constructor described below you would declare and construct a Stack as follows.

```
Stack s = new Stack(10);  // Stack holds 10 chars
s.push('G');
s.push('u');
s.push('s');
while (!s.isEmpty())  // prints suG
    System.out.print(s.pop());
```

If you are implementing the generic stack, your Stack class will be the following.

```
public class Stack<E> implements StackInterface<E> { /* your co
de */ }
```

To reference the generic Stack class in your code, you must perform a generic type invocation, which replaces E with some concrete value, such as Integer. Note the type must not be a primitive type.

```
Stack<Integer> integerStack = new Stack<Integer>();
```

You can think of a generic type invocation as being similar to an ordinary method invocation, but instead of passing an argument to a method, you are passing a type argument — `Integer` in this case — to the `Stack` class itself.

You can use whatever means you desire for you implementation. The only rules are (1) you have to implement the interface and (2) you have to implement the semantics of a stack. You can even assume that the users of your Stack will be nice – for example, if your implementation uses an array, you can assume that user will not push more elements than the array will hold. Consider the following example implementations.

Example Implementation 1 – char Stack

The stack is implemented in a `private char[]` elements. For this implementation you could provide a constructor `Stack(int size)` where `size` is the number of elements you stack can hold. If you pursue this path, your class constructor and `push` method could look like the following. Realize this simple example does not perform error checking – for example, the user could `push` more elements than the stack can hold. Perhaps you want to include some error checking in your implementation.

```
public class Stack implments StackInterface {
private char[] elements;
private final int size;
private int top;
public Stack(int size) {
    size = s > 0 ? s : 10;
    top = -1;
    elements = new char[size];
}
public void push(char ele) {
    // use size to throw FullStackException exception
    elements(++top) = ele;
}
public char pop() {
  if (!isEmpty())
     return elements(--top);
  else
      return '\0';  // or throw EmptyStackException exception

...
}
```

Example Implementation 2 – generic Stack

The stack is implemented as a `private` array of objects. For this implementation, you could provide a constructor `Stack(int size)` where `size` is the number of elements you stack can hold. If you pursue this path, your class constructor and push method could look like the following. Realize this simple example does not perform error checking – for example, the user could push more elements than the stack can hold. Perhaps you want to include some error checking in your implementation.

```
public class Stack<E> implements StackInterface<E> {
    private final int size;
    private int top;
    private E[] elements;

    public Stack() {
        this(10);
    }

    public Stack(int s) {
        size = s > 0 ? s : 10;
        top = -1;

        elements = (E[]) new Object[size]; // create array
    }

    public void push(E pushValue) {
        if (top == size - 1) // if stack is full
          throw new FullStackException(String.format("Stack is
full, cannot push %s", pushValue));

        elements[++top] = pushValue; // place pushValue on Stac
k
    }

// more code
}
```

Example Implementation 3 – Using a linked list like we did for `Queue` in class

The stack is implemented as a linked list similar to the way we implemented a Queue in class. This results in a private inner class something like the following.

```
   private class Node {
      private E item;
      private Node next;
   }
```

StackTester.java

You are to implement a tester of your `Stack.java` in a file `StackTester.java`. You should think of various test cases to demonstrate that you have successfully implemented a `Stack` ADT. You should create stacks with various types to demonstrate you have successfully implemented

If you are test a generic Stack, you should realize that you cannot create a `Stack<int> si = new Stack<int>(10)` because the elements must be an Object. Recall that Java will box primitive data types into an out of Objects for you so it often appears as if you can put primitive data types into classes such as our `Stack` class. The following is an example of boxing into and out of `ArrayList<Double>`.

```
ArrayList<Double> ald = new ArrayList<Double>();
ald.add(2.0);
ald.add(4.0);
double d = ald.get(0);
```

Palindrome Checker Lab

Learning Objective(s):

- Practice solving the palindrome problem with a stack ADT.

Palindrome Checker

Points: Complete this entire section for 15 points.

We added `palindrome` methods to our Twitter class in Twitter Revisited (page 0). This lab will use our `Stack` ADT to perform palindrome checking. You can see how this is done be considering the palindrome hannah. We know the length to be 6 so we will have to push 3 characters and pop 3 characters. For strings with an even number of characters, you will push half and pop half. For strings with an odd number of characters, you will push up to the middle character, skip the middle character, and then pop the rest. I show a simple algorithm as a sequence of statements. You have to capture this algorithm in a method that uses a loop so that it works with an arbitrary string. For example the method signature should be something like the following.

```
boolen palindromeChecker(String s) // return true if s is palin
drome
```

The algorithm as a sequence of statements for checking the String “hannah” is as follows.

```
Stack<Character> s = new Stack<Character>(10);
s.push(‘h’);
s.push(‘a’);
s.push(‘n’);
‘n’ == s.pop(); // true
‘a’ == s.pop(); // true
‘h’ == s.pop(); // true
```

You should ignore punctuation, capitalization, and spaces. For example, `"Madam, I'm Adam is a palindrome"` is a palindrome. You should be able to process palindromes with an odd number of characters. For example `"Ada"` is a palindrome with 3 characters.

You have choices as to how you want to implement your palindrome checker. For example you could add the method to your `StackTester.java` from Stack ADT (page 0) or you could update the `palindrome` methods in your `Twitter` class from Twitter Revisited (page 0) or you can create a new class.

Parentheses Balance Checker Lab

Learning Objective(s):

- Practice solving the balanced parantheses problem with a stack ADT.

Parentheses Balance Checker

Points: Complete this entire section for 15 points.

For this lab, use your Stack ADT to implement a parentheses balance checker. In programming, parentheses have to be balanced, meaning every left parenthesis has a matching right parenthesis. You should be able to mix other characters in with the parentheses. These characters will be ignored. You should only be pushing and popping the parentheses. The following are some example input lines.

```
(((()))      // balanced
(((()        // unbalanced
(()(())(())  // balanced
(()()        // unbalanced
```

MyMath Package Lab

Learning Objective(s):
- Practice creating a Java package.
- Practice importing a package that you create.

Java MyMath Class – Revisited with Packages

Points: Complete this entire section for 20 points.

In this lab, you will create a package in which will be a `MyMath` class that is a math library similar to Math. We are going to create a new Netbeans project that uses code we created in Lab 4.5. We will create a new project and we will copy and modify the source code we already created into our new MyMath.java file for this new Netbeans project. Your `MyMath` should include the following methods.

- `public static double average(double[] da)` – returns the average of da.
- `public static double largest(double[] da)` – returns the largest in da.
- `public static int numDigits(int x)` – returns the number of digits x.
- `public static int coins(int cents)` – returns the mimimum coins to make cents

Creating MyMath in a package mymath

1. Select File New Project. This places you in the Netbeans New Project Wizzard
2. Panel 1 – Choose Project:
    - a. Select Java
    - b. Select Java Application
    - c. Next
3. Panel 2 – Name and Location:
    - a. Enter Project Name: MyMath.

- b. Enter Project Folder:
- c. You will see a check in Create Main Class. The value in this will be something like the following, which you should Change this to be mymath.MyMath. We want to create a mymath package in which to place our MyMath class.

```
lab10.pkg2project.Lab101Project
```

- d. Finish

4. This will result in an edit tab for MyMath.java, which has a `package mymath;` statement followed by a main() function. You do not want a main() in MyMath.java because we are going to create a MyMath class in the package mymath. Later we will be able to import our MyMath class by using the following statement.

   `java import mymath.MyMath;`

5. Delete the main() code from MyMath.java.
6. Copy the code from MyMath Take 2 (page 0) into the MyMath.java tab. You will have to update the average and largest methods because I have changed the signature so that the formal parameter is a double[]. Make sure all of the methods are static. For example the numDigits() method should have the following signature. We want our functions to be static because they are not dependent upon creating an object of type MyMath. If you think about the Java Math class, it has methods and constants that you can access without creating objects of type Math. All you have to do is use the Math.sin() function call. As you should know by this time, a static method does not require creating an object. In this way our MyMath class will be mimicking the Java Math class. We want to be able to accomplish the following. You should notice how the above import is similar to the imports we have done in order to access various Java classes. For examples,

```
public static int numDigits(int x)

import mymath.MyMath;
int n = MyMath.numDigits(105);

import java.io.File;
import java.util.Scanner;
import javax.swing.JButton;
```

7. Right-click on MyMath project and select Clean and Build. This will create a JAR file in the distribution (or dist) folder the Netbeans maintains for you. This JAR file is what we will import in the Part B.

Creating UseMyMath to import mymath.MyMath

1. Select File New Project. This places you in the Netbeans New Project Wizzard
2. Panel 1 – Choose Project:
    - a. Select Java
    - b. Select Java Application
    - c. Next
3. Panel 2 – Name and Location:
    - a. Enter Project Name: UseMyMath (or some other name if you like).
    - b. Enter Project Folder:
    - c. You will see a check in Create Main Class. The value in this will be something like the following. Change this to be UseMyMath.

    ```
    java usemymath.UseMyMath
    ```

- d. Finish
    1. This will result in an edit tab for MyMath.java, which has a main() method, which we will edit.
    2. In this step we will add our mymath package to the Libraries used by our UseMyMath project. Right-click on the Libraries Icon under our UseMyMath project and select Add Project…. This will provide you a selection box with our projects. Select MyMath and Add Project Jar Files.
    3. Edit the UseMyMath.java tab to add the following import statement.

```
```java
import mymath.MyMath;
```
```

1. We can now call the methods in our `MyMath` class. Since we made these methods `static`, we do not declare objects. We simply access them as we do those in Java's Math library. The following are a couple of statements you can place in your main().

```
System.out.println("Largest of 1,2: "+MyMath.larges
t(1,2));
System.out.println("Num digits in 10023: "+MyMath.numDig
its(10023));
System.out.println("Average of 1,2,3,4: "+MyMath.averag
e(1,2,3,4));
System.out.println("Num coins in 99: "+MyMath.coins(9
9));
```

2. Update your `main()` to use all of your `MyMath` methods.
3. If you click on the Libraries icon of the UseMyMath project, you will notice MyMath.jar file is one of the Libraries. MyMath.jar contains the MyMath class you created in Part A.

Graphics

Learning Objective(s): Graphics

Graphics

Graphics is divided into two parts - Simple Graphics and Graphics Events. Files (.java) for each part are provided on Canvas.

Files > Handouts > 10Graphcs > GraphicsCode > SimpleGraphics contains .java source files that can be used when completing Simple Graphics lab. You may download the files, incorporate them into your JDK, study them, run them, and modify them. You should study the code so that you understand the basic workings of graphics. If you simply download and run them without study, you will not learn.

Files > Handouts > 10Graphics > GraphicsCode > GraphicsEvents contains .java files that can be used when completing Graphics Events lab. You may download the files, incorporate them into your JDK, study them, run them, and modify them. You should study the code so that you understand the basic workings of graphics. If you simply download and run them without study, you will not learn.

Programming Contest

To receive maximum points on this lab, submit a solution to Programming Contest (page 0) Problem H - Permutation Array.

Submission Requirements

Follow guidance in Sample Lab Submission (page 0) for submitting your lab points.

Sample Lab

See previous labs for a sample lab specification and solution.

Questions

Graphics Repeat

Learning Objective(s): Graphics Repeat

Graphics Repeat

Points: Complete this entire section for 5 points.

Implement the `JFrameOnly.java` , `BasicJFrame.java` , `BasicJPanel.java` classes that we discussed in class. You will find all three files on Canvas.

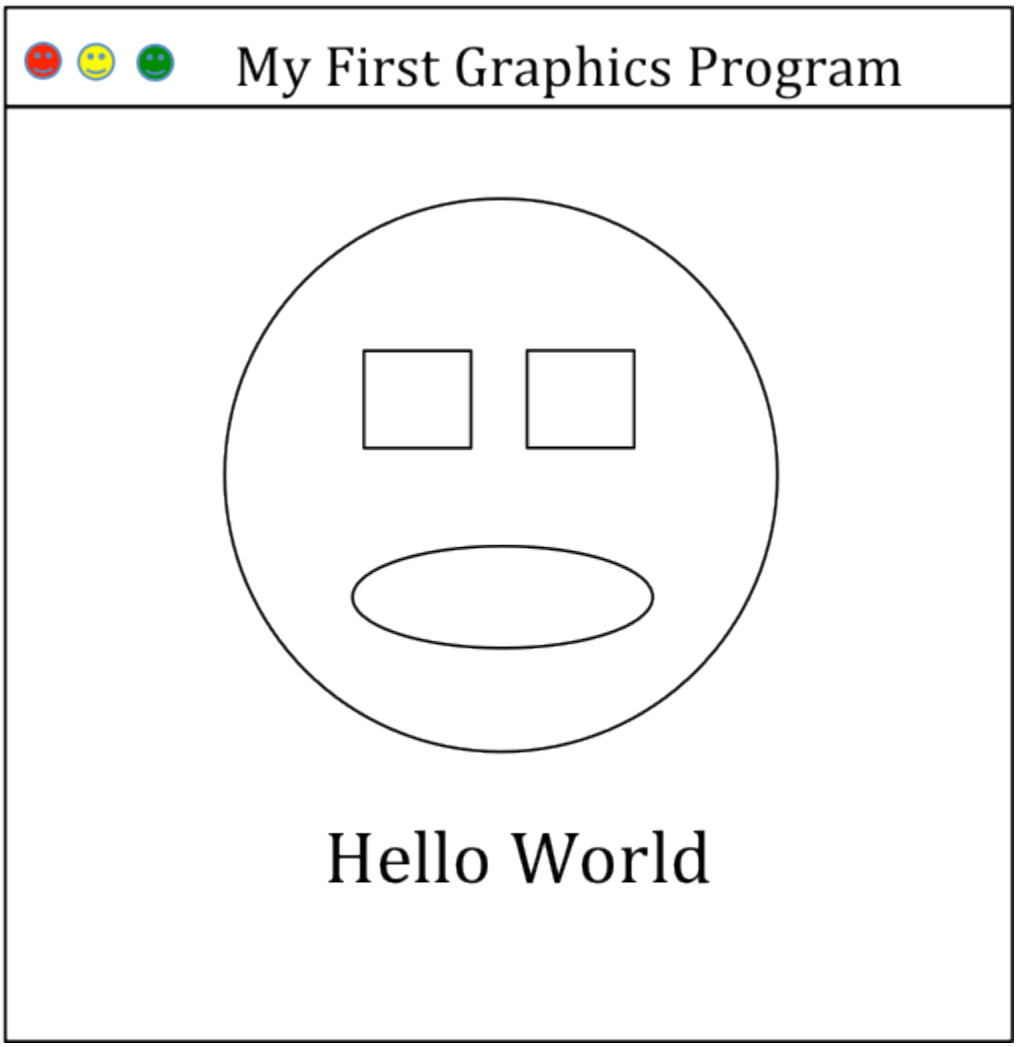
First Graphics Program

Learning Objective(s): First Graphics Program

First Graphics Program

Points: Complete this entire section for 10 points.

Create a Java Graphics program that has a `JPanel` added to a `JFrame`. Overide the `JPanel's paintComponent()` method so that it draws the following window. You can fill you eyes and mouth with a color if you like. You are also welcome to add other shapes and text to your window.



Growing Circle Animation

Learning Objective(s): Growing Circle Animation

Growing Circle Animation

Points: Complete this entire section for 5 points.

Implement the `GrowingCircleAnimation` as defined in our textbook Section 5.3.3. Realize that this program consists of two classes, `GrowingCircleAnimation.java` and `CircleInfo.java`. Both files are on Canvas. You will need to put both of these into Netbeans. Some notes for understanding this code are provided as follows.

1. `GrowingCircleAnimation extends JPanel` and `implements ActionListener`. Extending `JPanel` allows `GrowingCircleAnimation` overrides the `PaintComponent(Graphics g)` method, which gets called every time our window requires painting.
2. The `main()` method in `GrowingCircleAnimation` establishes a `Swing Timer`, which is named `frameTimer`. This `frameTimer` calls the `actionPerformed()` method of our `GrowingCircleAnimation` object `drawingArea` every 20 milliseconds.
3. The `actionPerformed()` method in `GrowingCircleAnimation` calls the AWT `repaint()` method.
4. The AWT `repaint()` method registers an asynchronous request to AWT that our component (which is `drawingArea`) needs to be repainted.
5. AWT will then call the `paintComponent()` method of `drawingArea`.
6. The `paintComponent()` method calls the `drawFrame()` method which calls the 100 `circleData[i].draw(g)` methods causing all of the circles to be redrawn.

Simple Animation Starter

Learning Objective(s): Simple Animation Starter

Simple Animation Starter

Points: Complete this entire section for 10 points.

Our textbook provides the `class SimpleAnimationStarter` in the file `SimpleAnimationStarter.java`. `SimpleAnimationStarter` is similar to `GrowingCircleAnimation` ; however, the skeleton code prints the frame number as it counts upwards. Use `SimpleAnimationStarter` to construct your own simple animation. The `SimpleAnimationStarter.java` on Canvas is the one demonstrated in class. You should think of some other animation. Perhaps a filled circle that orbits some object.

Using the `SimpleAnimationStarter` as the basis for Project 5 can be a good idea. The following are examples created by former students. Both projects are in `Google Drive > 00UMW > 1Java > 0Class > SimpleGraphics`

- `OrbitingPlanets.java` - created by John Pruchnic
- `ProjectGUI.java` and `CelestialObject.java` - created by Billy McCord. Billy began with John's code

Java Fonts

Learning Objective(s): Java Fonts

Java Fonts

Points: Complete this entire section for 5 points.

Modify the `RandomStringWithMain.java` file. The goal of this section is for you to manipulate Java `Fonts`. You should change the program so that it has 7 random fonts instead of 5. You should also change the program so that it places 32 versions of the string in the window.

CGTemplate

Learning Objective(s): Java Fonts

CGTemplate

Points: Complete this entire section for 10 points.

Study the `CGTemplate.java` to understand how it works. Update the file so that it draws additional shapes in various colors.

Hello World Repeat

Learning Objective(s): Hello World Repeat

Hello World GUI Repeat

Points: Complete this entire section for 5 points.

Use the `HelloWorldGUI1.java` and `HelloWorldGUI2.java` files to accomplish the following.

- First execute the `HelloWorldGUI1.java` and `HelloWorldGUI2.java` .
- Change the string in both of them so that it is `“Hello UMW!”` .
- For `HelloWorldGUI2.java` , you should change the name of the two `private static` classes.
    - `HelloWorldDisplay` – change to `UMWWorldDisplay`
    - `ButtonHandler` – change to `UMWButtonHandler`

Changing these will force you to study the code to see how these classes are used in the `main()` method.

Capitalizer Repeat

Learning Objective(s): Capitalizer Repeat

Capitalizer Repeat

Points: Complete this entire section for 5 points.

Use the `Capitalizer.java` file to accomplish the following. * First execute the existing the `Capitalizer.java` . * Change the initial string that is displayed to be a string of your choice. * There is not many lines of code in `Capitalizer.java` , but several lines are rather tricky. In particular, study the following code to make sure you understand what it is doing.

```
JPanel buttonPanel = new JPanel();
buttonPanel.add
  (
    new JButton
      (
        new AbstractAction("To Lower Case") {
            public void actionPerformed(ActionEvent e) {
                area.setText(area.getText().toLowerCase());
            }
        }
       )
   );
```

Move A Line Repeat

Learning Objective(s): Move A Line Repeat

Move A Line Repeat

Points: Complete this entire section for 10 points.

Use the `CGMoveALine.java` file to accomplish the following. * First execute the existing `CGMoveALine.java`. * Change the orientation of the line that is moved. For example, draw a horizontal line. * Study the code in `CGMoveALine.java` so that you understand it.

Netbeans GUI Builder

Learning Objective(s): Netbeans GUI Builder

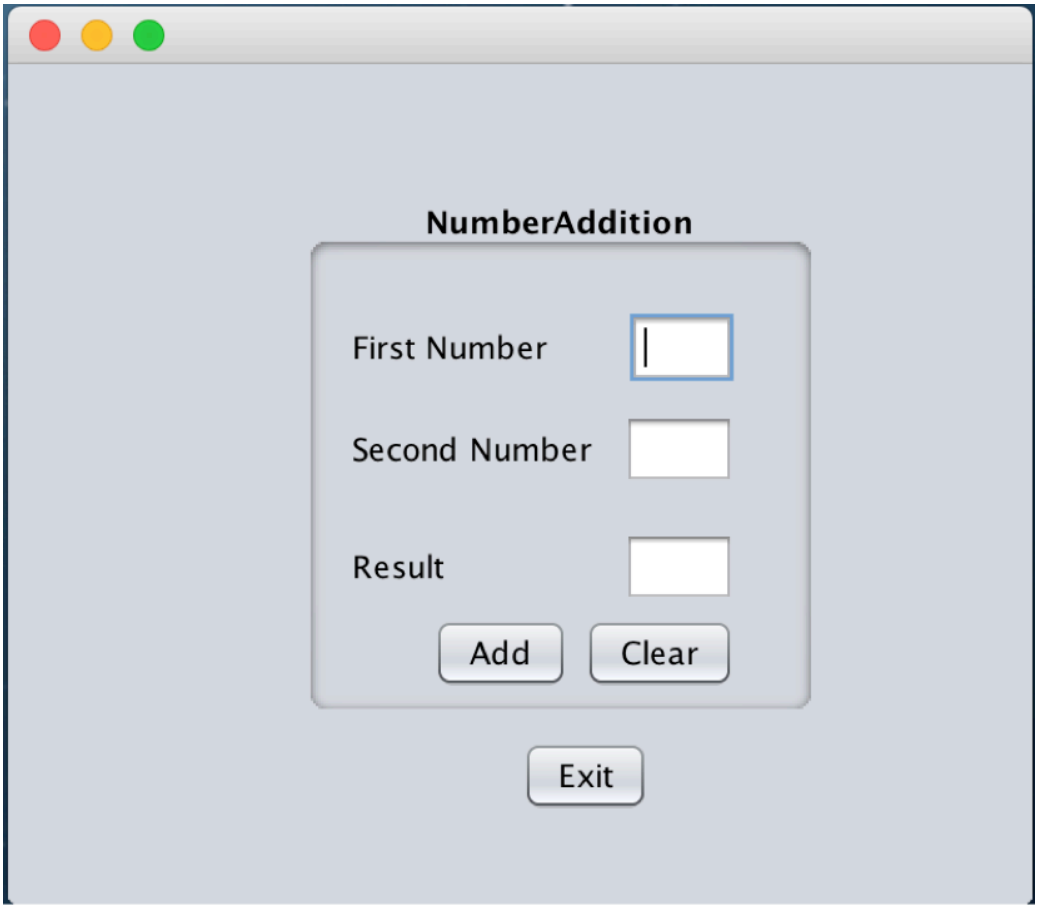
Netbeans GUI Builder

Points: Complete this entire section for 20 points.

This section contains my notes when using the NetBeans GUI builder. The following link is the Netbeans description of these steps. You may use either to complete this section.

Netbeans Description (https://netbeans.org/kb/docs/java/gui-functionality.html)`

This exercise creates a simple adder that looks like the following.



The Steps to creating this simple adder using the Netbeans GUI builder are the following.

1. Create a project. Similar to creating a regular project.
    - a. File > New Project, Java Application, Next,
    - b. NumberAddition is project name (place in desired folder)
    - c. Deselect Create Main Class checkbox
    - d. Finish
2. Create a JFrame container. This step creates a top-level JFrame in the Design Panel which you can drag and drop GUI components onto it to build the front-end of your GUI.
    - a. Right-click on NumberAddtion, select New > Other
    - b. In New File dialog box, chose Swing GUI Forms category and the JFrame Form type. Next.
    - c. Enter NumberAdditionUI as class name and my.numberaddition as package.
    - d. Finish
3. Using the Design Panel's Palette to drag and drop the GUI components on your front end. The Palette has several categories. This program uses two categories: Container and Controls.
    - a. Drop a Panel from the Container category. This is a JPanel that will be in your JFrame. The JPanel will hold all of the Control components except for the Exit button. Size the Panel appropriately. We add the Name Number addition to the Panel by the following.
        - i. Right-click, Properties, ellipsis button next to Border
        - ii. Select Titled Border
        - iii. Change the Title in this Panel to be Number Addition, OK
    - b. Drop 3 labels, 3 text fields, and 2 buttons on your Panel arranged as shown above. Drop another button on your JFrame (outside of your Panel) that will be the Exit button. At this point the control components have names like jLabel1, jTextField1, etc.
    - c. Edit the control components (double click slowly on them) so they have the correct names. First Number, Second Number, etc. You should make the text fields to be blank. Make sure the text

fields are editable. Right-click, Properties, edit text check box. I think they are editable by default.

- d. You can preview the look of your panel by clicking on the Preview button. Has an eye-ball in the icon.

4. Adding semantics to your panel so that it actually processes numbers and buttons. We do this by adding actions to our three buttons. The steps are similar for each button.
    - a. Exit Button – Right-click on Exit button, select Events > Action > actionPerformed. This takes you from the Design Panel to the Edit Panel, which is the Netbeans editor we have been using. You will be placed in the source code such that you are editing the method jButton3ActionPerformed, which is called with the Exit button is selected. The jButton3ActionPerformed method is an event handler that is called with the Exit button is selected.
        - i. Replace the // TODO comment with System.exit(0);
        - ii. Select Design (at the top of the edit window next to Source) to go back to the GUI designer.
    - b. Clear Button – Right-click on the Clear button, select Events > Action > actionPerformed just like you did for the exit button. You will edit the Clear button event handler, jButton2ActionPerformed.
        - i. Replace the //TODo comment with the following lines of code.

```
jTextField1.setText("");
jTextField2.setText("");
jTextField3.setText("");
```

        - ii. You should realize that the three JFields you dropped on the Panel are variables names jTextField1, jTextField2, and jTextField3. These variable reference JTextField objects so you are simply calling methods to put an empty string in them.
    - c. Add Button – Right-click on the Add button, select Events > Action > actionPerformed. You will edit the Add button event handler, jButton1ActionPerformed.

- i. Replace the //TODO comment with the following lines of code.
  ```
  java double num1, num2, result; num1 = Double.parseDouble(jTextFiel
  ```
- ii. You should realize this is basic programming.

5. Run your program. You may have to select my.NumberAddition.NumberAdditionUI as the main class.
6. Creating a stand-alone JAR file for your NumberAddition program.
   - a. Select Run > Clean and Build Main Project
   - b. This creates the file NumberAddition/dist/NumberAddition.jar file. You can double click on NumberAddition.jar to run your program.

EndorSmints

Learning Objective(s): EndorSmints

EndorSmints

Points: Complete this entire section for 20 points.

The goal for this section is to place a GUI on your Endor Smints project. If you followed the design presented in the Endor Smints project, creating a GUI is not too difficult. You have two choices.

1. Use the `EndorGui.java` class provided on Canvas. You simply pair this class with your `EndorSmints` class, and you should have a GUI running rather quickly. Study the `EndorGui.java` code so that you understand the design, how the design maps to the GUI, and how the design incorporates the EndorSmints. For example, you should study all of the components and how they are placed in `JPanels`, which are placed in the `JFrame`. The components are `JLabel`, `JButton`, `JComboBox`, `JCheckBox`. You should also understand how the Action Listener is added to the components, how it is called, and how it determines what component has been selected
2. Implement your own GUI. You can use the Netbeans GUI builder to drag and drop a GUI of your choice. If you do this, most of the remaining steps will not apply.

Recursion, Searching, Sorting Overview

Learning Objective(s): Recursion, Searching, Sorting

Recursion, Searching, Sorting

This lab has three separate topics - recursion, searching, and sorting.

Programming Contest

To receive maximum points on this lab, submit a solution to Programming Contest (page 0) Problem I - Stack Machine.

Submission Requirements

Follow guidance in Sample Lab Submission (page 0) for submitting your lab points.

Sample Lab

See previous labs for a sample lab specification and solution.

Questions

Points: To earn these 10 points, you have to write or type answers all questions; otherwise you receive 0 points.

1. Define Big-O in your own words.
2. Describe how a stack is used in method calls
3. Describe the Big-O of linear search, binary search, selection sort, bubble sort, and merge sort using your own words.
4. At this point in our class, do you consider yourself a programmer?

Recursive Power Method

Learning Objective(s): Recursive Power Method

Recursive Power Method

Points: Complete this entire section for 10 points.

Write an iterative and recursive method that computes xn. The iterative method will use a loop, and (of course) the recursive method will call itself. You should recognize the following, which should be helpful in designing your algorithms and implementing your Java code.

```
X3 is X * X * X
X4 is X * X * X * X, which is X * X3
```

The method signatures for your recursive and iterative methods can be

- `double powRecursive(double x, int n)` - computes and returns x^n
- `double powIterative(double x, int n)` - computes and returns x^n

Test your `powRecursive()` and `powIterative()` methods to make sure they work.

Update your `powRecursive()` so that it works for both positive and negative exponents. To do this, you should realize that the following equation is true.

$x^{-n} = 1/x^n$

The following code shows example calls.

```
double thousand = powRecursive(10.0, 3);   // 1_000
double point001 = powRecursive(10.0, -3);  // 0.001
```

Recursive Fibonacci Method

Learning Objective(s): Recursive Power Method

Recursive Fibonacci Method

Points: Complete this entire section for 10 points.

Write iterative and recursive methods that computes the nth Fibonacci number. A Fibonacci number is one from the sequence is 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, … The zeroth Fibonacci number is 0, the 1st Fibonacci number is 1, the 2nd is 1, the 3rd is 3, the 4th is 5, the 5th is 5, the 6th is 8, the 7th is 13, and so on. As you can see, the Fibonacci number f_n is computed by adding f_{n-2} and f_{n-1}. For example the 8th Fibonacci number is 13, which results by adding the 6th and 7th Fibonacci numbers, which are 5 and 8. It is easy to see that the recursive definition of Fibonacci is given by the following pseudo code.

```
function fibo(N) returns an integer
  if N is less or equal to 1
    return 1
  else
    return fibo(N-1) + fibo(N-2)
```

An alternative (but equivalent) algorithm uses the conditional operator.

```
function fibo(N) returns an integer
  return (N<2) ? N : fibo(N-1) + fibo(N-2)
```

These are correct algorithms, but they are inefficient and they have an exponential Big-O(2^n).

An iterative algorithm to compute Fibonacci numbers can be visualized by the following.

```
Iteration fibo(n)                X     Y
                                 0        1
         1        2                1        1
         2        3                1        2
         3        4                2        3
         4        5                3        5
         5        6                5        8
         6        7                8        13
         7        8                13        21
         8        9                21        34
         9        10                34        55
```

You need to initially assign x = 0 and y = 1. Then on each loop iteration, y is placed into x, and x+y is placed into y.

The method signatures for you recursive and iterative methods can be

```
int fiboRecursive(int n)
int fiboIterative(int n)
```

When implementing `fiboRecursive` , use a static integer that can count the number of calls.

After you have tested `fiboRecursive` and `fiboIterative` to ensure they are correct, perform the following evaluations of your algorithms.

- What value of the input parameter causes your fiboIterative to generate an incorrect answer?
- Why does this input parameter generate an incorrect answer? How does this relate to Lab 1?
- What value of the input parameter causes your fiboRecursive to generate a stack overflow exception (StackOverflowError)?
- Why does this input parameter generate a stack overflow exception? What is a stack overflow exception?
- How many calls to `fiboRecursive` result when the input parameter is 4, 5, 6? What is the relationship between the input parameter and the number of calls? Does the relationship support the fact that `fiboRecursive` is an $O(2^n)$ algorithm?

Recursive Thinking

Learning Objective(s): Recursive Thinking

Recursive Thinking

Points: Complete this entire section for 10 points.

Analyze the following code to determine what it computes. You are welcome to type the code into Netbeans and run it several times to see what it does.

```
public static int doesSomething(int a, int b) {
    if (b == 0)     return 0;
    if (b % 2 == 0) return doesSomething(a+a, b/2);
    return doesSomething(a+a, b/2) + a;
}
```

Update the above `doesSomething` method with the following changes to create a new method `doesSomething2` .

- Replace the + operator is replaced with a * operator.
- Replace return 0 with return 1.

What does `doesSomething2` compute?

Recursive Simple Maze

Learning Objective(s): Recursive Simple Maze

SimpleMaze

Points: Complete this entire section for 10 points.

The `SimpleMaze.java` file on Canvas under Files->Labs->Recursion->SimpleMaze demonstrates the power of recursion in solving mazes. Study the code and update the code to have a different maze. This will require you to create a new maze in the variable

```
static int [][] maze;
```

The following are some helpful hints concerning the code.

- The walls of the maze are the number 2 and paths of the maze on the number 1.
- The current code is created with final ints WIDTH and HEIGHT that define the size of the maze. If you create a larger maze, you will have to change these variables.
- You will also discover a method int[][] generateMaze() that is not implemented. You could create this function such that it generates a random maze. Then you could create loop that generated a maze and then solved it. This would be a simpler version of the fancy graphical maze contained in the file `Maze.java` on Canvas under Files->Labs->Recursion->Maze. Exploring this step is optional.

Recursive Array Printing

Learning Objective(s): Recursive Array Printing

Recursive Array Printing

Points: Complete this entire section for 10 points.

Write a recursive method

```
public static void printArray(int[] ia, int i).
```

The method will print the ith element of `ia` and then call itself to print the $i\text{-}1^{th}$ element. Your method shall ensure the formal parameter `i` is with the index range of `ia`. The following are examples of calling `printArray`.

```
printArray(new int[]{1,2,3,4,5,6},5);
```

Results in the following displayed in the console output.

```
6 5 4 3 2 1
```

```
printArray(new int[]{1,2,3,4,5,6},7);
```

Results in the following displayed in the console output.

```
Array index is invalid
```

Recursive ArrayList Printing

Learning Objective(s): Recursive ArrayList Printing

Recursive ArrayList Printing

Points: Complete this entire section for 10 points.

Write a recursive method

```
public static void printArrayList(ArrayList<Integer> ial)
```

The method will print the first element of the `ia` l, remove it, and then call itself to print the remaining elements. You should include error checking so that you do not try to print an empty `ArrayList` . The following are examples of calling `printArrayList` .

```
Integer[] ia = {1,2,3,4,5,6};  // Notice the type of ia
ArrayList<Integer> ial = new ArrayList<Integer>(Arrays.asList(i
a));
printArrayList(ial);
```

results in the following displayed in the console output.

```
1 2 3 4 5 6
```

You should also realized that at this point your `printArrayList` method has removed all elements of `ial` , which means the following statement prints `true` to the console output.

```
System.out.println(ial.isEmpty());
```

Number Searching

Learning Objective(s): Number Searching

Searching Through Numbers

Points: Complete this entire section for 10 points.

You should create the code for 12.1 if you need practice working with two-dimensional arrays and using file I/O. Working with two-dimensional arrays is information that you should know for the final.

For this lab, you write the following method.

`public static int[] twodArraySearch(int[][] ia2d, int ele)` - searches for `ele` in the two-dimensional array `ia2d`, and returns an integer array that (1) is `null` if `ele` is not in `ia2d` (2) contains two elements if `ele` is in `ia2d` where the first element is the row and the second element is the column . The following is an example call.

```
retIa = twodarraySearch(ia2d, 4);
```

1. If 4 is in ia2d, then retIa[0] is the row and retIa[1] is the column.
2. If 4 is not in ia2d, then retIa is null.

You can use whatever means you want to create two-dimensional arrays to pass to your `twodArraySearch` method. The following is one technique that reads information from a file.

Another way to practice with nested loops to fill an array is to write code that reads in a 2D grid of numbers into a two-dimensional array. The grid contains 5 rows and 10 columns. The grid will be in a file called `numbers.txt`, which you create. The following is an example.

```
18        100        98        57        89        79        5
5        85        29        24
34        87        82        83        16        33        1
1        90        58        50
95        4        70        63        30        52        4
2        40        15        44
8        53        43        97        59        54        5
6        75        76        81
1        23        86        69        36        66        6
8        17        14        28
```

You then ask the user which number they want to search for. Your program should search the two-dimensional array and report either the row and column that the number was found in, or that the number was not found, if it isn't in the grid.

- You should declare the array as a two-dimensional array of the needed size.
- Next, open up the `numbers.txt` file and read in the numbers into the array.
- Also, since you created the `numbers.txt` file to match your program, you know how many numbers are in the file, so there's no need to use "eof" - you can assume all the numbers can be read in before hitting the end!
- Ask the user to enter a number to search for.
- Write code to search for this number in the array. The algorithm you use should be like the linear search algorithm, with the addition of searching in two dimensions.
- If you find the item, print the row and column you found it on, otherwise a "not found" message.

Recursive Linear Search

Learning Objective(s): Recursive Linear Search

Recursive Linear Search

Points: Complete this entire section for 10 points.

Write a recursive linear search method

`public static int recursiveLinearSearch(int[] ia, int ele, int pos)` - The method searches for `ele` in position `pos` of `int[] ia`, returning `pos` if `ele` is in `ia[pos]`; otherwise, the method returns a call to itself that looks in the next position. Your method should return a -1 if the value is not found or `pos` in invalid. In fact, the recursion causes `pos` to become equal to `ia.length`, which returns -1 for `ele` not found. The following are examples of calling `recursiveLinearSearch`.

```
int val = recursiveLinearSearch(new int[]{1,2,3,10,5,6},10,0);
System.out.println(val);
```

Results in the following displayed in the console output.

```
3
```

```
int val = recursiveLinearSearch(new int[]{1,2,3,10,5,6},8,0);
System.out.println(val);
```

Results in the following displayed in the console output.

```
-1
```

Recursive Binary Search

Learning Objective(s): Recursive Binary Search

Recursive Binary Search

Points: Complete this entire section for 10 points.

This lab is designed so that you re-practice writing static methods. You place your binary search methods in a `BinarySearch` class that are called from a `BinarySearchTester` class. You can implement a binary search method that searches for `Strings` , `ints` , or `doubles` . This section describes a binary search method that searched `String` s.

When you create your Netbeans project, you should create your `main()` in a class named `BinarySearchTester` . If you recall this is the second step in creating a new project.

- 1. The Netbeans second panel – Name and Location:
    - a. Enter Project Name: BinarySearch.
    - b. Enter Project Folder:
    - c. You see a check in Create Main Class. Make sure you change this to be.

  `BinarySearchTester`

  - d. We continue to use the for now.
  - e. Finish
- 1. Now right-click on the icon and select New Java Class… This creates the New Java Class panel.
    - a. Change the Class Name: to be `BinarySearch`
    - b. Finish.

When editing your `BinarySearch` class, you should create two methods as shown during our class lecture.

```
private static int binarySearchString(String[] sa, String ele,
int first, int last) {
  // your code here
}
public static int binarySearchString(String[] sa, String ele) {
  return binarySearchString(sa, ele, 0, sa.length-1);
}
```

You should notice that both methods have the same name. The `private` method is called by your `public` method. This approach allows users to call your `binarySearchString` in a more natural way – with an array of `Strings` and a `String` element. Remember that the method `binarySearchString` only works with an array that is sorted in alphabetic order.

Since you have declared `binarySearchString` to be a `static` method, you do not need to create a `BinarySearch` object in your `BinarySearchTester` class. Instead, you call `binarySearchString` as follows.

```
String[] sa = { "Brandalee", "Cooper", "Dottie", "Ellie", "Emil
y",
                "Gusty", "Jeremy", "Jerri Anne", "Zachary" };
System.out.println(BinarySearch.binarySearchString(sa,"Gust
y"));
```

Notice how we use the `BinarySearch` class name as a prefix when making the call. This should remind you of calling `Math.random()` .

You should also call the Arrays.binarySearch method, which looks like the following.

```
System.out.println(Arrays.binarySearch(sa, "Zachary"));
```

Searching Through Strings

Learning Objective(s): Searching Through Strings

Searching Through Strings

Points: Complete this entire section for 10 points.

You are to create search algorithm that returns whether a word is a Scrabble word. There is a file with Scrabble words that you use to initialize a data structure. Since the words in the file are alphabetized, you can read the words into an array and perform a binary search on the words. Alternative, you can just perform a linear search through the array. You can use the Java `Arrays.binarySearch()` method if you like.

You program reads the words in the file `scrabbleWords.txt` to establish the data structure to be searched. Your program iteratively prompts a user for words and inform the user if their entered word is an allowable Scrabble word or not. An example run of your program may look as follows.

```
Welcome to the Scrabble word checker.
Enter a word or q to quit: aardvark
aardvark is a Scrabble word
Enter a word or q to quit: humongous
humongous is not a Scrabble word
Enter a word or q to quit: zyzzyva
zyzzyva is a Scrabble word
Enter a word or q to quit: q
Done
```

The design and implementation of your program is up to you.

Updating Hash.java

Learning Objective(s): Updating Hash.java

Updating Hash.java

Points: Complete this entire section for 10 points.

Canvas Files->Labs->Searching->Hash.java uses a recursive data structure to implement a key-value hash table. The key is a `String`, which is hashed into the table. When collisions occur, the recursive data structure is used to build a list of Strings that collide. The method hashValue computes a hash value by summing the values of each char in the String. This means that strings such as `"Hello"` and `"Heoll"` collide. Update the code so that the `hashValue` method uses the Java String method `hashCode` when computing the hash value. Realize the method still has to modulo the value so that it returns a value between 0 and 12.

wordCount with Map

Learning Objective(s): wordCount with Map

wordCount() with Map

Points: Complete this entire section for 10 points.

This lab updates the `Book` class from the Book Project to include a second method that counts words. The method `int wordCount(String word)` uses a Java Map. The updated constructor uses some nice code that removes punctuation from your ```bookString`. The purpose of this lab is to introduce you to alternative ways of solving the same problem. You should study the code.

You update your Book class so that it contains three private instance variables. Note that bookString already is part of your Book class.

```
private String bookString;
private String[] words;
private Map<String,Integer> counts;
```

You update your `Book` constructor such that it contains the following code.

```
public Book(String fileName) {
  readBook(fileName);
  words = bookString.replaceAll("\\p{Punct}+", "").toLowerCas
e().split("\\s");
  counts = new HashMap<String,Integer>();
  for (String w : words)
     counts.put(w, counts.getOrDefault(w,0) + 1);
}
```

You add the following `wordCount` method to your class.

```
public int wordCount(String word) {
     return counts.getOrDefault(word.toLowerCase(),0);
}
```

You should update your tester to call `wordCount` and `countWords`. You get different results. For example, wordCount ignores case so The and the are counted as the same word. Another example is the go-to is converted to the word goto.

Sorting Animation

Learning Objective(s): Sorting Animation

Sorting Animation

Points: To earn these 10 points, you have to explore some of these websites.

- Visualization of Sorting (https://www.cs.usfca.edu/~galles/visualization/ComparisonSort.html)
- Visualization of Sorting (https://www.youtube.com/user/AlgoRythmics)
- President Obama on Sorting (https://www.youtube.com/watch?v=k4RRi_ntQc8)
- Cowgirl Sorting (https://www.youtube.com/watch?v=B6BzDrQFRpk&feature=fvst)

Sort a Few Numbers

Learning Objective(s): Sorting a Few Numbers

Sort a Few Numbers

Points: Complete this entire section for 10 points.

Create two Java methods for sorting `int` s. The first method has 2 parameters and the second method has 3 parameters. By implementing these methods, you will appreciate sorting gets complicated as more numbers are sorted.

```
int[] sort(int i1, int i2)
int[] sort(int i1, int i2, int i3)
```

The following code snippet demonstrates using `sort` .

```
int[] sortedNumbers;

sortedNumbers = sort(3, 100, -1);
System.out.println(Arrays.toString(sortedNumbers));
// prints [-1, 3, 100]
```

The array `sortedNumbers` contains `{-1, 3, 100}` .

Test your `sort` methods to make sure they are correct.

Sort a Few Strings

Learning Objective(s): Sorting a Few Strings

Sort a Few Strings

Points: Complete this entire section for 10 points.

Create two Java method for sorting `String` s. The first method has 2 parameters and the second method has 3 parameters.

```
ArrayList<String> sort(String s1, String s2)
ArrayList<String> sort(String s1, String s2, String s3)
```

The following code snippet demonstrates using `sort` .

```
String a = "watermelon";
String b = "apple";
String c = "grape";
ArrayList<String> sortedStrings;

sortedStrings = sort(a, b, c);
System.out.println(sortedStrings.toString());
// prints ["apple", "grape", "watermelon"]
```

The `ArrayList sortedStrings` contains `{"apple", "grape", "watermelon"}` .

Test your `sort` methods to make sure they are correct.

Sort a Bunch of Strings

Learning Objective(s): Sort a Bunch of Strings

Sort an Arbitrary Number of Strings

Points: Complete this entire section for 10 points.

Canvas Files->Labs->Searching->Sort.java contains the sort methods we developed in class, which sort int[]. Update the sort methods in Sort.java to sort `ArrayList<String>` .

Create a Java program that iteratively interacts with a user. On each iteration, the user will enter a first name and Enter or just Enter. Use a `ArrayList<String>` to store the first names. When the user depresses Enter without a name, your program will sort the `ArrayList<String>` alphabetically and print the names alphabetically. You should sort this `ArrayList<String>` with each of the sort methods in your updated `Sort.java` file. The following is an example run of your program.

```
Welcome to the Name Sorter.
Enter name: Gusty
Enter name: Zachary
Enter name: Emily
Enter name: Coletta
Enter name:
Coletta
Emily
Gusty
Zachary
```

Recursive Number Conversion

Learning Objective(s): Recursive Number Conversion

Recursive Integer to Binary

Points: Complete this entire section for 10 points.

Write a recursive method `public static void intToBinary(int val)` that prints the binary equivalent of the `int` parameter `val`. Recall the converting from decimal to binary is done by using the modulo operator and the division operator. Consider the integer number 13, which is 0b1101. The following shows the two operations.

```
Step 1:
13 % 2 yields the digit 1 and 13 / 2 yields 6
Step 2:
 6 % 2 yields the digit 0 and  6 / 2 yields 3
Step 3:
 3 % 2 yields the digit 1 and  3 / 2 yields 1
Step 4:
 1 / 2 yields the digit 1 and  1 / 2 yields 0 so we stop.
```

The binary equivalent of 13 is 0b1101, which is the digits concatenated in reverse order of the steps.

The recursion stops when `val / 2` is 0. Each recursive call will generate a digit using `val % 2`. You have to arrange the recursive call and the print so the prints occur in reverse order.